

МИНОБРНАУКИ РОССИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«БЕЛГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНОЛОГИЧЕСКИЙ
УНИВЕРСИТЕТ им. В.Г. ШУХОВА»
(БГТУ им. В.Г. Шухова)

Институт информационных технологий и управляющих систем

Кафедра программного обеспечения вычислительной техники и автоматизированных систем

Направление подготовки 09.04.04 Программная инженерия

Направленность (профиль) образовательной программы Разработка программно-информационных систем

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

на тему:

Реализация алгоритма извлечения звука
музыкального инструмента из звукового потока
на основе методов искусственного интеллекта

Магистрантка: Барышникова Варвара Дмитриевна

Зав. кафедрой: канд. техн. наук, доц. Поляков Владимир Михайлович

Руководитель: канд. физ.-мат. наук, доц. Осипов Олег Васильевич

К защите допустить

Зав. кафедрой _____ /Поляков В.М./

« _____ » _____ 2026 г.

Белгород 2026 г.

МИНОБРНАУКИ РОССИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«БЕЛГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНОЛОГИЧЕСКИЙ
УНИВЕРСИТЕТ им. В.Г. ШУХОВА»
(БГТУ им. В.Г. Шухова)

Институт информационных технологий и управляющих систем

Кафедра программного обеспечения вычислительной техники и автоматизированных систем

Направление подготовки 09.04.04 Программная инженерия

Направленность (профиль) образовательной программы Разработка программно-информационных систем

Утверждаю:

Зав. кафедрой _____

« _____ » _____ 2026 г.

ЗАДАНИЕ

на выпускную квалификационную работу магистрантки

Барышниковой Варвары Дмитриевны

1. Вид выпускной квалификационной работы (ВКР): *магистерская диссертация.*
2. Тема ВКР *Реализация алгоритма извлечения звука музыкального инструмента из звукового потока на основе методов искусственного интеллекта* утверждена приказом по университету от «27» января 2026 г. №2/88.
3. Срок сдачи магистранткой законченной ВКР: «9» июня 2026 г.
4. Исходные данные: разделение музыкальных источников, выделение гитары из полифонических композиций, глубокое обучение, нейронные сети, архитектура Demucs, архитектура Open-Unmix, методы дообучения моделей, параметрически эффективная настройка, метод LoRA, спектральная аугментация, обработка аудиосигналов, спектральное перекрытие, метрики оценки качества разделения, фреймворк PyTorch, платформа Hugging Face.
5. Содержание ВКР:
 - Введение,
 - Анализ состояния исследования в области извлечения источников звука,
 - Реализация и проектирование архитектуры программного обеспечения,
 - Проведение вычислительных экспериментов,
 - Заключение,
 - Список литературы,
 - Приложение А,
 - Приложение Б.

6. Перечень графического материала: 18 рисунков, 3 таблицы. Презентация включает: титульный слайд, актуальность, цель и задачи, постановку задачи, обзор литературы и используемые технологии разработки, сравнение моделей извлечения аудиоисточников и их ограничения, схема системы извлечения источников, диаграмму классов (модули формирования набора данных, обучения моделей, оценки обучения моделей), диаграмму последовательности сбора набора данных, формирование выборок данных, реализация алгоритма дообучения модели Demucs методом LoRA (архитектурные изменения и контуры обучения), схему реализации дообучения модели OpenUnmix, графиков итогов проведения дообучения базовых моделей, графики с анализом значений метрик SDR, SIR, SAR, SI-SDR, таблицу с оценкой и анализом полученных результатов, итоги.

Дата выдачи задания: «28» января 2026 г.

(подпись руководителя)

Задание приняла к исполнению

(подпись магистрантки)

КАЛЕНДАРНЫЙ ПЛАН

№ п/п	Наименование этапов работы	Срок выполнения этапов работы	Примечание
1	Анализ предметной области задачи разделения музыкальных источников. Постановка цели и задач исследования.	02.02.26 – 15.02.26	Выполнено
2	Обзор существующих архитектур и методов разделения источников звука (Demucs, Open-Unmix). Выбор стратегий дообучения нейронных сетей.	16.02.26 – 22.03.26	Выполнено
3	Сбор и подготовка размеченного набора данных акустической гитары и пианино.	23.03.26 – 29.03.26	Выполнено
4	Разработка программного конвейера дообучения моделей Open-Unmix и Demucs.	30.03.26 – 29.04.26	Выполнено
5	Проведение серии экспериментов: оценка базовой модели htdemucs_6s, полное дообучение Open-Unmix, дообучение Demucs методом LoRA.	30.04.26 – 17.05.26	Выполнено
6	Анализ и визуализация полученных результатов. Построение графиков.	18.05.26 – 31.05.26	Выполнено
7	Оформление пояснительной записки. Подготовка выступления.	01.06.26 – 08.06.26	Выполнено

Магистрантка _____ Барышникова Варвара Дмитриевна
(подпись) (Ф.И.О.)

Руководитель _____ Осипов Олег Васильевич
(подпись) (Ф.И.О.)

Результаты проверки ЭВ ВКР на заимствование

Кафедра программного обеспечения вычислительной техники и автоматизированных систем

Магистрантка Барышникова В.Д.

(Ф.И.О.)

МПИ-241

(подпись)

(группа)

(дата)

Тема ВКР: Реализация алгоритма извлечения звука музыкального инструмента из звукового потока на основе методов искусственного интеллекта.

ВКР прошла проверку на объём заимствований.

Итоговая оценка заимствований:

Работу проверил

канд. физ.-мат. наук, доцент

(уч. степень, должность)

Хлопов А.М.

(подпись)

(дата)

(Ф.И.О.)

Руководитель ВКР

канд. физ.-мат. наук, доц.

(уч. степень, должность)

Осипов О.В.

(подпись)

(дата)

(Ф.И.О.)

ОПРЕДЕЛЕНИЯ, СОКРАЩЕНИЯ И ОБОЗНАЧЕНИЯ

SiSec 2018 (Signal Separation Evaluation Campaign) — кампания по оценке разделения сигналов 2018 года.

SiSec 2018 MUS — задача, которая решалась во время кампании SiSec 2018. Задача состояла в оценке систем, которые извлекают музыкальные инструменты из популярных музыкальных аудиопотоков, миксов музыкальных инструментов.

MUSDB18 — набор данных музыкальных композиций с разделенными источниками, который был составлен в рамках задачи SiSec 2018 MUS.

MSS (music source separation) — задача выделения источников из музыкального потока.

SOTA (state of art) — «состояние искусства», стандарт, признанный в исследовательской среде в рамках какой-либо исследовательской задачи.

MSST-WebUI (Music-Source-Separation-Training WebUI) — веб-интерфейс, который объединяет множество современных моделей разделения источников.

BSS Eval (Blind Source Separation Evaluation) — стандартный набор метрик для оценки качества разделения источников (Blind Source Separation Evaluation). Включает три основных показателя — SDR, SIR и SAR — которые совместно дают полную картину качества работы модели: общую чистоту сигнала, уровень подавления помех и отсутствие артефактов.

SDR (Signal to Distortion Ratio) — метрика, оценивающая общую чистоту выделенного сигнала. Учитывает все виды ошибок: помехи от других инструментов, артефакты обработки и шум. Чем выше, тем лучше качество разделения.

SIR (source to interference ratio) — метрика, оценивающая способность модели подавлять помехи от других источников. Показывает, насколько чисто выделен целевой инструмент без примесей остальных.

SAR (sources to artifacts ratio) — метрика, оценивающая отсутствие искусственных искажений в выделенном сигнале. Отражает, насколько естественно звучит результат без металлических призвуков, щелчков и размытости.

SI-SDR (Scale-Invariant SDR) — метрика, оценивающая качество сигнала независимо от его громкости. Аналог SDR, но не учитывает различия в абсолютной амплитуде, оценивая только форму волны.

STFT (Short-Time Fourier Transform) — оконное преобразованием Фурье

(ОПФ).

BLSTM (Bidirectional Long Short-Term Memory) — двунаправленная долговременная краткосрочная память, архитектура рекуррентной нейронной сети, в которой два независимых слоя с долговременной краткосрочной памяти обрабатывают входную последовательность одновременно в прямом (от прошлого к будущему) и обратном (от будущего к прошлому) направлениях, что позволяет модели учитывать как предыдущий, так и последующий контекст для каждого элемента последовательности.

PCM (Pulse Code Modulation) — импульсно-кодовая модуляция, метод цифрового представления аналогового сигнала, при котором непрерывный сигнал преобразуется в последовательность дискретных отсчётов.

CPU (Central Processing Unit) — центральный процессор.

GPU (Graphics Processing Unit) — графический процессор; графический вычислительный ускоритель.

PCI (Peripheral Component Interconnect) — это стандарт высокоскоростной шины, которая соединяет периферийные устройства (например, видеокарту) с центральным процессором и оперативной памятью на материнской плате.

ReLU (Rectified Linear Unit) — линейная функция активации в нейронных сетях, возвращающая входное значение для положительных аргументов и ноль для отрицательных.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	8
1 АНАЛИЗ СОСТОЯНИЯ ИССЛЕДОВАНИЯ В ОБЛАСТИ ИЗВЛЕЧЕНИЯ ИСТОЧНИКОВ ЗВУКА	11
1.1 Постановка задачи	12
1.2 Вклад кампании SiSEC MUS 2018 в задачу разделения источников	12
1.3 Способы представления звукового сигнала для задачи разделения источников	13
1.3.1 Временное представление	13
1.3.2 Частотно-временное представление	14
1.4 Обзор современных архитектур моделей разделения источников	16
1.4.1 Архитектура модели Open-Unmix	17
1.4.2 Архитектура модели Spleeter	23
1.4.3 Архитектура модели Demucs	24
1.5 Анализ методов дообучения предобученных моделей разделения источников	27
1.5.1 Полное дообучение	28
1.5.2 Методы «параметрического» дообучения (адаптация части параметров)	29
1.5.3 Адаптация низкого ранга (LoRA, Low Rank Adaptaion) .	30
1.6 Метрики оценки качества (SDR, SIR, SAR)	33
1.6.1 Декомпозиция сигнала по методу BSS Eval	34
1.6.2 Метрика SI-SDR, масштабно-инвариантное отношение сигнала к искажению	36
1.7 Выводы	37
2 РЕАЛИЗАЦИЯ И ПРОЕКТИРОВАНИЕ АРХИТЕКТУРЫ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ	39
2.1 Общая схема взаимодействия компонент разрабатываемой системы	39
2.2 Модуль сбора и подготовки данных	43
2.2.1 Загрузка исходных аудиозаписей. Клиент взаимодействия с сервисом Freesound	43
2.2.2 Синтез аудиомиксов из загруженных аудиофайлов и постобработка	46
2.2.3 Состав и количество выборок	47
2.2.4 Интеграция с платформой HuggingFace	50

2.3	Адаптация Open-Unmix для полного дообучения	52
2.3.1	Контур дообучения	52
2.3.2	Контур валидации	54
2.3.3	Спектральное маскирование	55
2.4	Дообучение модели Demucs методом LoRA	57
2.4.1	Интеграция LoRA в архитектуру Demucs v4	57
2.4.2	Контур дообучения	58
2.4.3	Валидационный контур	60
2.5	Модуль оценки дообученных моделей	60
2.5.1	Загрузка моделей для сравнительного анализа	61
2.5.2	Адаптеры тестовых данных	62
2.5.3	Процедура оценки	62
2.6	Выводы	63
3	ПРОВЕДЕНИЕ ВЫЧИСЛИТЕЛЬНЫХ ЭКСПЕРИМЕНТОВ	65
3.1	Инфраструктура и среда выполнения	65
3.2	Определение проблемы спектрального перекрытия	66
3.2.1	Физическое обоснование проблемы	66
3.2.2	Описание эксперимента	67
3.2.3	Количественные результаты и субъективная оценка	67
3.2.4	Статистическая значимость и выводы	68
3.3	Количественная оценка качества разделения	69
3.4	Анализ влияния спектральной аугментации	71
3.5	Сравнение вычислительных затрат	73
3.6	Субъективная оценка качества	73
3.7	Выводы	74
	ЗАКЛЮЧЕНИЕ	76
	СПИСОК ЛИТЕРАТУРЫ	77
	ПРИЛОЖЕНИЕ А Репозитории и материалы исследования	81
	ПРИЛОЖЕНИЕ Б Исходный программный текст модулей системы дообучения	82

ВВЕДЕНИЕ

Задача автоматического разделения музыкальных источников является одной из ключевых в области цифровой обработки сигналов и машинного обучения. Технологии и методы извлечения отдельных инструментов из сложных многоинструментальных записей находят применение в музыкальном продюсировании, реставрации архивных аудиоматериалов [1], а также лежат в основе систем интеллектуального анализа музыкальных композиций [2, 3].

Современные подходы к извлечению аудиоисточников из таких музыкальных записей опираются на архитектуры глубокого обучения, обрабатывающих сигнал либо в виде спектрограмм, либо в виде исходных временных отсчетов. Развитие этой области привело к появлению мощных универсальных моделей: благодаря обучению на крупных размеченных наборах данных, такие модели успешно и с высокой точностью выделяют стандартные классы инструментов (например, ударные, бас) [1].

Современные нейросети хорошо справляются с базовыми задачами разделения звука, но сталкиваются с серьезными трудностями при работе с гармоническими инструментами, такими как фортепиано и гитара. Их частотные диапазоны сильно пересекаются, что делает качественное разделение нетривиальной задачей. Тем не менее, потребность в точном выделении партий таких инструментов высока: это необходимо как для профессионального звукорежиссирования, так и для автоматического анализа музыки. На практике существующие универсальные модели имеют здесь два главных ограничения. Во-первых, при работе со схожими по тембру инструментами они часто создают неточные частотные маски. Это приводит к артефактам и «протеканию» одного инструмента в дорожку другого [4]. Во-вторых, чтобы настроить модель на работу с новыми, специфичным инструментом, её нужно либо переобучать целиком (что требует огромных вычислительных мощностей), либо использовать методы эффективной адаптации, которые в основном разрабатываются для текстовых и генеративных моделей, а не для аудио.

В настоящее время в научной литературе существуют ограниченное количество работ, в которых реализуется применение эффективного дообучения именно для нейросетевых моделей разделения аудиоисточников: к примеру, в рамках статьи [5] исследуется возможность и эффективность применения промтов к извлечению источников из аудиопотока. В работе [6]

впервые параметрически эффективный метод низкоранговой адаптации [7] был применён к модели извлечения источников AudioSep для изучения эффективного дообучения при небольшом количестве эпох и показал, что качество выделения значительно улучшилось после проведённого дообучения.

Успешное применение метода низкоранговой адаптации к задачам обработки речевых сигналов было произведено в рамках статьи [8]. Это позволяет также предположить эффективность метода и для задач разделения музыкальных источников, где также требуется адаптация общих признаков к специфичным классам.

Отсутствие исследований, посвященных применению методов параметрически эффективного дообучения (PEFT) к задачам разделения аудио, формирует явный научный пробел. Это обуславливает необходимость сравнительного анализа того, как полное обновление весов и эффективная адаптация влияют на качество и скорость работы современных архитектур извлечения источников.

Цель работы: повышение качества и эффективности извлечения музыкальных инструментов из звукового потока за счет применения методов дообучения специализированных моделей искусственного интеллекта, используемых для разделения аудиоисточников.

Для достижения цели были выделены следующие задачи:

- Проанализировать современные архитектуры извлечения аудиоисточников и определить ограничения в их работе.
- Сформировать специализированный размеченный набор данных, необходимый для дообучения базовых моделей и решения выявленных ограничений.
- Спроектировать и реализовать алгоритм дообучения базовых моделей разделения аудиоисточников.
- Провести серию вычислительных экспериментов, сравнить производительность и качество разделения дообученных моделей с использованием объективных метрики и субъективной оценки.

Объект исследования: нейросетевые архитектуры разделения музыкальных источников, работающие в частотной и временной области.

Предмет исследования: методы полного дообучения и параметрически эффективной адаптации моделей для выделения музыкальных инструментов.

Настоящая работа состоит из введения, трёх глав, заключения, списка

использованных источников и приложений. В первой главе представлен теоретический обзор архитектур разделения источников, методов адаптации нейронных сетей и метрик оценки качества. Во второй главе описаны проектирование экспериментального конвейера, формирование набора данных и программная реализация алгоритмов дообучения. Третья глава содержит результаты проведённых экспериментов, сравнительный анализ моделей и выводы по полученным данным.

1 АНАЛИЗ СОСТОЯНИЯ ИССЛЕДОВАНИЯ В ОБЛАСТИ ИЗВЛЕЧЕНИЯ ИСТОЧНИКОВ ЗВУКА

Выделение аудио-источников (Audio Source Separation) — процесс извлечения заранее определённого набора источников звука из смешанного аудиосигнала. Цель такой задачи — выделить конкретные источники, например вокал, инструменты или фоновый шум, из сложной аудиозаписи (рисунок 1.1).

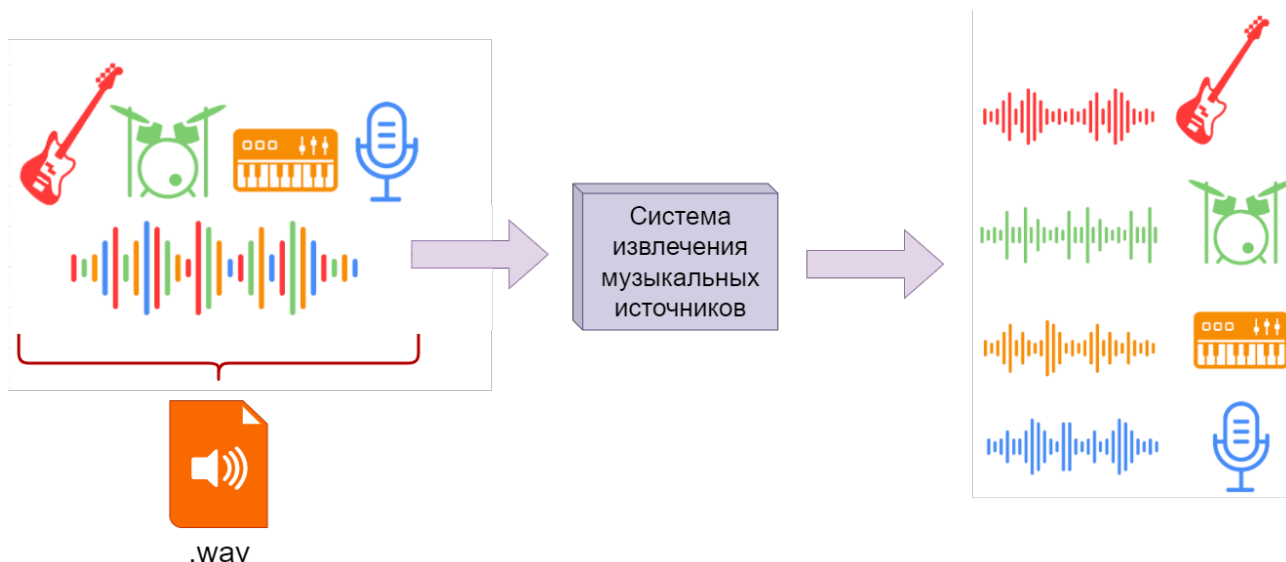


Рисунок 1.1 – Упрощенный процесс разделения источников из аудиозаписи.

Авторский материал

Для разделения источников используются алгоритмы обработки сигналов, которые используют различные техники: спектральный анализ, анализ во временной и частотной областях, машинное обучение.

Итогом выполнения задачи декомпозиции смешанного аудиосигнала являются выходные данные в виде компонент, представляющих собой изолированные аудиодорожки исходного источника (например, партию гитары, барабанов или вокала). Такая аудиодорожка является единицей музыкального аудиопотока в задаче выделения источников. Когда происходит запись в студии звукозаписи, например, музыкального трека, то происходит запись отдельных музыкальных инструментов и вокала и образуется финальная композиция. И цель задачи извлечения сигналов, в том, чтобы воссоздать эти индивидуальные дорожки из смешанного сигнала-музыкальной дорожки.

Притом, такая задача подразумевает получение из одного аудиотрека (аудиопотока) несколько источников (то есть задача сложнее, чем из потока

шума получить какой-то один источник – в статье [11] приводится пример появления задачи – потребность услышать и вычленив из общего шума общественного места выделить голос и речь своего собеседника).

Таким образом, необходимо провести обзор и анализ методов разделения музыкальных источников, являющимися стандартами в наше время, определить возможные методы дообучения таких моделей, а также метрики, которыми можно оценить проведённое дообучение.

1.1 Постановка задачи

Задача извлечения источников из музыкального сигнала формулируется следующим образом [12]: «Дан звуковой файл в формате .wav, содержащий запись нескольких музыкальных инструментов. Звуковой файл представляет собой оцифрованный аналоговый сигнал $x(t)$. Необходимо найти аудиосигналы $\hat{s}_i$, удовлетворяющие следующим условиям».

$$\begin{cases} x(t) = \sum_{i=1}^N s_i(t) + n(t), \\ \hat{s}_i(t) = \mathcal{F}(x(t), i) \approx s_i(t), \end{cases}$$

где

- $x(t)$ — исходный смешанный аудиосигнал;
- $s_i(t)$ — эталонный аудиосигнал i -го источника;
- N — количество источников в композиции;
- $n(t)$ — шумовой компонент (помехи, внешние источники);
- $\mathcal{F}(x(t), i)$ — оператор извлечения i -го источника, реализуемый нейросетевой архитектурой;
- $\hat{s}_i(t)$ — аудиосигнал i -го источника, полученный оператором извлечения источников.

1.2 Вклад кампании SiSEC MUS 2018 в задачу разделения источников

В статье, посвящённой реализации нейронной сети Open-Unmix [13] упоминается крупная международная кампания SiSEC 2018 [14, 15], в рамках которой были стандартизированы подходы к оценке качества разделения аудиоисточников. Среди основных задач данной кампании была задача извлечения из профессионально записаной музыки музыкальных инструментов по следующим общим категориям:

- барабаны (drums),
- бас (bass),
- вокал (vocals),
- другие музыкальные инструменты и звуки (other).

В рамках кампании SiSEC Mus 2018 сообществом исследователей был сформирован эталонный набор данных MUSDB18 [16], содержащий 150 музыкальных композиций с разделенными аудио источниками в сумме составляющие 10-часовой плейлист. Набор данных MUSDB18 всё ещё является самым большим бесплатным набором в открытом доступе и активно используется в задачах дообучения нейронных сетей, решающих задачу разделения источников [11] (при необходимости или скудности материалов этого набора данных исследователи прибегают к самостоятельному сбору специфичных наборов данных [17]). Но чаще всего обучение больших базовых моделей, решающих задачу извлечения аудиоисточников, происходит именно на этом наборе.

1.3 Способы представления звукового сигнала для задачи разделения источников

Разные нейросетевые методы разделения музыкальных источников работают с разным представлением звукового сигнала. От того, какой способ был выбран, зависит эффективность самого метода и его архитектура. В современной литературе по цифровой обработке сигналов и машинному обучению выделяют три основных класса представлений [18, 19]:

- временное (waveform),
- частотное (spectral),
- частотно-временное (time-frequency).

1.3.1 Временное представление

Наиболее фундаментальным способом представления звука является дискретизированный временной сигнал $x[n]$, где $n = 0, 1, \dots, N - 1$ — номер отсчёта, N — общая длина сигнала. Значение $x[n]$ соответствует амплитуде звукового давления в момент времени $\frac{n}{f_s}$ где f_s — частота дискретизации (обычно 44100 Гц или 48000 Гц для музыкального контента) [20].

С точки зрения физики временной сигнал отражает колебание давления воздуха, которые были зафиксированы микрофоном, записывающим аудиосигнал. Монофонический звуковой сигнал (моносигнал) представляет

собой последовательность вещественных чисел $x[n]$, где каждый отсчёт отражает мгновенное значение амплитуды. Для удобства обработки значения обычно нормализуются к диапазону $[-1, 1]$ или сохраняются в стандартном цифровом формате РСМ (импульсно-кодовая модуляция) с фиксированной разрядностью (например, 16 или 24 бита). Именно такой формат данных лежит в основе несжатых форматов (.wav, .aiff), а также служит промежуточным представлением при декодировании сжатых форматов (.mp3, .aac) перед их обработкой нейросетевыми алгоритмами.

В задачах разделения источников временное представление используется в архитектурах, работающих во временной области, таких как Conv-TasNet [21] и Demucs [1, 11], которые работают непосредственно с сырыми отсчётами, применяя одномерные свёртки для извлечения признаков.

1.3.2 Частотно-временное представление

Большинство современных методов разделения музыкальных источников работают во частотно-временной области, применяя оконное преобразование Фурье (Short-Time Fourier Transform, STFT) для перевода исходного аудиосигнала в спектральное представление. Характерным примером такой архитектуры является модель Open-Unmix [13].

Оконное преобразование Фурье (ОПФ) вычисляется путём разбиения дискретного сигнала $x[n]$ на перекрывающиеся сегменты (фреймы) с помощью скользящего окна. На каждом шаге локальный фрагмент сигнала умножается на оконную функцию $w[n]$, после чего к нему применяется дискретное преобразование Фурье. Для обеспечения наложения соседних сегментов сигнала окно анализа последовательно смещается вдоль него с шагом H , который строго меньше длины окна L [22]:

$$X[m, k] = \sum_{n=0}^{L-1} x[n + mH] \cdot w[n] \cdot e^{-j2\pi kn/L},$$

где

- $X[m, k] \in \mathbb{C}$ — комплексное значение ОПФ для m -го временного сегмента и k -го частотного отсчёта;
- L — длина окна, количество отсчётов в одном сегменте (обычно это 2048 или 4096 отсчётов);
- H — шаг смещения окна, определяющий степень перекрытия соседних участков (обычно это $H = L/4$ или $H = L/2$);
- $w[n]$ — оконная функция, применяемая к сегменту сигнала;

- $k = 0, 1, \dots, L/2$ — индекс частотной компоненты (соответствует физической частоте $f_k = k \cdot f_s/L$);
- $m = 0, 1, \dots, M - 1$ — индекс временного сегмента, где M — общее число фреймов;
- j — мнимая единица.

Физически оконное преобразование Фурье представляет собой переход от одномерного временного сигнала к двумерному частотно-временному представлению. Каждый комплексный коэффициент $X[m, k]$ характеризует амплитуду и фазу частотной компоненты f_k в конкретный момент времени $t_m = mH/f_s$.

Спектрограмма представляет собой двумерное представление сигнала, которое формируется на основе оконного преобразования Фурье (ОПФ) путём вычисления модуля или квадрата модуля его комплексных коэффициентов. В зависимости от цели анализа выделяют амплитудную и энергетическую спектрограммы:

$$S[m, k] = |X[m, k]| \quad (\text{амплитудная}),$$

$$P[m, k] = |X[m, k]|^2 \quad (\text{энергетическая}),$$

На практике для сжатия широкого диапазона аудиосигнала часто применяется логарифмическое масштабирование:

$$S_{\log}[m, k] = \log(1 + |X[m, k]|) \quad (\text{логарифмическая}).$$

Визуально спектрограмма представляет собой двумерное изображение (тепловая карта), где по оси абсцисс откладывается время, по оси ординат — частота, а яркость цвета на графике соответствует энергии соответствующей спектральной компоненты. Для музыкальных сигналов такая визуализация позволяет наглядно выделить гармоническую структуру инструментов, проявляющуюся в виде горизонтальных полос на частотах, кратных основной частоте тона. Пример спектрограммы музыкального сигнала приведён на рисунке 1.2.

В контексте задач разделения музыкальных источников спектрограмма служит в качестве основного входного представления для моделей, работающих в частотной области (например, Open-Unmix [13]). В рамках такого подхода нейросетевая архитектура обучается предсказывать матрицы

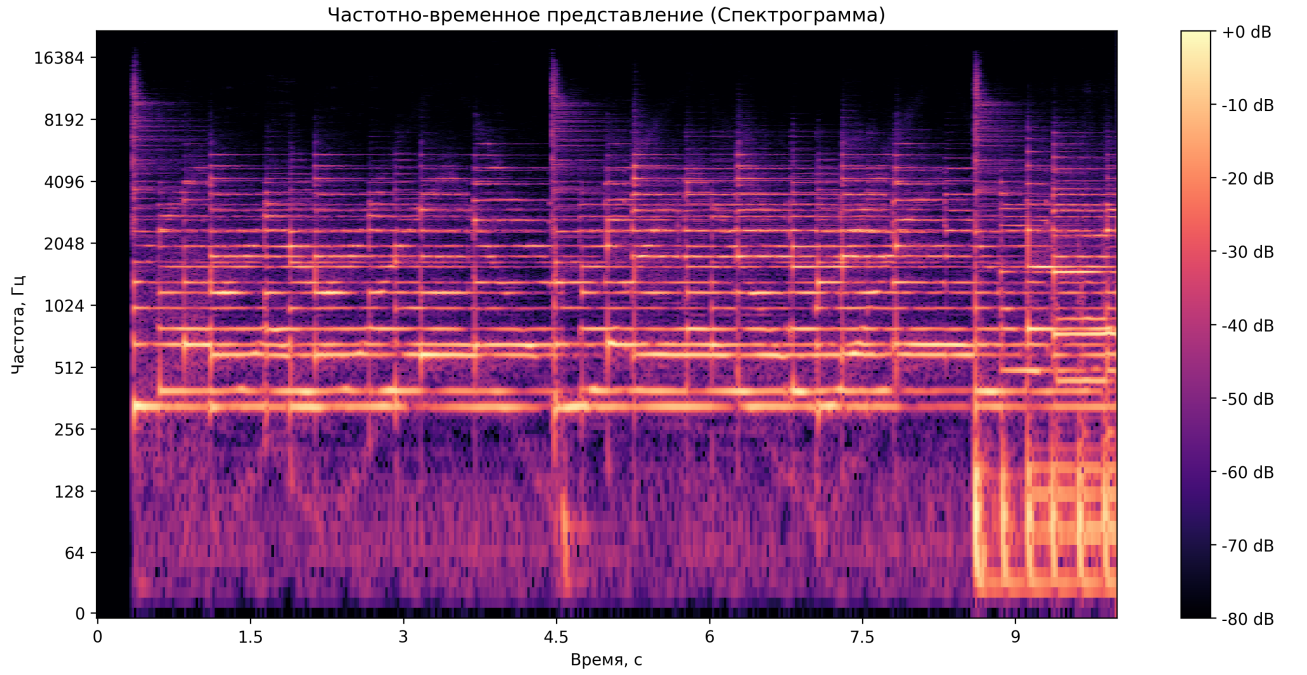


Рисунок 1.2 – График спектрограммы музыкальной композиции. Авторский материал

частотно-временных масок $M_c[m, k] \in [0, 1]$ для каждого источника c , которые затем применяются к исходному аудиосигналу для выделения отдельных компонент источников:

$$\hat{S}_c[m, k] = M_c[m, k] \cdot S_{\text{mix}}[m, k],$$

где

- S_{mix} — спектрограмма исходного музыкального файла;
- $\hat{S}_c[m, k]$ — оценка спектрограммы c -го источника.

Для восстановления временного сигнала из спектрограммы применяется обратное ОПФ (*iSTFT*, *inverse STFT*) (формула 1.1).

$$\hat{x}[n] = \frac{1}{L} \sum_{m=0}^{M-1} \left(\frac{1}{2\pi} \sum_{k=0}^{L-1} \hat{X}[m, k] \cdot e^{j2\pi kn/L} \right) \cdot w[n - mH], \quad (1.1)$$

где $\hat{X}[m, k]$ — комплексная спектрограмма с применённой маской (сохраняется фаза исходной композиции или восстанавливается итеративными методами).

1.4 Обзор современных архитектур моделей разделения источников

Современные архитектуры извлечения аудиоисточников можно условно разделить на два основных класса в зависимости от того, в какой области они обрабатывают аудиосигнал: во временной области (Demucs [1]) или в

частотной (Open-Unmix [13] и Spleeter [2]). Далее рассмотрим эти основные виды архитектур и сравним их способы реализации и применение.

1.4.1 Архитектура модели Open-Unmix

Архитектуры, работающие в частотной области (например, Open-Unmix [13] и Spleeter), основаны на предварительном преобразовании входного временного аудиосигнала в спектрограмму с использованием оконного преобразования Фурье. В таком представлении каждый элемент матрицы спектрограммы соответствует амплитуде (энергии) конкретной частотной компоненты в заданный момент времени.

В процессе работы нейросетевая модель анализирует эту спектрограмму и предсказывает частотно-временные маски для каждого заданного источника. Далее полученная маска поэлементно умножается на спектрограмму исходного аудиомикса, после чего к результату применяется обратное оконное преобразование Фурье (iSTFT) для получения итоговой аудиодорожки источника.

Детальная структура модели Open-Unmix, которая является на текущей момент эталонной реализацией спектрального подхода, представлена в работе [13]. Схема ее архитектуры приведена на рисунке 1.3.

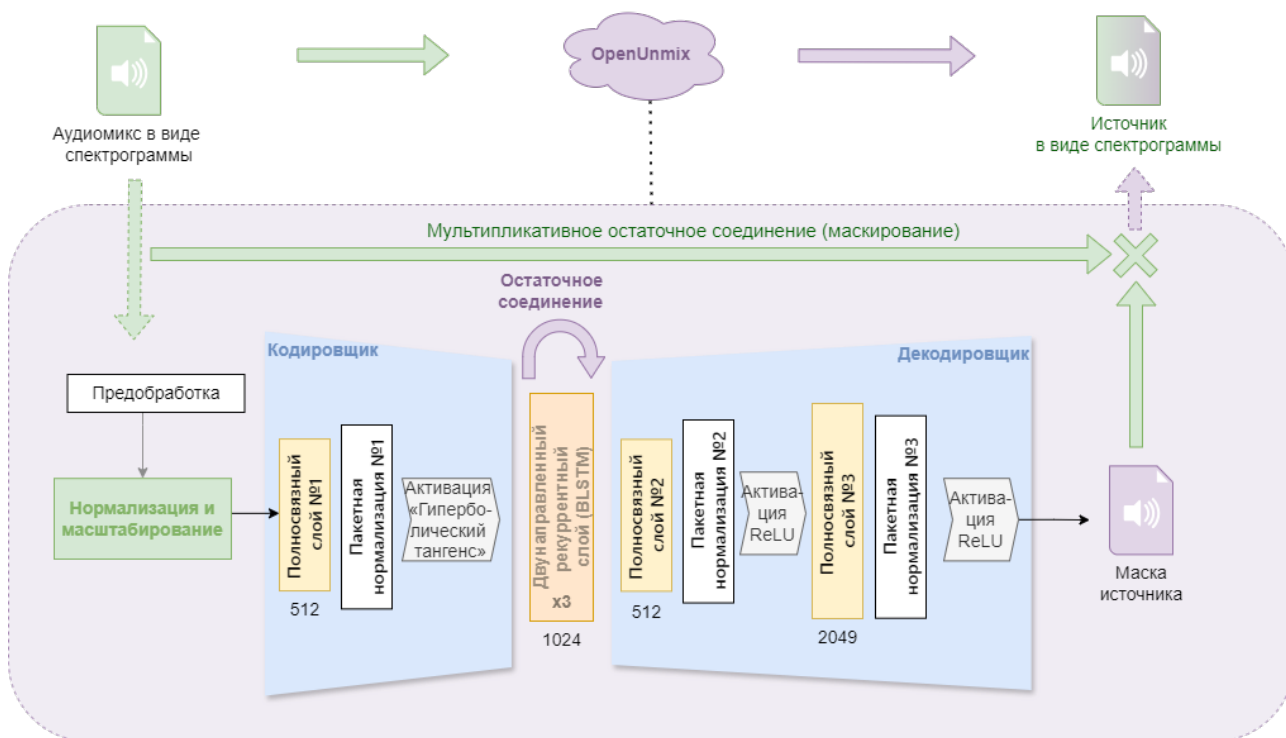


Рисунок 1.3 – Архитектура модели Open-Unmix. Авторский материал

Общая идея модели Open-Unmix заключается в преобразовании исходного файла аудиомикса музыкальных инструментов в формате

спектрограммы в набор спектрограмм отделенных музыкальных инструментов-источников из этого микса.

Модель Open-Unmix состоит из следующих компонентов:

- 1) Предобработка данных — так как предобученная модель Open-Unmix обучалась на наборе данных MUSDB18 [16], аудиозаписи которого подвергались AAC-сжатию, в результате которого диапазон спектрограмм обрезался до 16 кГц, в модели Open-Unmix на этапе предобработки входного сигнала выполняется «обрезка» спектрограммы по оси частот с исключением частотных ячеек, соответствующих более высоким частотам (больше 16 кГц).
- 2) Входная нормализация и масштабирование — блок обучаемых аффинных параметров. Он выполняет нормализацию входных спектрограмм для улучшения качества работы модели с помощью поэлементного сдвига и масштабирования входных спектрограмм, выравнивая распределение амплитуд по частотным каналам перед подачей на основные слои модели, что ускоряет сходимость градиентного спуска.
- 3) Блок кодирования признаков (кодировщик) — это совокупность механизмов и линейных слоёв внутри Open-Unmix, которая занимается сжатием входного аудиопотока в компактный набор признаков — то есть выполняет проекцию высокоразмерного частотно-канального представления в компактное скрытое (латентное) пространство. В отличие от классических кодировщиков, данный блок не стремится восстанавливать входной сигнал, а формирует признаки, оптимальные для последующего предсказания маски. Процесс кодирования включает:
 - Линейную проекцию (fc1, Fully Connected, первый полносвязный слой) — линейный слой без смещения, формирующий точку максимального сжатия данных («узкое горлышко»). Ограничение размерности на этом этапе принуждает модель отбрасывать избыточную спектральную информацию и шум, формируя наиболее плотное и информативное представление перед рекуррентной обработкой.
 - Пакетную нормализацию (bn1, Batch Normalization layer, первый слой пакетной нормализации) — это модуль, который выполняет стандартизацию выходных значений полносвязного слоя, обеспечивая согласованность признаков перед их подачей на нелинейное преобразование tanh.

- Функцию активации гиперболического тангенса ($\tanh$) — нелинейная функция активации, ограничивающая значения активации диапазоном $[-1, 1]$, что необходимо для устойчивой работы рекуррентных слоёв. Формула функции гиперболического тангенса имеет вид:

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} \quad (1.2)$$

где x — выход слоя нормализации.

- 4) Центральный рекуррентный модуль — представляет собой три последовательных рекуррентных слоя двунаправленной долговременной краткосрочной памяти (Bidirectional Long Short-Term Memory, BLSTM). Его главная задача — анализ временных зависимостей в сигнале. Такая многослойная архитектура обеспечивает формирование признаков по принципу иерархии: начальные слои фиксируют кратковременные локальные особенности звучания (езкие всплески амплитуды, атаки звуков), а последующие слои последовательно объединяют эти данные, выявляя долгосрочные музыкальные закономерности (мелодические фрагменты, гармонические последовательности).

Для предотвращения исчезновения градиентов и сохранения спектральной информации при глубокой обработке на промежуточных этапах применяется механизм *остаточного соединения* (*Skip Connection*). На схеме 1.3 он изображён фиолетовой стрелкой над блоком «Двунаправленный рекуррентный слой» и обозначает объединение входного тензора с результатом обработки блока BLSTM. Математически эта логика представлена в формуле 1.3:

$$h = x + \mathcal{F}_{\text{BLSTM}}(x), \quad (1.3)$$

где

- x — входное представление размера 512,
- $\mathcal{F}_{\text{BLSTM}}(\cdot)$ — преобразование, выполняемое стеком BLSTM-слоёв.

То есть данные из начала блока копируются в конец, минуя сложные вычисления. Это нужно, чтобы важная информация не потерялась при глубокой обработке. Такой приём обеспечивает прямой градиентный поток и ускоряет сходимость оптимизатора.

- 5) Блок декодирования (декодировщик) — отвечает за восстановление размерности тензора до исходных размеров для последующего

формирования маски. В отличие от классических декодеров, данный блок не обучается восстанавливать исходный сигнал, а предсказывает коэффициенты маски, которые при поэлементном умножении на входной спектр определяют целевой источник.

Архитектурно декодировщик зеркален кодировщику и включает:

- Два полносвязных слоя (fc2 и fc3) — слой fc2 проецирует объединённый вектор признаков из расширенного пространства обратно в компактное представление, а слой fc3 расширяет его до исходного числа частотно-канальных коэффициентов, формируя основу для последующего вычисления весовой маски.

- Два слоя пакетной нормализации (bn2 и bn3) — стабилизируют распределение сигналов внутри пакета на промежуточных этапах восстановления.

- Функцию активации ReLU — функция применяется после нормализации и на финальном выходе блока декодировщика. Формально функция определяется следующим образом:

$$\text{ReLU}(x) = \begin{cases} 0, & \text{если } x < 0, \\ x, & \text{если } x \geq 0, \end{cases}$$

Функция ReLU заменяет все отрицательные значения нулями. Это обеспечивает два важных свойства: во-первых, модель фокусируется только на значимых признаках, подавляя фоновый шум; во-вторых, гарантируется неотрицательность итоговой маски, что соответствует физической природе амплитудного спектра, потому что интенсивность частотных компонент не может принимать отрицательные значения.

6) Механизм мультипликативного маскирования (Multiplicative Skip-Connection) — механизм финального выделения целевого источника, при котором исходный спектр аудиомикса поэлементно умножается на предсказанную маску только на выходе сети. Такой подход предпочтительнее прямой генерации спектрограммы «с нуля», так как позволяет переиспользовать фазовую информацию из исходного сигнала, избегая артефактов реконструкции. Модель Open-Unmix вычисляет широкополосную маску, значение которой в каждом частотно-временном элементе отражает долю энергии целевого источника в общем аудиомиксе. Процесс итогового разделения

описывается формулой поэлементного умножения:

$$\hat{S} = X \odot \hat{M},$$

где

- $X \in R^{T \times F}$ — исходная спектрограмма аудиомикса (суммарное представление всех инструментов),
- $\hat{M} \in R^{T \times F}$ — предсказанная нейросетью маска (выход модуля декодирования после функции активации ReLU, обеспечивающей неотрицательность значений),
- $\hat{S}$ — результирующая спектрограмма целевого аудиоисточника.

Таким образом, архитектура Open-Unmix реализует сквозной конвейер частотно-временного разделения источников, в котором входная спектрограмма аудиомикса последовательно проходит через блок кодирования признаков, двунаправленную рекуррентную сеть для моделирования временных зависимостей и блок восстановления размерности, формирующий оценку маски для извлечения целевого сигнала. Финальное выделение изолированной дорожки осуществляется путём поэлементного умножения предсказанной маски на исходную спектрограмму, что обеспечивает восстановление целевого источника с сохранением фазовой информации и минимизацией артефактов, характерных для генеративных моделей.

Модель Open-Unmix обладает рядом сильных сторон, однако имеет и специфические ограничения, которые важно учитывать при выборе базовой модели. Среди преимуществ можно отметить:

- 1) Авторы модели Open-Unmix называют её эталонной реализацией [13] для задач разделения источников. Текст исходной программы, предобученные веса и документация находятся в открытом доступе, что обеспечивает полную воспроизводимость результатов и упрощает интеграцию в исследовательские и прикладные проекты.
- 2) Применение двунаправленной рекуррентной LSTM-сети позволяет анализировать сигнал как в прямом, так и в обратном направлении [28]. Это преимущество важно для разделения гармонических инструментов, где для корректного выделения аудиоисточника требуется понимание всей музыкальной композиции в целом.
- 3) По сравнению с современными тяжёлыми гибридными или генеративными архитектурами, Open-Unmix показывает конкурентноспособные значения метрик на эталонных наборах данных

при значительно меньших требованиях к вычислительным мощностям, что делает её удобной для экспериментов в условиях ограниченных ресурсов.

- 4) Чёткое разделение на блоки кодирования, рекуррентной обработки и декодирования позволяет адаптировать архитектуру под специфические задачи: замену слоёв BLSTM на трансформерные слои, добавление многозадачного обучения или расширение конвейера на новые классы источников.

Но модель Open-Unmix также имеет следующие ограничения:

- 1) Так как слои LSTM идут последовательно, это ограничивает возможности параллелизации при обучении и запуске модели. Обработка длинных аудиопоследовательностей требует значительного времени и памяти, что затрудняет применение модели в системах реального времени с жёсткими требованиями к низкой задержке.
- 2) Подход на основе идеальных масок, которым оперирует модель Open-Unmix, предполагает, что источники в аудиомиксах независимы и их можно суммировать. Когда инструменты занимают один частотный диапазон, модель вынуждена идти на компромисс при оценке весов. Это приводит к взаимному «перетеканию» аудиосигналов: в выделенной дорожке одного инструмента остаются слышимые фрагменты другого, так как сеть не может однозначно разделить общую энергию в этих частотах.
- 3) Поскольку модель предсказывает только громкость частот (амплитуду), а фазу просто копирует из исходного микса, это часто приводит к звуковым искажениям. Особенно это заметно там, где волны разных инструментов в миксе конфликтовали друг с другом, вызывая на выходе характерные «фазовые артефакты», «размытость звука» или «металлический» призыв.

Архитектура Open-Unmix представляет собой сбалансированное решение для задачи разделения музыкальных источников, сочетающее интерпретируемость, воспроизводимость и конкурентоспособное качество. Основные ограничения модели связаны с вычислительной сложностью рекуррентных блоков и допущениями, лежащими в основе подхода маскирования.

1.4.2 Архитектура модели Spleeter

Модель Spleeter — это система разделения музыкальных источников, разработанная компанией Deezer в 2019 году [2]. Модель основана на архитектуре U-Net [23], работающей в частотной области, и обучена на синтетически сгенерированном наборе данных из 25 тысяч композиций. В отличие от рекуррентных подходов, модель Spleeter использует полностью сверточную архитектуру типа U-Net, что обеспечивает высокую скорость обработки за счёт параллелизации вычислений.

Модель Spleeter стала одним из первых доступных промышленных решений для разделения звука. Она поддерживает выделение 2, 4 или 5 источников (вокал, ударные, бас, фортепиано и категория «остальное»), что обеспечило ей широкую популярность благодаря простоте использования.

Согласно официальной документации [29], архитектура Spleeter состоит из следующих ключевых блоков:

- 1) Подготовка входных данных. Исходный аудиосигнал $x(t)$ преобразуется в комплексную спектрограмму с помощью оконного преобразования Фурье с размером окна 4096 и шагом сдвига 1024.
- 2) Кодировщик. Его задача — проанализировать входную спектрограмму и постепенно выделить из неё самые важные особенности звука. Для этого используется цепочка сверточных блоков. Каждый такой блок последовательно выполняет три действия:
 - применяет свёртку 3×3 с шагом 2, чтобы сжимать данные и уменьшать их размерность;
 - проводит пакетную нормализацию;
 - использует функцию активации ReLU.
- 3) Блок сжатия — это центральная часть сети, где данные достигают минимальной размерности. Такое ограничение заставляет модель отбрасывать шум и второстепенные детали, фокусируясь исключительно на самых важных абстрактных характеристиках музыкального сигнала.
- 4) Декодировщик с остаточными связями. Этот блок постепенно восстанавливает исходную размерность спектрограммы с помощью транспонированных сверток, объединяя сжатые признаки с деталями более ранних слоев для повышения точности восстановления.
- 5) Генерация маски и восстановление сигнала. На выходе модель формирует весовую маску, которая поэлементно умножается на

исходную спектрограмму аудиомикса. Затем с помощью обратного оконного преобразования Фурье полученный спектр преобразуется обратно в готовый аудиосигнал целевого источника.

Модель Spleeter имеет следующие преимущества:

- 1) Высокая скорость выполнения запроса. Благодаря полностью сверточной архитектуре без рекуррентных связей, все вычисления могут быть распараллелены на графическом процессоре. Обработка трека происходит в 5–10 раз быстрее, чем в реальном времени, что делает модель пригодной для потоковых сервисов.
- 2) Эффективное сохранение деталей. Архитектура модели Spleeter, основанная на модели U-Net с механизмом skip-connections позволяет восстанавливать высокочастотные компоненты, которые часто теряются в архитектурах с сильным сжатием, обеспечивая «прозрачное» звучание.
- 3) Устойчивость к переобучению. Использование пакетной нормализации и умеренной глубины сети (по сравнению с Transformer-моделями) снижает риск переобучения на небольших наборах данных.

Также существуют значимые ограничения в работе модели Spleeter:

- 1) Частотные артефакты. Работа исключительно в магнитудной области с сохранением фазы из аудиомикса приводит к артефактам в областях сильного спектрального перекрытия источников.
- 2) Малая гибкость архитектуры. Фиксированная структура U-Net сложнее адаптируется под новые классы источников или многозадачное обучение по сравнению с модульными подходами (Open-Unmix, Demucs).
- 3) Модель Spleeter является устаревшей и не применяется в современных исследованиях в связи с тем, что больше не поддерживается разработчиками. У модели сохраняются зависимости от устаревших версий библиотек Python, что становится проблемой при исследовании.

1.4.3 Архитектура модели Demucs

В статье [1] разработчики социальной сети Facebook описывают новый подход к методу извлечения источников. Demucs (Hybrid Transformer Demucs) на данный момент является одним из стандартов индустрии (SOTA — state of the art) задачи извлечения источников.

Demucs [1] – это нейронная сеть глубокого обучения, которая была разработана для решения задачи выделения аудио-источников. В основе нейросети Demucs лежит свёрточная нейронная сеть UNet [23], которая

используется для семантической сегментации изображений. То есть Demucs использует подход сети Unet обработки изображений и применяет для обработки волн аудиосигнала по спектрограммам [1].

В статье музыкальная композиция сравнивается с коктейльной вечеринкой, где каждый голос, фоновая мелодия и шум – каждый этот элемент является стемом, и тогда задача выделения голоса собеседника, с которым ведется диалог на вечеринке, является как раз задачей разделения источников. Но задача выделения источников из музыкальной композиции отличается от задачи выделения голоса собеседника на в первую очередь тем, что в ней интересуется в этой задаче выделение не какого-то одного приоритетного источника звука из несвязанного фонового шума, а целый набор тонов и тембров в согласованном потоке.

Разработчики модели Demucs основываются на архитектуре Conv-Tasnet и адаптируют ее под извлечение источников, т.к. изначально модель Conv-Tasnet предназначалась для извлечения речи из аудиопотока [21].

Модель Demucs (в актуальных версиях v3/v4 [1]) представляет собой гибридную архитектуру, объединяющую свёрточные нейронные сети, трансформерные блоки и параллельную обработку аудиосигнала как во временной области (waveform domain), так и в частотной (спектрограмма на основе ОПФ). В отличие от традиционных методов, которые предсказывают только амплитудные маски, Demucs одновременно анализирует амплитуду и фазу сигнала. Это позволяет точнее восстанавливать временную структуру звука и избегать характерных искажений, возникающих при простом умножении спектра на маску. Архитектура модели Demucs представлена на рисунке 1.4.

Модель Demucs состоит из следующих основных компонент:

- 1) Гибридный кодировщик. Исходный аудиосигнал преобразуется в сжатое представление с помощью одномерных сверточных слоев. Особенность модели HT Demucs в том, что она анализирует сигнал одновременно и как звуковую волну, и как спектрограмму (в этом и заключается «гибридность» модели). Это позволяет объединить высокую точность во времени с пониманием частотной структуры моделью.
- 2) Блок разделения. На этом этапе используются слои трансформера. В отличие от рекуррентных сетей (применяемых, например, в Open-Unmix), механизм внимания позволяет модели анализировать всю композицию целиком, а не только короткие фрагменты. Это критически

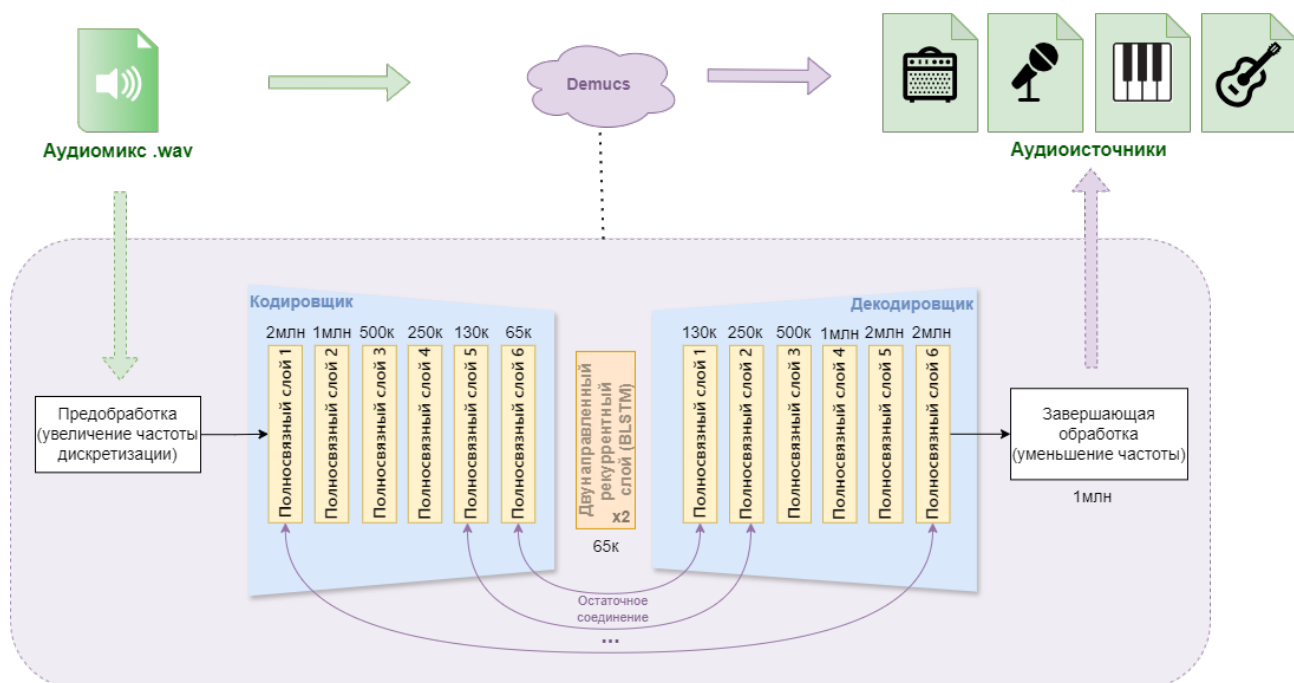


Рисунок 1.4 – Схема архитектуры модели Demucs. Авторский материал

важно для качественного разделения инструментов с долгим и сложным звучанием.

- 3) Гибридный декодировщик. Здесь сжатые данные преобразуются обратно в звуковой сигнал с помощью обратных сверточных слоев. Декодировщик также работает в двух режимах: он восстанавливает и временную волну (с помощью одномерных сверточных слоёв), и частотный спектр (с помощью двумерных свёрточных слоёв), а затем суммирует эти результаты для получения итогового звука.
- 4) Расчёт гибридной функции потерь. При обучении модель оценивает неточности одновременно и во временной, и в частотной области. Благодаря этому итоговый результат получается естественным на слух и корректным по своему спектральному составу.

Главное преимущество модели Demucs как модели, работающей непосредственно с исходной звуковой волной и обрабатывающей аудиосигнал в виде последовательности амплитудных значений (волн), заключается в сохранении полной информации о сигнале, включая его фазовые характеристики. Это критически важно для четкого и естественного воспроизведения резких звуков, таких как щипок струны гитары или удар по барабану, которые часто «размываются» при работе исключительно со спектрограммами.

Также к преимуществам модели Demucs относятся:

- 1) На эталонных наборах данных (например, MUSDB18) модель Demucs демонстрирует результаты лучше, чем рекуррентные архитектуры эталонных наборах данных (например, Open-Unmix), особенно по метрикам общего качества сигнала (SDR) и отсутствия посторонних шумов (SAR).
- 2) Благодаря механизму внимания модель анализирует не просто короткие отрезки звука, а целые музыкальные фразы. Это помогает ей гораздо эффективнее разделять инструменты со схожим тембром, опираясь на общую структуру композиции.
- 3) Поскольку модель оптимизируется одновременно и по временным, и по частотным характеристикам, итоговый результат получается естественным на слух и при этом математически точным по своему спектру.

Однако у модели Demucs также есть свои ограничения, которые создают препятствия при её использовании в исследовательских работах:

- 1) Из-за особенностей работы трансформера затраты ресурсов растут очень быстро при увеличении длины аудио. Это требует значительно больше видеопамяти и времени на обработку по сравнению с более легкими моделями.
- 2) Модель Demucs v4 содержит сотни миллионов параметров. Без специальных методов сжатия такую архитектуру практически невозможно запустить на устройствах с ограниченными ресурсами.
- 3) В случаях, если два инструмента играют одновременно и сильно перекрываются по частотам, модель иногда начинает «додумывать» звук, создавая фантомные шумы или искажения там, где их быть не должно.

1.5 Анализ методов дообучения предобученных моделей разделения источников

Существуют задачи, в рамках которых средств существующих предобученных нейронных сетей недостаточно и возникает потребность провести над выбранной нейронной сетью ряд действий, позволяющих «научить» её тому контексту, которому она не была «обучена» ранее, в процессе основного обучения, и которого не хватает для решения специфичной исследовательской задачи. При этом чаще всего такое обучение требуется провести без утраты тех «знаний», которым была обучена модель

первоначально, так как повторное переобучение модели с нуля — дорогостоящий процесс, требующий больших вычислительных мощностей, больших наборов данных, настройки всех параметров и т.д.

В таких случаях современные исследователи прибегают к подходу дообучения модели, в рамках которого модель не требуется переобучать заново, но необходимо провести её «тонкую настройку» (fine-Tuning) [31, 33] — то есть на уровне механики модели провести некоторые действия, чтобы научить нейронную сеть новым концептам с максимальным сохранением предыдущего накопленного опыта.

В случае задачи разделения источников в аудиосигнале обучение предобученной архитектуры модели с открытым исходным кодом на целевом наборе данных позволяет адаптировать модель к специфическим классам источников, о которых модель не «знала» и которые не умела извлекать до этого, без необходимости обучения «с нуля».

Метод «тонкой настройки» в настоящее время широко применяется на различных предобученных нейросетевых моделях, включая модели извлечения аудиоисточники. В зависимости от вычислительных ограничений и объёма данных для решения задачи дообучения применяется одна из трех основных стратегий: полная тонкая настройка, частичная настройка выходного блока и адаптация низкого ранга [32].

1.5.1 Полное дообучение

Полное дообучение представляет собой стратегию обновления всех весов предобученной модели на новом, целевом наборе данных (например, полное дообучение применялось в работе [34]). Все параметры предобученной модели размораживаются, и оптимизатор обновляет веса всех слоёв на новом наборе данных. Обучение проводится с пониженным коэффициентом обучения (learning rate), чтобы не разрушить уже усвоенные при первичном обучении спектрально-временных представлений, но позволить сети адаптироваться к новому распределению данных.

В контексте задач разделения музыкальных источников данный подход подразумевает дообучение всей архитектуры на целевом наборе данных, содержащем изолированные партии инструментов, отсутствующие в исходном распределении модели. На предварительном этапе конфигурация выходного слоя адаптируется под новое количество источников, после чего оптимизатор обновляет веса всех слоёв сети, обеспечивая полную адаптацию

внутренних спектрально-временных представлений к специфике целевых классов.

Основное преимущество данного подхода — максимальная гибкость адаптации модели к специфике задачи, в том числе к особенностям частотного диапазона и перекрытию гармоник между инструментами. Также к преимуществам можно отнести:

- 1) Простота реализации, метод не требует модификации архитектуры, чаще всего достаточно внести небольшое количество изменений.
- 2) При достаточном объёме наборов данных данный метод демонстрирует хорошее качество разделения источников [1, 35].

Несмотря на высокую эффективность, стратегия сопряжена с существенными вычислительными и методическими ограничениями. Полное обновление весов требует значительного объёма видеопамяти для хранения градиентов и состояний оптимизатора, что часто делает процесс недоступным в средах с ограниченным аппаратным бюджетом [7]. Кроме того, метод критически зависит от объёма и качества целевой выборки: при недостатке данных высок риск переобучения на специфичные акустические условия записей, а некорректно подобранные гиперпараметры могут привести к катастрофическому забыванию базовых признаков. В связи с этим полное дообучение целесообразно применять при наличии достаточных вычислительных ресурсов и сбалансированного набора данных.

1.5.2 Методы «параметрического» дообучения (адаптация части параметров)

Существует класс методов, которые отталкиваясь от концепции метода полного дообучения, дообучают не полностью всю модель, а только часть её весов: замораживают основную массу весов предобученной модели и обучают лишь малую добавочную часть параметров (обычно < 5% от общего числа). Такие методы называют «частичным дообучением» или «параметрически эффективная адаптация» [36].

Суть метода заключается в том, что все слои кодировщика и рекуррентного/сверточного блока, размораживается только выходной слой (декодировщик или классификационная «голова», выходной слой) [33]. Модель использует предобученные признаки общего звукового представления, а перестраивается только механизм соответствия признаков в целевые маски источников [33].

Использование методов параметрически эффективной адаптации (PEFT) имеет несколько явных преимуществ перед полным переобучением модели:

- 1) Поскольку в процессе обучения обновляется лишь небольшая часть весов сети, процесс проходит значительно быстрее и требует гораздо меньше видеопамяти [33].
- 2) Заморозка основных слоев сети защищает модель от «катастрофического забывания»: она не утрачивает общие акустические закономерности, усвоенные при первоначальном обучении на больших массивах данных [30, 33].
- 3) Низкие затраты на обучение позволяют исследователю быстро проверять различные гипотезы, менять состав обучающей выборки и подбирать гиперпараметры, не ожидая результата несколько дней [33].

Однако у такого подхода есть и свои ограничения, которые важно учитывать:

- 1) Если целевой инструмент звучит принципиально иначе, чем то, на чем училась базовая модель, замороженные слои не смогут полноценно подстроиться под новые спектральные особенности этого инструмента. В результате качество разделения достигнет предела своей эффективности. [30].
- 2) Модель, эффективно дообученная на чистых студийных записях, часто теряет качество при работе с более сложными материалами. Например, при обработке концертных записей или аудиотреков с артефактами сжатия ей не хватает внутренней гибкости для полной перестройки своих представлений, в отличие от модели, обученной с нуля непосредственно на таких данных [30].

1.5.3 Адаптация низкого ранга (LoRA, Low Rank Adaptaion)

Метод LoRA (Low Rank Adaptaion — дословно переводится как «адаптация на основе низкого ранга») был разработан для больших языковых генеративных моделей [7], но нашёл применение в большом пласте работ по дообучению моделей в машинном обучении, например, активно используется при дообучении генеративных моделей [37]. Этот метод — наиболее распространённая реализация парадигмы параметрически эффективного дообучения.

Основное отличие LoRA от методов полного дообучения и частичного дообучения заключается в том, что здесь адаптируется некоторое количество

параметров, результатом метода LoRA являются те веса, которые были преобразованы. Низкоранговые методы дообучения основаны на идее замены части весов матриц на низкоранговые приращения, которые обучаемы, а исходные веса при этом остаются неизменными. Подход LoRA схематично представлен на рисунке 1.5.

Суть подхода LoRA представлена в названии метода: исследователи при создании этого метода проверяли гипотезу, которая заключалась в том, что внутренняя размерность (ранг) матрицы весов $\Delta W = W_{fine_tuning} - W_0$ (где W_{fine_tuning} — веса модели после дообучения, W_0 — веса исходной модели) в больших моделях обладает низким рангом, несмотря на то, что исходная матрица весов W_0 имеет полный ранг. Это явление объясняется избыточной параметризацией современных нейросетей [38]: число обучаемых параметров значительно превышает минимально необходимое для решения задачи, что позволяет оптимизатору концентрировать полезные градиенты в низкоранговом подпространстве без потери обобщающей способности (то есть «предыдущих знаний»).

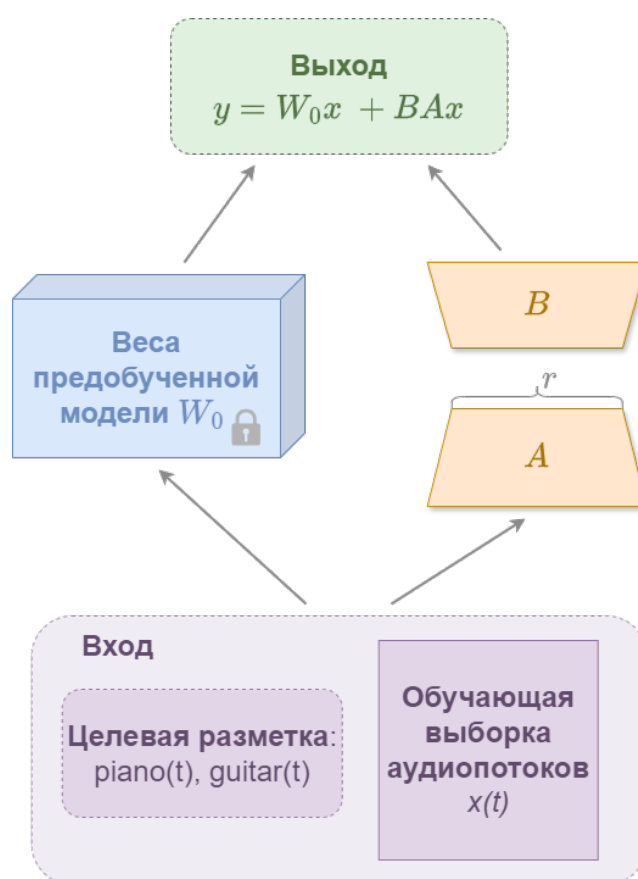


Рисунок 1.5 – Схема обучения модели подходом LoRA. Авторский материал

Такое предположение позволяет декомпозировать изменение весов в виде произведения двух матриц более низкого ранга, чем матрица весов (а не

оптимизировать саму матрицу весов изменяемого слоя):

$$\Delta W = BA,$$

где

- $A \in R^{r \times k}$ — матрица размерности $r \times k$,
- $B \in R^{d \times r}$ — матрица размерности $d \times r$,
- $r \ll \min(d, k)$ — ранг матрицы W , должен быть как можно меньше.

В процессе дообучения методом LoRA корректируются только матрицы A и B (считаются их градиенты и обновляются), что позволяет существенно снизить объём обучаемых параметров и требования к памяти GPU. Модель обучается лишь в низкоранговом подпространстве, которое описывается этими низкоранговыми матрицами, причём веса модели W_0 замораживаются и не преобразуются. Для получения итогового результата матрицы складываются:

$$y = W_0x + \Delta Wx = W_0x + BAx, \quad (1.4)$$

где

- A инициализируется Гауссовским распределением $A = N(0, \sigma^2)$,
- B инициализируется 0,
- W_0 — исходная матрица обучаемой модели.

Таким образом, LoRA заменяет полное обновление весов на обучение только компактных адаптеров, что сокращает количество обновляемых параметров на 70–91% [7], снижает потребление видеопамати и минимизирует риск катастрофического забывания исходных знаний модели, который часто сопутствует методам полного дообучения. В задаче выделения аудио источников, метод LoRA может применяться к отдельным слоям или компонентам модели (например, слоям свёртки или частотно-временным модулям), что позволяет выполнять адаптацию параметров без изменения архитектуры исходной сети.

Среди преимуществ метода разработчики LoRA приводят следующие [7]:

- 1) Адаптеры LoRA небольшого размера, в связи с чем требования к размеру ресурсов для их расположения достаточно снижены.
- 2) Поскольку градиенты вычисляются только для небольшого числа новых параметров, нагрузка на видеокарту резко снижается. Это позволяет дообучать крупные нейронные сети даже на бесплатных тарифах облачных сервисов (например, Google Colab).

- 3) Веса адаптеров А и В сохраняются отдельно от основной модели. Их можно быстро загружать, заменять или комбинировать без внесения изменений в архитектуру исходной модели. В рамках задачи извлечения источников, это позволяет поддерживать библиотеку адаптеров для различных инструментов и комбинировать их при запуске модели.
- 4) Поскольку основные веса модели W_0 остаются замороженными, она не теряет общие закономерности, которые усвоила при первоначальном обучении на больших массивах данных.

Однако у метода есть и свои ограничения, которые важно учитывать при выборе стратегии дообучения:

- 1) Необходимость подбора ранга r . Значение ранга r является гиперпараметром, требующим эмпирической настройки. Если выбрать слишком маленькое значение, модель не сможет достаточно подстроиться под новую задачу. Если слишком большое — пропадёт главное преимущество метода, заключающееся в виде экономии ресурсов, и он станет почти сопоставим с полным дообучением.
- 2) Метод LoRA исходит из того, что для адаптации к новой задаче нужны лишь небольшие корректировки весов. Если новые данные сильно отличаются от тех, на которых модель училась изначально (например, переход от чистых студийных треков к зашумлённым концертным записям), этих корректировок может не хватить. В таких ситуациях полное переобучение часто показывает лучшие результаты и быть более эффективным решением.

1.6 Метрики оценки качества (SDR, SIR, SAR)

В основных исследованиях, направленных на изучение дообучения моделей разделения источников, полученные результаты оцениваются при помощи метрик, которые являются стандартами в части оценки [4].

Чтобы объективно сравнить, насколько хорошо модели разделяют звук, недостаточно субъективной оценки. В этой области приняты математические стандарты, которые сравнивают предсказанный моделью сигнал аудиисточника $\hat{s}_k$ с идеальным эталоном s_k [4].

Главная идея этих метрик заключается в том, чтобы разделить общую ошибку на независимые составляющие. Оценивается не просто общая разница в сигналах, а определяется, что именно пошло не так: остались ли в выделенной дорожке источника звуки других инструментов (интерференция)

или добавил ли алгоритм свои посторонние шумы и искажения (артефакты). Такой подход позволяет независимо оценить два ключевых аспекта: насколько чисто и естественно выделен целевой инструмент и насколько эффективно подавлены все остальные.

В данной работе применяются классические метрики семейства BSS Eval (Blind Source Separation Evaluation) [4], а также их современная модификация SI-SDR. Выбор конкретных показателей для каждой модели опирается на стандарты, заданные в оригинальных статьях этих архитектур:

- для Open-Unmix [13] ключевым показателем традиционно выступает общий SDR;
- для Demucs [1] и Spleeter [2] принято оценивать полный набор характеристик: SDR, SIR и SAR.

Следование этим устоявшимся стандартам оценки гарантирует, что оценка будет методологически корректна, а полученные результаты можно будет честно сопоставить с эталонными показателями.

1.6.1 Декомпозиция сигнала по методу BSS Eval

Согласно методологии оценки сигнала BSS Eval [4], восстановленный сигнал аудиисточника $\hat{s}$ может быть представлен в виде суммы независимых друг от друга составляющих в соответствии с формулов 1.5:

$$\hat{s} = s_{\text{target}} + e_{\text{interf}} + e_{\text{noise}} + e_{\text{artif}}, \quad (1.5)$$

где

- $\hat{s}$ — сигнал, полученный алгоритмом извлечения аудиисточников;
- s — эталонный сигнал исходного аудиисточника;
- s_{target} — компонента, представляющая собой проекцию сигнала $\hat{s}$ на пространство истинного сигнала s ;
- e_{interf} — компонента интерференции (другие посторонние источники, не относящиеся к источнику s);
- e_{noise} — компонента шума (фоновый шум, шум оборудования, не связанный с источниками);
- e_{artif} — компонента артефактов (искажения, внесённые алгоритмом).

Поскольку в стандартных музыкальных датасетах (например, MUSDB18) нет отдельной дорожки с фоновым шумом, компонентой e_{noise} пренебрегают и её объединяют с артефактами e_{artif} . Оба этих слагаемых представляют собой искажения, не относящиеся ни к целевому инструменту, ни к интерференции. В

популярной библиотеке `mir_eval` шум также не выделяется отдельно, если для него не задан строгий эталон. Таким образом, формула декомпозиции сигнала упрощается до следующей:

$$\hat{s} \approx s_{\text{target}} + e_{\text{interf}} + e_{\text{artif}}.$$

На основе этого разложения вычисляются три основные метрики (в децибелах, дБ), каждая из которых отвечает за свой аспект качества:

- SDR показывает общее качество разделения, учитывая все типы ошибок сразу.
- SIR демонстрирует, насколько эффективно модель подавила посторонние инструменты.
- SAR оценивает, насколько чистым получился звук без посторонних шумов, внесенных самим алгоритмом.

Метрика *SDR* (*Signal-to-Distortion Ratio*, отношение сигнала к искажению) оценивает общее отношение энергии оцениваемого сигнала s_{target} к суммарной энергии всех ошибок, возникающих в процессе разделения. (например, наличие артефактов и наложения на источник других аудиоисточников).

Формула 1.6 описывает математическое определение метрики SDR:

$$\text{SDR} = 10 \log_{10} \left(\frac{\|s_{\text{target}}\|_2}{\|e_{\text{interf}} + e_{\text{artif}}\|_2} \right), \quad (1.6)$$

где $\|\cdot\|_2$ — квадрат Евклидовой нормы вектора дискретного сигнала.

Чем выше значение метрики SDR, тем лучше качество полученного источника. Значение 0 дБ означает что энергия полезного сигнала равна энергии ошибок. Положительные значения указывают на преобладание полезного сигнала. Увеличение значения метрики SDR на 10 дБ соответствует десятикратному улучшению отношения сигнал к ошибке. В исследованиях по разделению аудиоисточников [1, 2, 13] метрика SDR служит критерием при сравнении качества моделей.

Метрика *SIR* (*Signal-to-Interference Ratio*, отношение сигнала к интерференции) измеряет способность модели подавлять другие источники в исходном аудиомиксе. Эта метрика игнорирует артефакты и шум, фокусируясь исключительно на «чистоте» разделения: насколько хорошо алгоритм отделил, например, вокал от аккомпанемента. Формула 1.7 описывает математическое определение метрики SIR:

$$SIR = 10 \log_{10} \left(\frac{\|s_{\text{target}}\|_2}{\|e_{\text{interf}}\|_2} \right). \quad (1.7)$$

Высокие значения SIR (например, >10–15 дБ) означают, что в выделенной дорожке практически не слышно других инструментов и это свидетельствуют о качественном разделении источников. Однако высокий SIR не гарантирует высокого субъективного качества: сигнал может быть «чистым» от других инструментов, но при этом содержать сильные артефакты обработки (эффект «под водой», фазовые искажения), которые метрика SIR не учитывает, но учитывают другие метрики.

Метрика *SAR* (*Signal-to-Artifacts Ratio*, *отношение сигнала к артефактам*) оценивает естественность звучания выделенного источника, абстрагируясь от наличия других инструментов. Метрика чувствительна именно тем к искажениям, которые вносит метод разделения: эхо, фазовые сдвиги, «музыкальный шум». Формула 1.8 описывает математическое определение метрики SAR:

$$SAR = 10 \log_{10} \left(\frac{\|s_{\text{target}} + e_{\text{interf}}\|_2}{\|e_{\text{artif}}\|_2} \right). \quad (1.8)$$

Низкие значения метрики SAR часто указывают на наличие слышимых артефактов, даже если значение общей метрики SDR высокое. Для задач, где важно естественное звучание (например, реставрация или улучшение качества), SAR является критически важным показателем. Для моделей, работающих со спектрограммами (Open-Unmix [13], Spleeter [2]), низкое значение SAR часто указывает на ошибки, возникающие при некорректном применении частотной маски.

1.6.2 Метрика SI-SDR, масштабно-инвариантное отношение сигнала к искажению

Классические метрики чувствительны к громкости сигнала: например, если модель выдаёт правильный по форме, но более тихий сигнал, значение метрики SDR будет занижено.

Метрика *SI-SDR* (*Scale-Invariant SDR*, *масштабно-инвариантное отношение сигнала к искажению*) устраняет эту зависимость: она предварительно нормализует предсказанный сигнал по громкости относительно эталона. Таким образом, метрика оценивает только сходство формы сигнала, игнорируя разницу в общей амплитуде.

Пусть s — эталонный сигнал, $\hat{s}$ — оцениваемый предсказанный сигнал. Определим проекцию оценки $\hat{s}$ на направление эталонного сигнала s :

$$\alpha = \frac{\langle \hat{s}, s \rangle}{\|s\|_2}, \quad s_{\text{proj}} = \alpha s,$$

где

- $\langle \cdot, \cdot \rangle$ — скалярное произведение,
- α — оптимальный коэффициент масштабирования.

Тогда итоговая метрика вычисляется следующим образом:

$$\text{SI-SDR} = 10 \log_{10} \left(\frac{\|s_{\text{proj}}\|_2}{\|s_{\text{proj}} - \hat{s}\|_2} \right).$$

Эта метрика особенно полезна для моделей временной области (например, Demucs или Conv-TasNet [1]), где выходная амплитуда может варьироваться в процессе обучения. Кроме того, метрика SI-SDR является дифференцируемой функцией, что позволяет использовать её непосредственно в качестве функции потерь при обучении нейронных сетей.

1.7 Выводы

В данной главе были рассмотрены теоретические основы задачи разделения музыкальных источников. Были рассмотрены основные способы представления аудиосигнала (временной и частотный), а также математическое определение оконного преобразования Фурье, лежащее в основе формирования спектрограмм. Также были рассмотрены принципы работы ключевых современных нейросетевых архитектур, выявлены их особенности и ограничения. Был рассмотрен математический аппарат для объективной оценки их качества таких архитектур. Изучены стратегии дообучения предобученных нейросетевых моделей, сравнены подходы полного дообучения и параметрически эффективной настройки.

Проведённый в главе анализ позволил сделать выбор моделей и стратегий дообучения для дальнейшего исследования:

- 1) В качестве базовых моделей выбраны Open-Unmix и Demucs. Они имеют открытый исходный код, модульную структуру и хорошо зарекомендовали себя в задачах разделения гармонических инструментов. Модель Spleeter исключена из рассмотрения из-за устаревшей архитектуры и проблем с совместимостью современных библиотек.

2) На основе изученных стратегий сформирован план проведения дообучения: к трансформерной архитектуре Demucs будет применен метод эффективной настройки низкого ранга, а для рекуррентной модели Open-Unmix выбрано полное обновление параметров, что обусловлено её архитектурными ограничениями.

Таким образом, теоретическая база полностью сформирована, что позволяет перейти к экспериментальному этапу исследования.

2 РЕАЛИЗАЦИЯ И ПРОЕКТИРОВАНИЕ АРХИТЕКТУРЫ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

На основе теоретического анализа первой главы были сформированы требования к архитектуре программной системы дообучения моделей извлечения источников. Данная глава посвящена проектированию и реализации конвейера дообучения, поддерживающего различные стратегии оптимизации для соответствующих архитектур в соответствии с их архитектурными и вычислительными особенностями. Основным решением стало применение к гибридной трансформерной архитектуре модели Demucs параметрически эффективной настройки LoRA, а для рекуррентной модели Open-Unmix, работающей в спектральной области, реализовать полное обновление параметров сети. Выбор обусловлен различиями в структуре моделей.

Метод LoRA эффективно работает со слоями внимания и линейными проекциями в архитектуре Demucs, позволяя адаптировать крупную сеть при минимальном количестве обучаемых параметров. В случае Open-Unmix основу вычислений составляют рекуррентные блоки BLSTM. Прямое применение низко-ранговых адаптеров к рекуррентным слоям требует сложной модификации алгоритма обратного распространения ошибки и не гарантирует стабильной сходимости на малых объёмах данных. Поэтому полное дообучение является наиболее надёжным и воспроизводимым вариантом для корректной настройки временных зависимостей под специфику целевого набора данных.

В последующих разделах главы подробно описывается структура модулей системы дообучения: подготовки и обработки данных, алгоритмы интеграции LoRA-адаптеров в слои трансформеров модели Demucs, конфигурация алгоритмов полного дообучения рекуррентной сети Open-Unmix, а также механизмы обеспечения стабильности обучения, логирования и сохранения обученных моделей.

2.1 Общая схема взаимодействия компонент разрабатываемой системы

Реализованная система дообучения спроектирована по модульному принципу с чётким разделением ответственности между компонентами подготовки данных, адаптации архитектуры, оптимизации и оценки качества. Архитектура системы включает три основные внутренние подсистемы:

- модуль работы с набором данных,
- модуль адаптации и запуска моделей,
- модуль проведения экспериментов и оценки результатов,
- модуль интеграции с внешними платформами для обеспечения воспроизводимости всего процесса эксперимента — от загрузки сырых данных до публикации готовых дообученных моделей.

Схема взаимодействия модулей представлена на рисунке 2.1. На схеме для каждого модуля приведен набор содержащихся внутри него классов и взаимодействие их между собой.

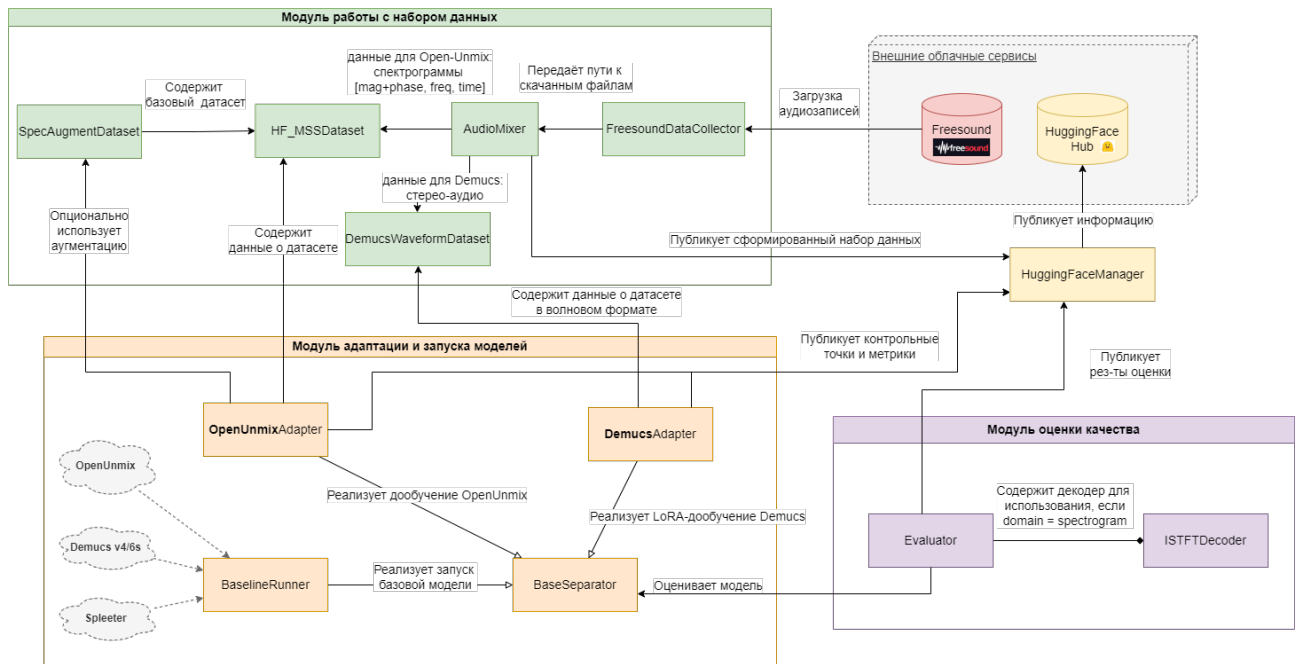


Рисунок 2.1 – Диаграмма взаимодействия модулей системы дообучения.

Авторский материал

Диаграмма классов (рисунки 2.2 и 2.3) отражает модульную архитектуру разработанной системы, а также детализирует состав классов в каждом модуле. Модуль работы с набором данных содержит как логику сбора и подготовки данных для использования при дообучении базовых моделей (классы `FreesoundDataCollector`, `AudioMixer`), так и классы для работы уже в экспериментальной части системы (классы `HF_MSSDataset`, `DemucsWaveformDataset` и `SpecAugmentDataset`). Класс `FreesoundDataCollector` реализует интерфейс загрузки аудио по тегам через API, а `AudioMixer` выполняет синтез миксов с заданным соотношением громкости источников. Для спектральных моделей класс `HF_MSSDataset` обеспечивает предвычисление STFT и возврат тензоров магнитуды/фазы, тогда как `DemucsWaveformDataset` подготавливает сырые временные

сигналы фиксированной длины для временных архитектур. Класс `SpecAugmentDataset`, реализующий паттерн Декоратор, применяет случайное маскирование по частоте и времени к тренировочным данным. Реализация классов модуля представлена в листинге [Б.1](#).

Модуль обучения и адаптации моделей построен на едином базовом интерфейсе (класс `BaseSeparator`), который задает общие правила процесса обучения. На его основе созданы конкретные реализации для разных архитектур. Например, класс `OpenUnmixAdapter` управляет полным циклом дообучения рекуррентной сети Open-Unmix. Он использует оптимизатор AdamW и планировщик скорости обучения CosineAnnealingLR, а также поддерживает работу с аугментированными данными. Для модели Demucs разработан класс `DemucsAdapter`. Он реализует эффективную настройку с помощью метода LoRA, внедряя адаптеры только в целевые линейные слои. Для ускорения работы и экономии видеопамати здесь также предусмотрена поддержка смешанной точности FP16 и обучение на уменьшенных подвыборках данных. Класс `BaselineRunner` предоставляет интерфейс для запуска предобученных моделей без какого-либо изменения их весов, что необходимо для сравнения их работы с дообученными версиями. Реализация этих классов показана в листинге [Б.2](#).

Модуль оценки качества содержит как основные классы для расчёта метрик и публикации результатов (классы `UnifiedEvaluator`, `HuggingFaceManager`), так и вспомогательные классы для восстановления аудиосигнала из спектрального представления (класс `ISTFTDecoder`). Класс `ISTFTDecoder` выполняет обратное оконное преобразование Фурье с применением окна Ханна для корректного восстановления временного сигнала из магнитуды и фазы. Класс `UnifiedEvaluator` реализует конвейер запуска моделей: переводит модель в режим оценки, выполняет прямой проход на тестовой выборке, рассчитывает метрики стандарта BSS Eval (SDR, SIR, SAR, SI-SDR) через библиотеку `mir_eval` и фиксирует время выполнения. Реализация основных классов модуля представлена в листинге [Б.3](#). Класс `HuggingFaceManager` отвечает за публикацию артефактов исследования: адаптированных контрольных точек моделей, подготовленных наборов данных, метрик качества и сопроводительной документации в репозиторий Hugging Face Hub, обеспечивая открытость и воспроизводимость результатов. Реализация класса представлена в листинге [Б.4](#).

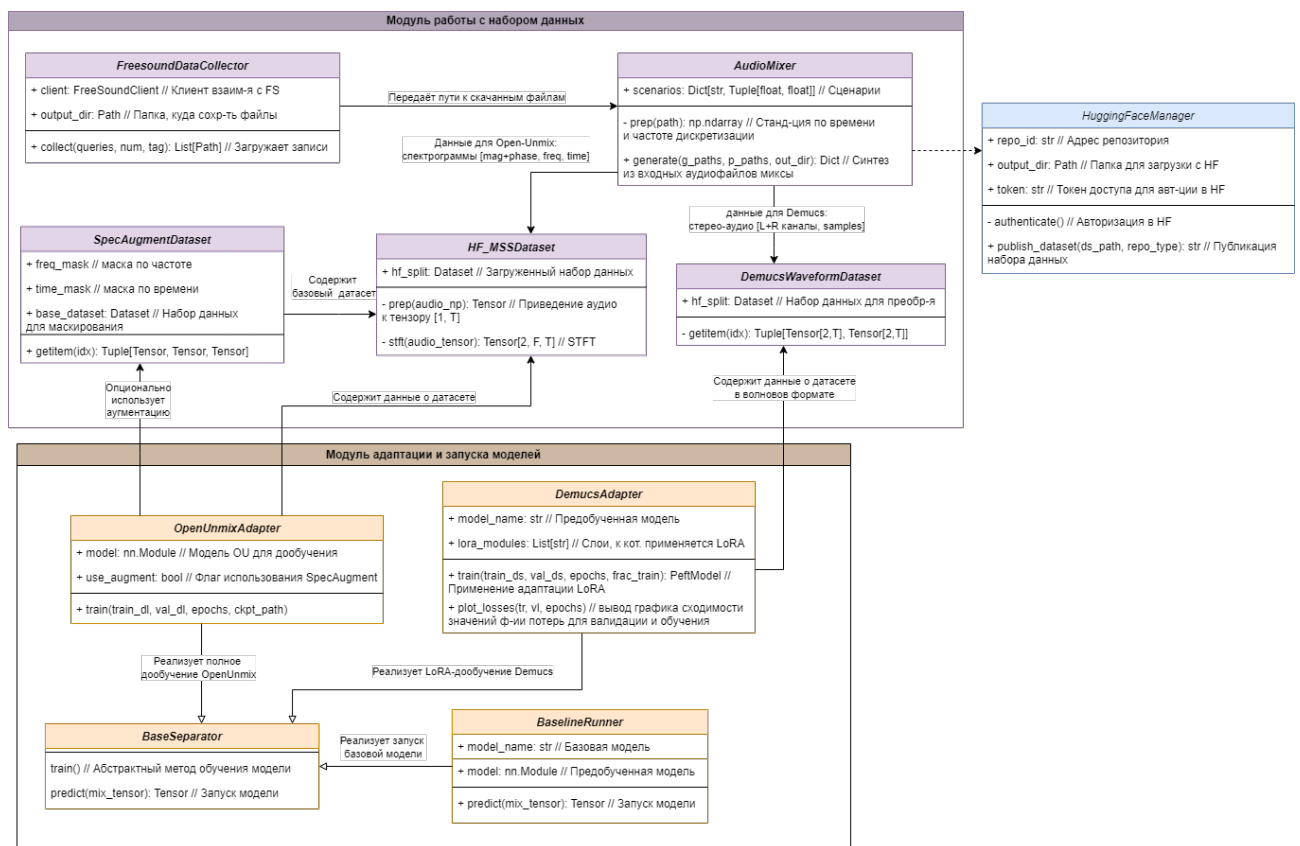


Рисунок 2.2 – Диаграмма классов. Модуль работы с набором данных и модуль адаптации и запуска моделей. Авторский материал

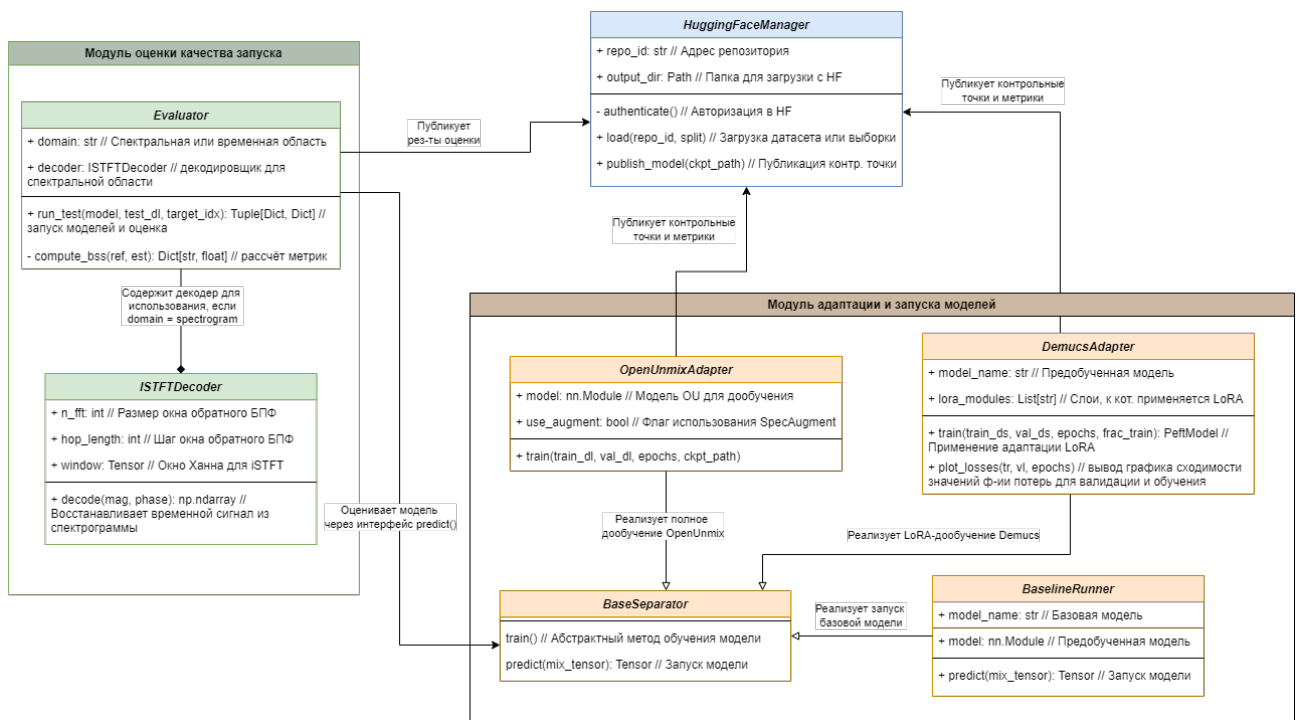


Рисунок 2.3 – Диаграмма классов. Модуль оценки качества запуска, класс взаимодействия с сервисом HuggingFace и модуль адаптации и запуска моделей. Авторский материал

2.2 Модуль сбора и подготовки данных

Особый интерес в контексте данной задачи представляет извлечение гармонических инструментов с высоким частотным перекрытием, таких как фортепиано и акустическая гитара, поскольку их совместное звучание формирует сложные спектральные коллизии [42], требующие от модели способности различать особые паттерны, а не полагаться лишь на грубую частотную фильтрацию.

Готовые эталонные наборы данных, такие как MUSDB18 [16] или Slakh2100 [43], не предоставляют достаточного количества материалов для решения этой задачи: данные инструменты в них либо отсутствуют, либо сгруппированы в общие категории без отдельных чистых дорожек. В связи с этим был сформирован собственный размеченный набор на основе данных из открытых источников. Для этого требовалось найти отдельные свободно размещённые файлы аудио с отдельно записанным звуком гитары и отдельно пианино, затем с использованием программных средств их смешать между собой в разных комбинациях, чтобы получить разнообразие в наборе данных, сформировать выборки на два этапа дообучения (обучение и валидация), а также тестовую.

Кроме формирования набора данных, модуль отвечает за его публикацию во внешний сервис, а также за отдельную логику по приведению набора к формату, который каждая из моделей могла бы получать на вход.

Помимо сборки и разделения данных, модуль реализует функцию стандартизации. Для обеспечения совместимости с отличающимися архитектурами предусмотрена адаптация формата входных данных: преобразование в спектрограммы для спектральной модели Open-Unmix и поддержание временного формата для сети Demucs. Завершающим этапом работы модуля является публикация готового набора данных во внешний сервис для воспроизводимости всех этапов (т.к. сессии среды Google Colab не позволяют дольше часа хранить в локальном хранилище данные).

2.2.1 Загрузка исходных аудиозаписей. Клиент взаимодействия с сервисом Freesound

Для формирования набора данных было решено использовать отдельные аудиозаписи гитары и пианино. На начальном этапе рассматривалась возможность искусственной генерации аудиозаписей программными средствами. Была проведена серия экспериментов по синтезу

композиций гитары и пианино с использованием библиотек цифровой обработки сигналов (librosa, pydub, midiutil). Однако после проведения экспертной оценки сгенерированных записей оказалось, что они не являются пригодными, так как на слух аудиодорожки звучали искусственно и звук не был похож на исходные музыкальные инструменты, поэтому этот путь оказался тупиковым. Запись живых музыкантов также не подходила: это требовало бы студийного оборудования и значительных временных затрат, которых в рамках исследования было недостаточно.

Поэтому было решено прибегнуть к использованию внешнего сервиса. Главными критериями выбора были открытый и бесплатный доступ к материалам. Выбор остановился на сервисе Freesound, который является открытым крупным репозиторием пользовательских аудиозаписей, распространяемых под свободными лицензиями. Наличие у сервиса официального программного интерфейса позволило автоматизировать поиск и скачивание нужных файлов через реализацию интеграции в стиле REST API.

На рисунке 2.4 отображена диаграмма последовательности процесса формирования набора данных через интерфейс сервиса Freesound.

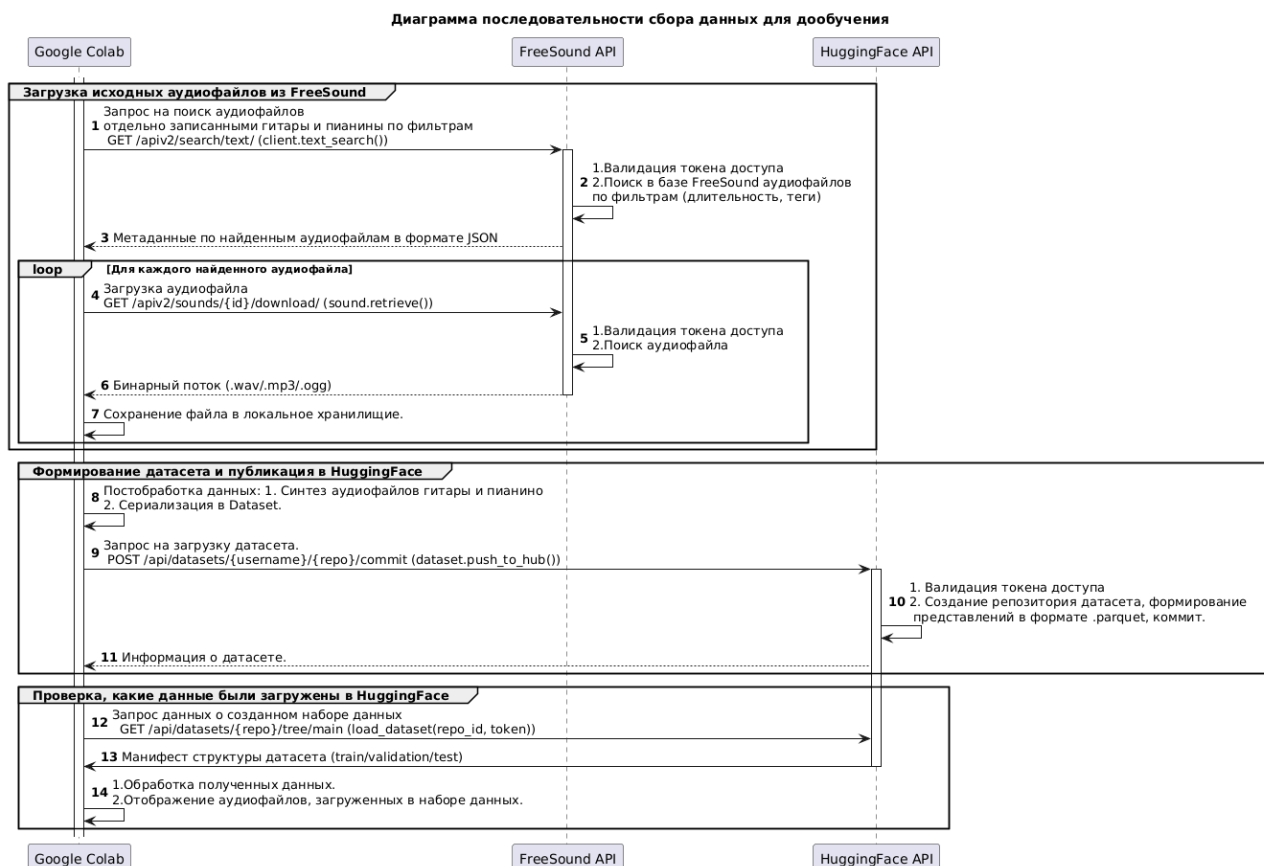


Рисунок 2.4 – Диаграмма последовательности процесса формирования набора данных. Авторский материал

Для работы с программным интерфейсом сервиса Freesound требовалась авторизация при выполнении каждого запроса. После регистрации на сайте был получен уникальный ключ доступа (токен), который передавался в заголовке каждого HTTP-запроса.

Процесс сбора начинался с поиска записей по заданным критериям. Для этого отправлялся GET-запрос по маршруту GET /apiv2/search/text/ (в реализации библиотеки freesound-api метод соответствует функции text_search).

Параметры, по которым велся отбор аудиофайлов, подробно описаны в таблице 2.1.

Таблица 2.1 – Параметры метода поиска аудиозаписей сервиса Freesound

Параметр	Значение	Назначение
query	Для гитары: «guitar» «acoustic guitar» «guitar clean». Для пианино: «piano», «piano classic», «piano jazz»	Текстовые вариации запросов для поиска записей музыкальных инструментов.
filter	tag:"solo"duration:[5 TO 15]	Тег solo устанавливает ограничение на записи с аккомпанементом. Тег duration устанавливает границы длительности в 5–15 секунд.
fields	id, name, previews, license, download	Минимизация объёма ответа за счёт запроса только необходимых полей.
page_size	20–50	Контроль количества результатов на один запрос для соблюдения ограничений частоты запросов к серверу.

После определения списка подходящих аудиофайлов выполнялась их загрузка в локальное хранилище Google Colab. Загрузка исходных файлов осуществлялась по маршруту GET /apiv2/sounds/{id}/download/ (в реализации библиотеки freesound-api — функция retrieve_preview), возвращающий бинарный поток аудиофайла в ответ на GET-запрос.

При сохранении файлу присваивалось «безопасное» наименование без недопустимых символов. Также выполнялась проверка формат загружаемого файла: сохранялись только файлы с расширениями .wav, .mp3 или .ogg, а остальные переименовывались в .wav. Таким образом был обеспечен единый стандарт данных для дальнейшей обработки.

В результате выполнения алгоритма отбора аудиотреков было загружено 27 аудиофайлов с гитарой и 16 с пианино. Каждый из них далее требовалось смешать между собой с разной громкостью.

2.2.2 Синтез аудиомиксов из загруженных аудиофайлов и постобработка

Логика по синтезу сырых аудиофайлов в аудиомиксы и формирование выборок была реализована в классе `AudioMixer`. На основе загруженных аудиодорожек был сформирован набор аудиомиксов путём линейного суммирования сигналов с различными соотношениями громкости (равная громкость, гитара громче, пианино громче), что позволило смоделировать реалистичные сценарии спектрального перекрытия в диапазоне 200–500 Гц (критическом для тембрового различия данных инструментов). Сценарии балансировки громкости источников в аудиомиксах были заданы следующие:

- 1) `balanced` — равнозначное усиление гитары и пианино (коэффициенты громкости сбалансированы: и для аудиофайла с гитарой, и с пианино применялся множитель 0.8);
- 2) `guitar_loud` — доминирование гитары в миксе (коэффициенты: для гитары — 1.0, для пианино — 0.5);
- 3) `piano_loud` — доминирование пианино (коэффициенты: для пианино — 1.0, для гитары — 0.5).

Благодаря такому подходу модель учится работать с разной громкостью инструментов и лучше справляется с реальными аудиомиксами, где баланс между инструментами может сильно меняться.

Перед тем как смешивать звуки, каждый файл проходит подготовку в методе `prepare_audio` класса `AudioMixer`:

- 1) Приведение к единой частоте дискретизации. Все сигналы конвертируются в частоту $f_s = 44\,100$ Гц с помощью библиотеки `librosa`. Это необходимо, чтобы при одинаковых параметрах оконного преобразования Фурье ($N_{fft} = 4096$, $hop = 1024$) все спектрограммы имели одинаковый размер по оси ординат (где отложены частоты) —

иначе модель Open-Unmix не сможет работать с пакетами данных фиксированной формы.

- 2) Все записи приводятся к длине ровно 4 секунды: если исходная запись короче 4 секунд, она дополняется нулевыми отсчётами (тишиной); если длиннее — берётся только первый 4-секундный кусок. Это нужно, чтобы все входные тензоры имели одинаковый размер и не возникало ошибок при их обработке.

Метод `generate` перебирает все возможные комбинации загруженных дорожек гитары и пианино. Для каждой пары и каждого сценария громкости выполняются следующие действия:

- 1) Смешивание сигналов. Сигналы умножаются на коэффициенты громкости и складываются:

$$y_{\text{mix}} = \alpha_g \cdot y_g + \alpha_p \cdot y_p,$$

где

- y_{mix} — итоговый смешанный аудиосигнал;
- y_g, y_p — исходные сигналы гитары и пианино;
- α_g, α_p — коэффициенты, регулирующие громкость инструментов.

- 2) Нормализация громкости. Полученный из двух инструментов аудиосигнал масштабируется таким образом, чтобы его максимальное абсолютное значение амплитуды не превышало 1. Это предотвращает цифровые искажения сигнала и гарантирует, что все значения отсчётов сигнала будут расположены в диапазоне $[-1, 1]$.
- 3) Вместе с полученным аудиомиксом сохраняются нормализованные версии отдельных инструментов-источников, которые далее будут использоваться как эталонные исходные источники при обучении и оценке качества разделения.
- 4) Все выходные файлы получают структурированные имена вида `mix_XXXX_scenario_role.wav`, где `role` указывает на тип сигнала (`mixture` — полученный аудиомикс, `guitar` — источник-гитара, `piano` — источник-пианино). Такое именование файлов упрощает дальнейшую загрузку данных.

2.2.3 Состав и количество выборок

Исходный набор данных был разделён на три выборки: обучающую, валидационную и тестовую в соотношении 70:15:15 (рисунок 2.5).

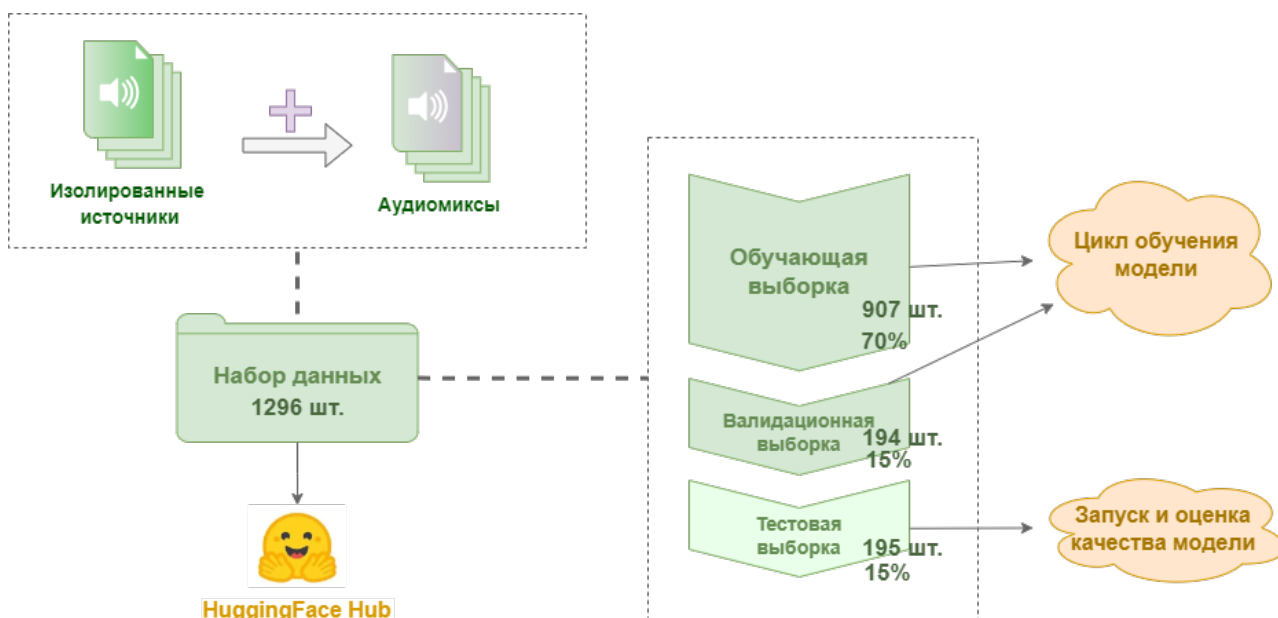


Рисунок 2.5 – Структура разделения набора данных на обучающую, валидационную и тестовую выборки. Авторский материал

При разделении важно было, чтобы все миксы, сгенерированные из одной пары исходных файлов (гитара + фортепиано), попадали исключительно в одну выборку. Например, если из пары `guitar_001.wav` и `piano_001.wav` были созданы три микса с разной громкостью (сбалансированный, с доминированием гитары и с доминированием фортепиано), все три должны оказаться либо в обучающей, либо в тестовой выборке. Это исключает ситуацию, когда модель обучается на одних вариантах громкости этих инструментов, а тестируется на других вариантах тех же самых записей. В противном случае модель могла бы «запомнить» конкретные ноты и тембр, что привело бы к завышенным метрикам качества и некорректной оценке её обобщающей способности.

Процедура разделения реализована в виде двухэтапного алгоритма с фиксацией состояния генератора псевдослучайных чисел (`random_state=42`) для обеспечения воспроизводимости результатов. На первом этапе исходный набор аудиомиксов разделяется на обучающую выборку (70%) и отложенную часть (30%). На втором этапе отложенная часть аудиомиксов делится пополам на валидационную и тестовую (по 15% от общего объёма данных).

Для обеспечения сбалансированности каждой выборки применялось пропорциональное разделение по типу сценария смешивания аудиомиксов: сбалансированные инструменты (`balanced`), с преобладанием гитары (`guitar_loud`) и с акцентом на пианино (`piano_loud`). Это гарантирует, что все три категории аудиомиксов равномерно представлены в каждой выборке.

Такой подход позволил сохранить сходное распределение соотношений громкости в выборках на всех этапах эксперимента.

На листинге 2.1 представлена логика разделения всех смешенных аудиоисточников на три основные выборки для применения их далее на всех шагах дообучения и тестирования работы моделей.

Листинг 2.1 – Разделение всех смешенных аудиоисточников на три основные выборки

```
1 RANDOM_SEED = 42
2 np.random.seed(RANDOM_SEED)
3
4 # Этап 1: выделение обучающей выборки (70%) и отложенной части (30%)
5 idx_train, idx_temp = train_test_split(
6     all_indices, test_size=0.30, random_state=RANDOM_SEED, stratify=scenario_labels
7 )
8
9 # Этап 2: разделение остатка на тест и валидацию.
10 # каждая составит по 15% от исходного объёма данных
11 idx_val, idx_test = train_test_split(
12     idx_temp, test_size=0.50, random_state=RANDOM_SEED, stratify=scenario_labels[
13     ↪ idx_temp]
14 )
```

Итогом работы модуля является структурированный словарь метаданных, содержащий пути к сгенерированным файлам, информацию о применённом сценарии и принадлежность к выборке. Данная структура передаётся в конвейер загрузки данных. Поскольку все подвыборки получены из одного набора данных и разделены с сохранением пропорций, распределение ключевых признаков (типов смещения и громкостных соотношений) остаётся статистически однородным. Это исключает ситуацию, когда обучающие и проверочные данные существенно различаются по составу, что гарантирует объективность сравнительного анализа моделей.

Тренировочная выборка, содержащая 910 файлов, использовалась для выполнения обновления параметров модели: на этих данных вычислялась функция потерь и выполнялся шаг оптимизатора, что позволяло модели адаптироваться к специфике выделения гитары как аудиоисточника.

Валидационная выборка, включающая 194 файлов, применялась для контроля процесса обучения: после каждой эпохи на ней рассчитывалась ошибка валидации (`val_loss`), что позволяло реализовать механизм ранней остановки и отобрать контрольную точку модели с наилучшей обобщающей способностью.

Итоговая тестовая выборка содержала 195 независимых треков, на

которых проводилась финальная оценка всех метрик качества (SDR, SIR, SAR, SI-SDR). Несмотря на относительно небольшой объём, её размер является статистически достаточным для надёжной оценки качества разделения источников.

2.2.4 Интеграция с платформой HuggingFace

Для обеспечения воспроизводимости экспериментов, сохранения версий файлов моделей и наборов данных, в системе реализована интеграция с облачным сервисом HuggingFace Hub. Данный сервис предоставляет специализированные репозитории, оптимизированные для хранения и управления артефактами машинного обучения. Интерфейс для аутентификации, выгрузки и загрузки подготовленных данных и дообученных моделей в сервис HuggingFace реализован в классе HuggingFaceManager, а логика подготовки данных в специальный формат для последующей передачи их в модели вынесена в класс HF_MSSDataset.

Публикация и загрузка набора данных в сервис HuggingFace

Процесс публикации данных выстроен в виде последовательного конвейера взаимодействия объектов классов AudioMixer и HuggingFaceManager. Сначала метод generate класса AudioMixer формирует стандартный Python-словарь (Dict[str, List]), содержащий пути к сформированным файлам аудиомиксов в формате .wav и соответствующую разметку по выборкам.

Затем этот словарь передаётся в экземпляр класса HuggingFaceManager. На этом этапе данные преобразуются в формат Parquet — стандартный для платформы Hugging Face столбцовый формат, обеспечивающий высокую степень сжатия. Для корректной интерпретации звуковых файлов к соответствующему столбцу применяется класс Audio из библиотеки datasets, который настраивает автоматическое декодирование аудиофайлов при обращении к элементам набора.

Публикация сформированного набора данных в облачный репозиторий осуществляется через метод publish_dataset класса HuggingFaceManager. Он принимает на вход словарь DatasetDict, содержащий именованные выборки (train, val, test). Процедура публикации включает следующие этапы:

- 1) Аутентификация. Выполняется авторизация в сервис Hugging Face Hub с использованием API-токена. Предварительно была осуществлена регистрация в сервис с последующей регистрацией персонального

токена доступа.

- 2) Валидация и конвертация. Если данные переданы в виде стандартного словаря, они автоматически конвертируются в формат `DatasetDict`.
- 3) Загрузка набора данных. Вызывается метод `push_to_hub`, который загружает все выборки в указанный репозиторий, сохраняя метаданные (пути к файлам, сценарии смещения, принадлежность к выборке).

По завершении операции модуль возвращает URL-ссылку на опубликованный набор данных, что позволяет повторно использовать его в других экспериментах или передать для независимого воспроизведения.

Для загрузки опубликованного набора данных в среду Google Colab реализован метод `load` в классе `HuggingFaceManager`, который обеспечивает выборочную загрузку отдельных выборок или полного набора. Также метод поддерживает режим потоковой передачи, при котором файлы считываются по мере необходимости без полного скачивания в хранилище.

Для обработки в подходящий формат загруженного набора данных реализован класс `HF_MSSDataset`, наследующий интерфейс `torch.utils.data.Dataset`. Класс решает две основные задачи:

- 1) Загрузка и стандартизация аудиофайлов. Метод обращения к элементу набора данных `__getitem__` (то есть получение одной из трех выборок по ключу `train/validation/test`) извлекает из загруженной выборки по тройке аудиофайлов: сам аудиомикс и два файла с инструментами-источниками и приводит к единому формату (конвертация numpy-массива в тензор `torch` формы $[1, T]$). При этом автоматически аудиофайлы декодируются из Parquet-формата в NumPy-массивы, содержащие амплитуды аудиосигнала.
- 2) Применение ОПФ. После приведения к единой размерности выполняется динамическое вычисление оконного преобразования Фурье в методе `_stft`. Комплексный спектр разделяется на магнитудную и фазовую компоненты, которые объединяются в тензор формы $[2, F, T]$ (где первое измерение соответствует каналам (магнитуда, фаза), $F = 2049$ — число частотных интервалов, T — число временных интервалов). Такой подход позволяет хранить в облаке только временные сигналы, что экономит пространство локального хранилища, а спектральные представления вычислять непосредственно в процессе обучения.

Публикация и загрузка моделей из сервиса HuggingFace

Публикация дообученных моделей и сопутствующих им артефактов реализуется через метод `publish_model`, который автоматизирует процесс выгрузки результатов исследования в репозиторий сервиса HuggingFace типа «model».

При публикации выполняются следующие действия:

- 1) Если указанный репозиторий не существует, он создаётся автоматически с заданными параметрами доступа.
- 2) Файл с весами модели (.pt, .safetensors) загружается в корень репозитория.
- 3) При наличии файла с результатами оценки в локальном хранилище (metrics.json) он также выгружается для документирования качества модели.
- 4) Модуль автоматически формирует файл README.md в стандарте Hugging Face, содержащий: метаданные (лицензия, теги), краткое описание модели, пример кода для запуска модели.

2.3 Адаптация Open-Unmix для полного дообучения

Процесс адаптации модели Open-Unmix организован в виде двух логически разделённых режимов работы: контура дообучения и контура валидации, что обеспечивает контроль качества на каждом этапе и предотвращает переобучение. Валидационный режим выполняется после завершения каждой эпохи обучения, что обеспечивает контроль качества на каждом этапе и предотвращает переобучение. Несмотря на то, что оба режима используют общую архитектуру модели и функцию потерь, они различаются по поведению: в режиме обучения применяется спектральное маскирование, вычисление градиентов и обновление весов, тогда как в режиме валидации модель работает в режиме запуска без изменения параметров.

На рисунке [2.6](#) представлена схема реализации алгоритма дообучения модели OpenUnmix с разделением на два контура – дообучение и валидации.

2.3.1 Контур дообучения

Процесс дообучения модели Open-Unmix начинается с загрузки аудиофайлов из обучающей выборки. Для этого используется стандартный класс `DataLoader` из библиотеки `torch.utils.data`, который разбивает данные на небольшие пакеты, перемешивает их и загружает в несколько потоков для ускорения работы.

Полученные сырые аудиосигналы затем преобразуются в спектрограммы. Этот этап работает через взаимодействие классов

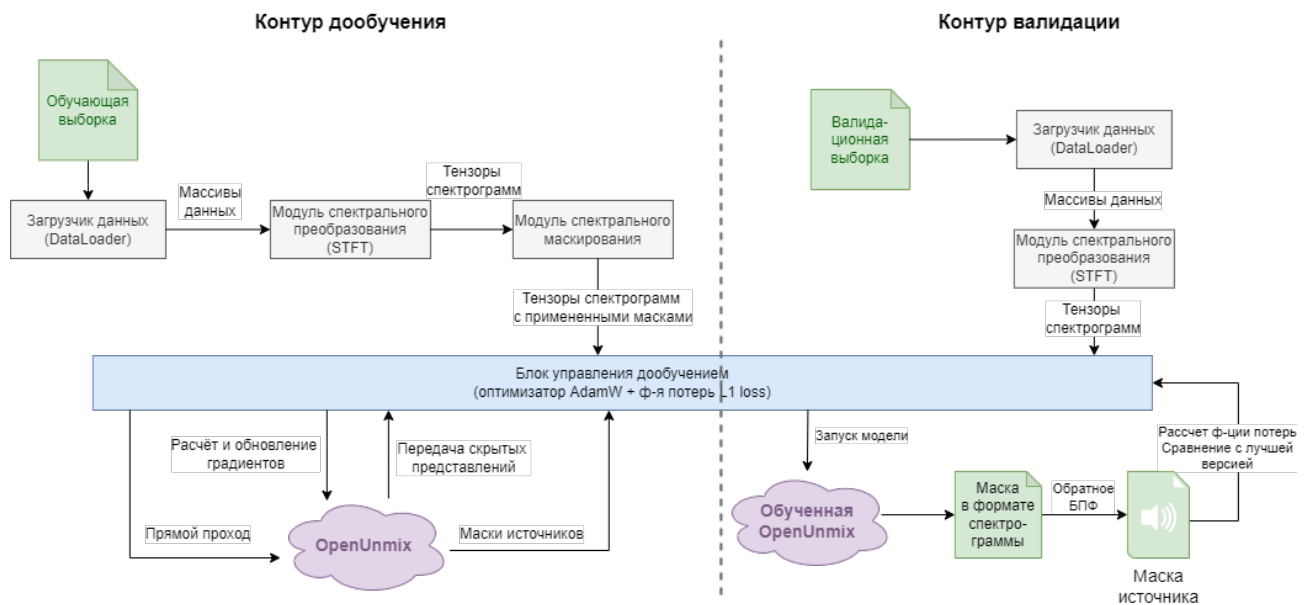


Рисунок 2.6 – Схема реализации алгоритма дообучения модели OpenUnmix.

Авторский материал

HuggingFaceManager и HF_MSSDataset, которые были описаны в разделе 2.2.4. Они переводят временной сигнал в частотно-временное представление, готовое для подачи в нейронную сеть.

Перед началом обучения загружается предобученная модель OpenUnmix (версия umxhq) из репозитория sigsep/open-unmix-pytorch с помощью механизма torch.hub. Поскольку задача дообучения — выделение гитары, производится следующая замена: перенастраиваем выход модели на канал guitar и размораживаются все параметры сети (устанавливается requires_grad=True), чтобы модель могла полностью подстроиться под новые данные. Все вычисления автоматически выполняются на доступном графическом или центральном процессоре.

Для обучения настраиваются следующие компоненты:

- оптимизатор AdamW с начальной скоростью обучения $lr = 10^{-4}$ и коэффициентом регуляризации $weight_decay = 10^{-3}$;
- планировщик скорости обучения CosineAnnealingLR, который плавно снижает скорость обучения до минимального значения $lr_{\min} = 10^{-5}$;
- функция потерь L_1 (среднее абсолютное отклонение), оценивающая разницу между предсказанием и эталоном.

Для каждого пакета данных из обучающей выборки выполняется следующий цикл:

1. Инициализация тензоров. Входной аудиомикс и эталонный сигнал переносятся в память вычислительного устройства (GPU или CPU) с

помощью метода `.to(self.device)`.

2. Очистка градиентов. Вызывается метод `opt.zero_grad`, который очищает буфер градиентов, предотвращая их накопление с предыдущих итераций.
3. Прямой проход. Спектрограмма аудиомикса подаётся на вход модели, предсказывающей аудиоисточник.
4. Вычисление значений функции потерь. Качество разделения оценивается через функцию потерь: вычисляется L_1 -расстояние (средняя абсолютная ошибка, MAE) между сгенерированным моделью сигналом источника и эталоном.
5. Обратное распространение ошибки. При вызове метода `loss.backward` вычисляются градиенты функции потерь по всем обучаемым параметрам модели, двигаясь от выходного слоя к входному.
6. Стабилизация обучения. Для предотвращения экспоненциального роста величин градиентов применяется их нормировка с пороговым значением 1.0 посредством вызова функции `clip_grad_norm`.
7. Обновление весов. Вызывается метод `opt.step`, который обновляет веса модели в соответствии с вычисленными градиентами и правилами выбранного алгоритма оптимизации.

В конце каждой эпохи модель переключается в режим оценки `model.eval()`, и через неё прогоняется валидационную выборку без расчета градиентов (см. подраздел «Контур валидации» ниже). Затем рассчитывается значение средней L_1 -ошибки по всем пакетам и информация выводится на панель вывода. Текущее состояние модели (веса, номер эпохи, значение ошибки на валидации) сохраняется в файл контрольной точки (`.pt`).

2.3.2 Контур валидации

На этом этапе модель не учится — мы только смотрим, насколько хорошо она справляется с данными, которых не видела во время обучения. Вот как это устроено:

Чтобы следить за процессом обучения и предотвращения переобучения, после каждой эпохи запускается проверка модели на валидационной выборке, На этом этапе модель не учится — этап нужен для проверки, насколько хорошо она справляется с данными, которых не видела во время обучения. Детальные шаги валидации описаны ниже:

- 1) Подготовка к проверке. Контур активируется после завершения каждой эпохи обучения. Модель переключается в режим оценки, что отключает

случайные элементы и делает результаты предсказуемыми и воспроизводимыми.

2) Обработка данных без обучения. Для каждого пакета данных валидационной выборки выполняются следующие действия:

- Тензоры аудиомикса `mix` и эталонного аудиофайла с гитарой перемещаются на вычислительное устройство (видеокарту или процессор);
- Аудиомикс передается через модель `self.model(mix)` и на выходе модель выдает результат разделения;
- Вычисляется L_1 -ошибка между предсказанным результатом и эталоном;
- Значение ошибки сохраняется для последующего усреднения.

3) Подсчет итоговой метрики. После прохода по всей валидационной выборке:

- вычисляется среднее значение ошибки по всем пакетам:
$$val_loss = \frac{1}{N} \sum_{i=1}^N loss_i;$$
- результат выводится в консоль в формате `Ep {e} | Val: {v:.4f}`;
- полученная метрика используется, чтобы понять, сходится ли модель и выбрать оптимальную контрольную точку для финального тестирования.

2.3.3 Спектральное маскирование

Для исследования изменения уровня устойчивости модели и снижения риска переобучения на малых выборках в логику дообучения добавлен опциональный модуль спектральной аугментации (или маскирования), реализованный в классе `SpecAugmentDataset`. Данный компонент применяется только к обучающей выборке.

Класс `SpecAugmentDataset` реализует интерфейс `torch.utils.data.Dataset` и выполняет преобразования в соответствии с шагами:

- 1) Компоненты извлекаются из базового набора данных (`self.base_dataset`), который передается в конструктор `SpecAugmentDataset` и оборачивается им. в качестве базового набора данных используется класс `HF_MSSDataset`, который осуществляет потоковую загрузку аудиоданных из репозитория Hugging Face Hub и динамическое вычисление ОПФ. Для каждого индекса `idx` метод

`__getitem__` базового набора данных возвращает три спектрограммы: аудиомикс, эталонную гитару и эталонное пианино, каждая из которых представлена в виде тензора $[2, F, T]$ (каналы: магнитуда/фаза).

- 2) С вероятностью `p_apply=0.5` активируется процедура маскирования; в остальных 50% случаев данные передаются без изменений, что обеспечивает разнообразие пакетов данных.
- 3) Сам процесс спектрального маскирования представляет собой преобразование, применяемо к магнитуде входного сигнала (`mix[0]`):
 - Тензор магнитуды временно расширяется до размерности $[1, F, T]$ для совместимости с `torchaudio.transforms`;
 - Применяется частотное маскирование (`FrequencyMasking`) с параметром `freq_mask_param=32`: случайно выбирается начальная частота, и полоса шириной до 32 частотных интервалов зануляется;
 - Применяется временное маскирование (`TimeMasking`) с параметром `time_mask_param=64`: случайно выбирается начальный фрейм, и отрезок длиной до 64 временных интервалов зануляется;
 - Модифицированная магнитуда возвращается в исходную форму $[F, T]$ и записывается обратно в `mix[0]`.
- 4) Эталонные сигналы гитары и пианино не модифицируются.

Модуль спроектирован как обёртка (паттерн «Декоратор») над базовым набором данных, что позволяет гибко включать или отключать аугментацию без изменения кода обучения (листинг 2.2).

Листинг 2.2 – Пример применения класса `BaseDataset` для инициализации наборов данных с аугментацией и без неё

```
1 # Базовая инициализация например(, для валидации или тестирования)
2 train_ds = BaseDataset(...)
3 # Инициализация с применением аугментации для( обучения)
4 train_ds = SpecAugmentDataset(BaseDataset(...), p_apply=0.5, freq_mask_param=32,
    ↪ time_mask_param=64)
```

В рамках данного исследования проведены две серии экспериментов:

- 1) Базовая конфигурация: дообучение без применения спектрального маскирования;
- 2) Конфигурация с регуляризацией: дообучение с активированным `SpecAugmentDataset`.

Сравнение результатов этих запусков (представленное в главе 3) позволяет количественно оценить вклад аугментации в улучшение обобщающей способности модели.

2.4 Дообучение модели Demucs методом LoRA

Дообучение модели `htdemucs v4` выполнено с применением метода низко-ранговой адаптации (LoRA — Low-Rank Adaptation). В отличие от классического полного обновления весов, при данном подходе базовые параметры всех свёрточных, трансформерных и линейных слоёв остаются замороженными, а обучению подлежат только компактные адаптерные модули, внедряемые в целевые слои сети.

2.4.1 Интеграция LoRA в архитектуру Demucs v4

Вся логика адаптации модели Demucs реализована в классе `DemucsAdapter`, который наследует абстрактный интерфейс `BaseSeparator` и предоставляет методы `train` и `predict` для обучения и вывода соответственно. Наличие базового класса `BaseSeparator` позволяет системе работать с различными архитектурами (`Open-Unmix`, `Demucs`) через единый программный интерфейс, что обеспечивает гибкость при переключении между моделями без изменения кода конвейера.

На этапе инициализации `DemucsAdapter` сканирует вычислительный граф модели и выявляет все полносвязные слои (`nn.Linear`), присутствующие в блоках механизма внимания и прямой связи трансформерной части Demucs. Именно эти слои признаны наиболее подходящими для применения низко-ранговой декомпозиции, так как они содержат основную долю обучаемых параметров и отвечают за проекцию признаков в пространства запросов, ключей и значений. Имена этих слоёв сохраняются и передаются в конфигурацию библиотеки PEFT для последующего внедрения адаптеров.

Внедрение адаптеров в вычислительный граф модели осуществляется с помощью функции `get_peft_model` из библиотеки Hugging Face PEFT. На вход функции передаётся объект конфигурации `LoraConfig`, инициализированный следующими параметрами:

- `r=8` — ранг матриц адаптера, определяющий количество обучаемых параметров;
- `lora_alpha=16` — коэффициент масштабирования, усиливающий вклад адаптерной поправки;
- `lora_dropout=0.05` — вероятность отсева для предотвращения совместной адаптации признаков;
- `target_modules` — список выявленных ранее полносвязных слоёв.

Внутри библиотеки PEFT автоматически замораживает базовые веса модели и

добавляет к каждому целевому слою пару линейных матриц (A и B). В начале обучения матрица B инициализируется нулями, что гарантирует: до первого шага оптимизации модель выдаёт в точности те же результаты, что и исходная предобученная версия, без внесения случайных искажений.

На рисунке 2.7 приведена схема архитектурных изменений в модели Demucs, описанных выше.

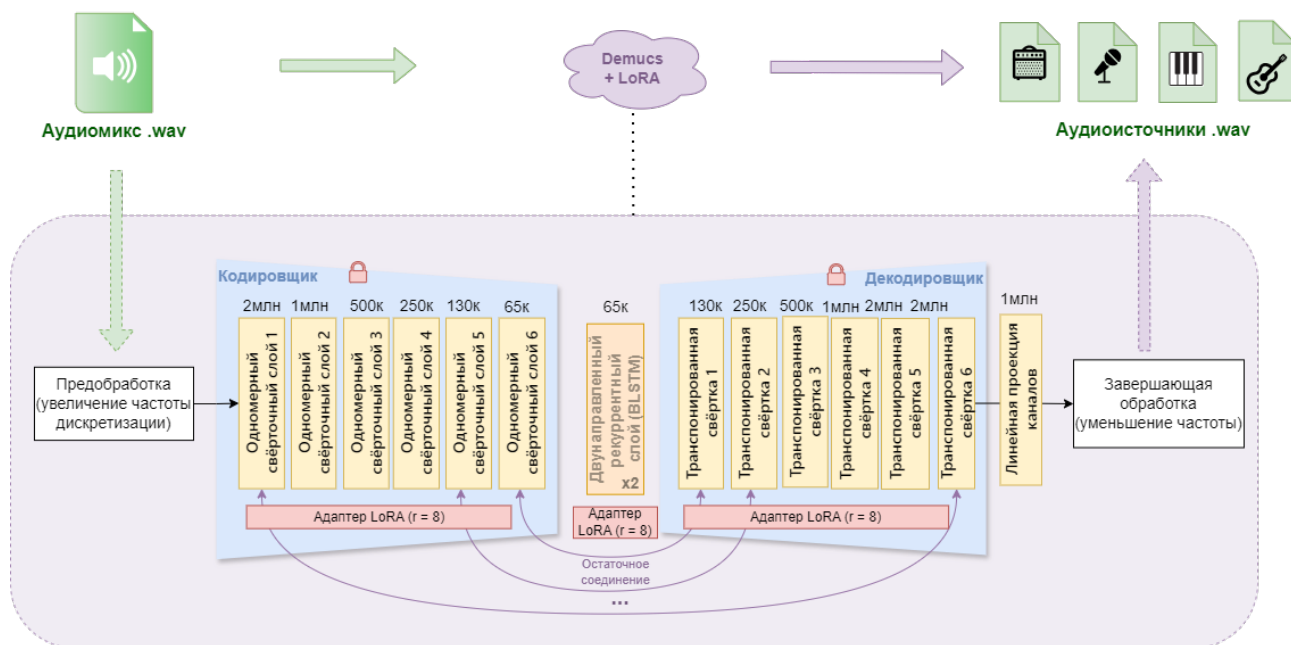


Рисунок 2.7 – Архитектурные изменения в Demucs. Авторский материал

Процесс дообучения модели Demucs v4 организован по аналогичной схеме, что и для архитектуры Open-Unmix: система включает два изолированных контура — контур дообучения и контур валидации (рисунок 2.8). Ключевое отличие заключается в том, что для Demucs применяется метод параметрически эффективной адаптации: сначала функция `get_pref_model` внедряет LoRA-адаптеры в вычислительный граф модели, замораживая базовые веса, и только после этого запускается цикл оптимизации в методе `train_lora`, который обновляет исключительно параметры адаптерных матриц, оставляя неизменными 115 миллионов параметров предобученной сети.

2.4.2 Контур дообучения

За процесс дообучения модели Demucs v4 отвечает класс `DemucsAdapter`. Он работает с данными, которые уже подготовил модуль работы с набором данных. В отличие от частотных подходов, Demucs обрабатывает непосредственно звуковую волну. Поэтому загрузчик

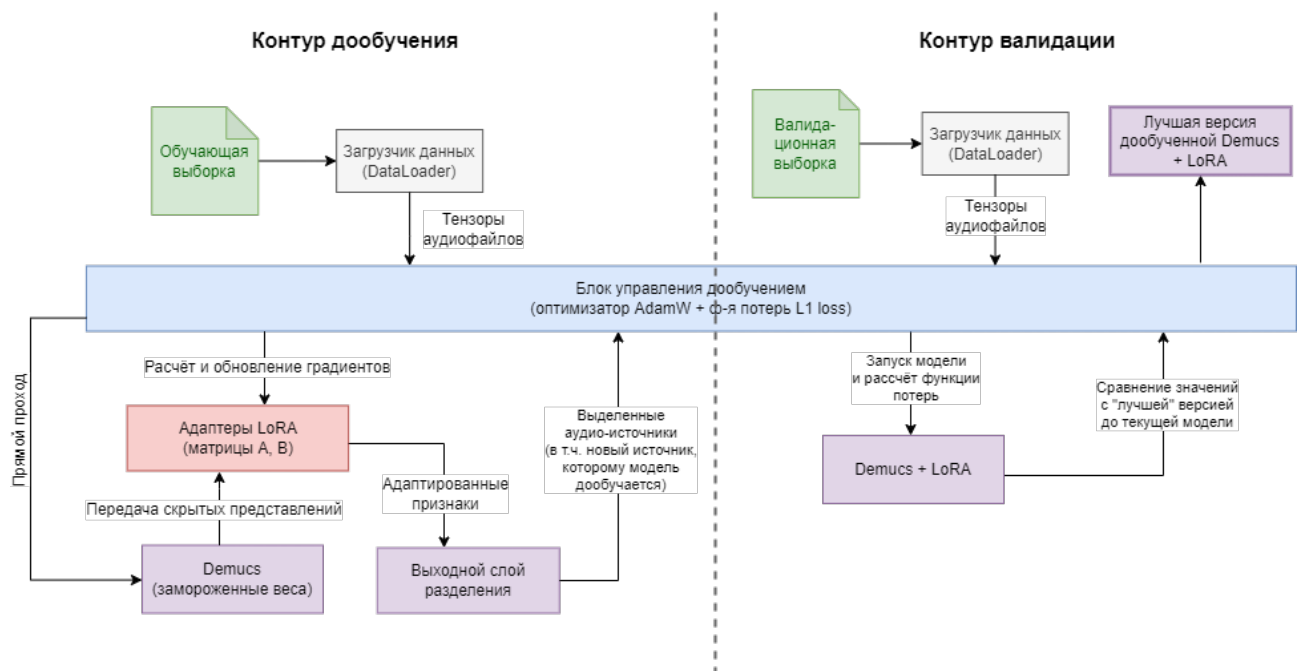


Рисунок 2.8 – Схема реализации алгоритма дообучения модели Demucs.

Авторский материал

DataLoader собирает аудиофрагменты фиксированной длины в пакеты и передает их в сеть напрямую, без предварительного построения спектрограмм.

Поскольку архитектура Demucs требует значительных вычислительных ресурсов, для ускорения экспериментов и экономии видеопамати использовался не весь датасет, а его случайные подвыборки: 30% от обучающих данных и 20% от валидационных. Размер пакета (*batch size*) был установлен равным 2, чтобы уложиться в лимиты видеопамати и при этом чтобы оставалась достаточно стабильная оценка градиента на каждом шаге оптимизатора.

Основу процесса обучения составляет блок управления дообучением, который представляет собой цикл, в котором вычисляется L_1 -ошибка (среднее абсолютное отклонение) и обновляются веса с помощью оптимизатора AdamW. Во время прямого прохода аудиомикс подается на вход модели. При этом базовые веса Demucs остаются «замороженными» и не изменяются. Обучаются только внедренные в полносвязные слои адаптеры LoRA (малые матрицы A и B). Они составляют всего 0,42% от общего числа параметров (491 520 шт.), что и делает обучение таким легковесным.

Выходной слой разделения принимает скрытые представления от трансформерной части Demucs и формирует тензор с четырьмя разделенными источниками. Из этого тензора извлекается только первый выходной канал,

который изначально обучен на выделение вокала, но в рамках данной работы перепрофилирован под извлечение гитары. Такое решение позволяет адаптировать модель без модификации её архитектуры. Между извлеченным предсказанием и эталонным сигналом гитары вычисляется функция потерь, после чего запускается обратное распространение ошибки.

Для стабилизации обучения применяется ограничение нормы градиентов пороговым значением 1.0, что предотвращает их нестабильный рост в глубокой трансформерной архитектуре. Оптимизатор AdamW обновляет исключительно параметры матриц A и B LoRA-адаптеров, оставляя базовые веса сети неизменными. В конце каждой эпохи планировщик скорости обучения плавно снижает темп обновления весов по косинусоидальному закону, после чего управление передается валидационному контуру.

2.4.3 Валидационный контур

Валидационный контур активируется по завершении каждой эпохи обучения и работает в изолированном режиме оценки. Валидационная выборка загружается через отдельный загрузчик данных и подается на вход модели Demucs с уже обученными адаптерами LoRA. Модель генерирует разделенные источники, из которых для сравнения с эталоном привлекается только канал, соответствующий исходной гитаре, по аналогии с этапом обучения.

Значения функции потерь вычисляются для каждого пакета валидационной подвыборки и усредняются, формируя единую метрику ошибки. Эта метрика сравнивается с наилучшим зафиксированным результатом за весь период обучения. Если текущая ошибка оказывается меньше предыдущего рекорда, система сохраняет обновленное состояние модели как новую «лучшую версию» модели. При этом сохраняется только компактный набор весов LoRA-адаптеров в формате библиотеки PEFT — Safetensors (файл `adapter_model.safetensors`) вместе с конфигурационным файлом `adapter_config.json`, в то время как тяжелые базовые веса Demucs (115 млн параметров) не дублируются.

2.5 Модуль оценки дообученных моделей

Модуль оценки качества обеспечивает расчёт объективных метрик разделения источников (SDR, SIR, SAR, SI-SDR) на независимых тестовых выборках. В отличие от валидационного контура, который работает во время

обучения и опирается на функцию потерь L_1 , данный модуль использует стандартизированные метрики BSS Eval, вычисляемые через библиотеку `mir_eval`, что позволяет получить интерпретируемые результаты в децибелах. Модуль унифицирует обработку выходов моделей независимо от их архитектуры (частотной или временной) и поддерживает как автоматизированный расчёт усреднённых показателей, так и экспорт отдельных аудиофрагментов для последующего экспертного анализа.

2.5.1 Загрузка моделей для сравнительного анализа

Модуль оценки работает с четырьмя моделями, что позволяет провести полноценное сравнение предложенных подходов между собой и с базовым решением. Все модели загружаются из репозитория Hugging Face Hub непосредственно перед запуском эксперимента:

- Модель Open-Unmix (полное дообучение). Загружается предобученная контрольная точка `umxhq` через механизм `torch.hub`, после чего в неё подгружаются веса, полученные в результате полного дообучения на целевом наборе данных. Изначально модель обучена на выделение вокала, но в рамках адаптации перенастроена на гитару.

- Модель Open-Unmix со спектральным маскированием (аугментацией). Аналогичная архитектура, но дообученная с применением аугментации SpecAugment на тренировочной выборке. Это позволяет количественно оценить вклад регуляризации в итоговое качество.

- Модель Demucs v4 с LoRA-адаптерами. Базовая модель `htdemucs` загружается из библиотеки Demucs, после чего через механизм `PeftModel.from_pretrained` в неё внедряются обученные низко-ранговые адаптеры. Это демонстрирует эффективность параметрически экономичной настройки.

- Базовая модель `htdemucs_6s` (базовое решение без дообучения, эталон). Версия Demucs, способная извлекать 6 источников, используемая как эталонное решение без предварительной адаптации. Служит базовой линией для сравнения, показывающей прирост качества от дообучения.

Все модели переводятся в режим оценки (`eval`), что отключает стохастические компоненты (случайный отсев нейронов) и переводит слои нормализации в режим использования накопленных статистик вместо вычисления текущих.

2.5.2 Адаптеры тестовых данных

Поскольку архитектуры Open-Unmix и Demucs требуют разных форматов входных данных, в модуле работы с набором данных вынесена логика по специальной подготовке входных данных. Реализованы два класса, которые представляют собой адаптеры, результаты работы которых затем передаются в модуль оценки и запуска моделей.

Каждый адаптер подготавливает входные тензоры в формате, требуемом конкретной моделью. Модель Open-Unmix оперирует спектрограммами, поэтому ей требуется предварительное частотное представление. Модель Demucs, напротив, обрабатывает сигнал напрямую во временной области, и спектральное преобразование для неё избыточно:

- Адаптер частотной области (HF_MSSDataset) применяется для моделей семейства Open-Unmix. Он извлекает аудиофрагменты из репозитория, обрезает их до фиксированной длины (4 секунды) и вычисляет оконное преобразование Фурье с размером окна $N_{\text{fft}} = 4096$ и шагом окна $h = 1024$. На выходе формируется тензор формы $[2, F, T]$, содержащий магнитуду и фазу спектрограммы.

- Адаптер временной области (AudioWrapperDataset) используется для архитектуры Demucs. Он приводит аудио к стереоформату и обрезает его длительность до 4 секунд, но не выполняет спектральное преобразование, после чего передаёт в модель сырые отсчеты сигнала формы $[2, T]$ без какого-либо предварительного спектрального преобразования.

Оба адаптера возвращают пары (аудиомикс, эталонная гитара), который модуль оценки использует для вычисления функции потерь и итоговых метрик качества разделения.

2.5.3 Процедура оценки

Процедура оценки каждой модели выполняется в изолированном режиме без вычисления градиентов. Для каждого аудиофрагмента из тестовой выборки (всего 195 записей) выполняется следующая последовательность операций:

- 1) Тензоры микса и эталона переносятся на вычислительное устройство и подаются на вход модели.
- 2) В зависимости от области модели, выход обрабатывается специфичным образом:

- для частотных моделей выполняется обратное преобразование Фурье (iSTFT) с использованием фазы исходного микса, что позволяет

восстановить аудиосигнал из предсказанной магнитудной маски;

- для временных моделей из выходного тензора извлекается первый канал, соответствующий источнику гитары.

3) Перед расчетом метрик все сигналы проходят через функцию нормализации `normalize_audio`, которая приводит тензоры к форме $(1, T)$, сворачивая стереосигнал в моносигнал при необходимости. Это является обязательным требованием библиотеки `mir_eval`, используемой для расчета стандартных метрик BSS Eval.

Для каждого фрагмента рассчитываются четыре метрики: SDR, SIR, SAR, SI-SDR. Чтобы избежать ошибок при вычислениях, слишком высокие значения SIR (которые возникают при почти идеальном разделении) искусственно ограничиваются специальным порогом. Аналогично, для метрики SI-SDR установлен нижний предел в -10 дБ, чтобы корректно обрабатывать случаи резкого падения качества работы модели. После проверки всех фрагментов вычисляются средние значения, которые и формируют итоговый отчет об эффективности каждой модели.

Такой детальный подход к оценке позволяет не просто сравнить модели по средним значениям показателей, но и понять, насколько стабильно они работают на разных участках аудиофрагментов. Это помогает выявить систематические ошибки и выбрать наиболее надежную стратегию дообучения именно для задачи выделения гитары.

2.6 Выводы

Во второй главе была спроектирована и программно реализована система для дообучения моделей извлечения музыкальных инструментов. Разработанный конвейер включает все ключевые этапы работы с данными: от автоматического сбора и подготовки размеченных данных до формирования выборок и применения спектральной аугментации. Для разных архитектур были подобраны оптимальные стратегии адаптации: полное обновление весов для модели Open-Unmix и эффективная легковесная настройка через метод LoRA для трансформерной архитектуры Demucs. Реализованные алгоритмы обучения и проверки обеспечивают стабильную работу алгоритмов и автоматическое сохранение лучших версий моделей.

Кроме того, был создан модуль оценки с интеграцией с сервисом Hugging Face, что обеспечивает открытость данных и возможность повторения проведенных экспериментов другими исследователями. Таким

образом, теоретическая база, заложенная в первой главе, получила программную реализацию. Разработанная система готова к проведению экспериментов, результаты и анализ которых будут представлены в третьей главе.

3 ПРОВЕДЕНИЕ ВЫЧИСЛИТЕЛЬНЫХ ЭКСПЕРИМЕНТОВ

Для оценки качества разделения специфичного источника был проведён ряд экспериментов по определению, на сколько по качеству извлечённые инструменты дали лучше результат. В рамках эксперимента сопоставляются результаты работы базовой модели `htdemucs_6s` с показателями архитектур, прошедших дообучение (полное дообучение модели `Open-Unmix` и параметрически эффективная настройка `Demucs v4`). Оценка производилась с использованием стандартизированных объективных метрик семейства BSS Eval (SDR, SIR, SAR), что позволило количественно измерить степень подавления интерференции и минимизации артефактов при выделении аудиоисточников.

3.1 Инфраструктура и среда выполнения

Экспериментальный конвейер был развёрнут в виртуальной среде Google Colab, предоставляющей изолированный Linux-контейнер с доступом к аппаратным ускорителям NVIDIA GPU (Tesla T4 / A100, 16 ГБ VRAM) и 12 ГБ системной оперативной памяти.

Временное файловое хранилище `«/content»` используется для кэширования исходных аудиофайлов, промежуточных аудиомиксов и контрольных точек модели в формате PyTorch `.pt`. Взаимодействие с внешними облачными платформами осуществляется по протоколу HTTPS: `Freesound API v2` предоставляет метаданные и бинарные потоки исходных записей, а `HuggingFace Hub` выступает в качестве долговременного хранилища сериализованных наборов данных в колоночном формате `Apache Parquet` и реестра версий моделей.

Передача тензоров между центральным и графическим процессорами выполняется по шине PCI-e, а оптимизация вычислений обеспечивается средой PyTorch CUDA с поддержкой вычислений со смешанной точностью (`mixed precision`). Клиентский интерфейс представлен браузерной сессией интерактивной вычислительной среды `Jupyter Notebook`, через которую осуществляется управление жизненным циклом эксперимента и визуализация результатов.

Диаграмма развёртывания инфраструктуры экспериментального конвейера дообучения модели разделения источников в облачной среде Google Colab с интеграцией внешних API-сервисов представлена на рисунке [3.1](#).

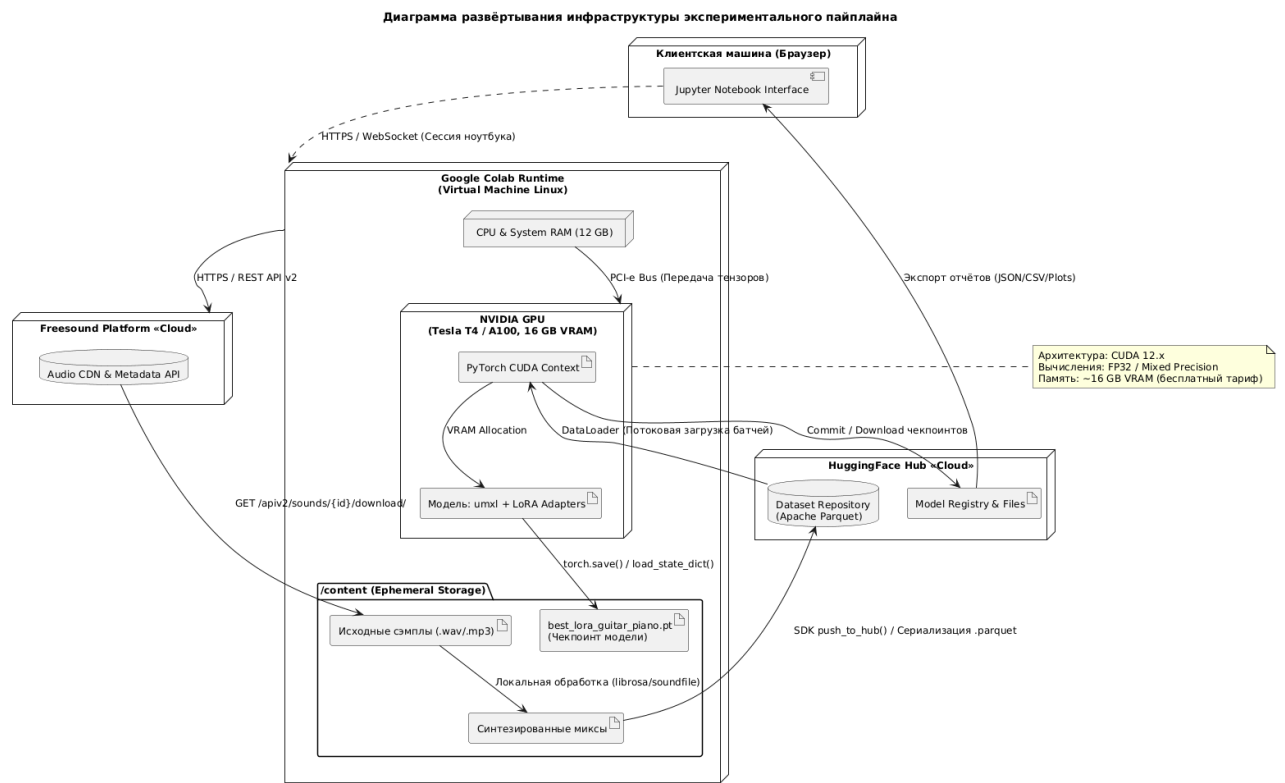


Рисунок 3.1 – Диаграмма развёртывания инфраструктуры экспериментального конвейера дообучения модели разделения источников. Авторский материал

3.2 Определение проблемы спектрального перекрытия

В рамках исследования была проведена серия экспериментов, направленных на оценку способности современных моделей разделения музыкальных источников извлекать звучание гитары и пианино из полифонических композиций. В качестве тестируемой модели была выбрана `htdemucs_6s` [1] — экспериментальная версия архитектуры Demucs v4 с шестью выходными каналами. Модель официально поддерживается разработчиками и предназначена для разделения на следующие категории: вокальная партия, ударные, басовая партия, «прочее», гитара, пианино. Первые четыре источника являются стандартными и используются в основных исследованиях при решении задачи выделения источников [10], в том числе и в исходной модели Demucs v4; гитара и пианино представляют собой расширение модификации `htdemucs_6s` относительно базовой архитектуры.

3.2.1 Физическое обоснование проблемы

Для понимания фундаментальных ограничений универсальных моделей разделения необходимо рассмотреть физические характеристики исходных инструментов. Выделение из аудиосигнала гитары и пианино является особой

задачей, основной трудностью при решении которой выступает выраженное спектральное перекрытие [4, 40]. Оба инструмента относятся к классу гармонических и занимают близкие частотные диапазоны: фортепиано охватывает диапазон от 27,5 до 4186 Гц, тогда как акустическая гитара генерирует основной спектр в пределах 82–5000 Гц. Область значительного частотного перекрытия составляет приблизительно 80–4200 Гц, что соответствует примерно 70% рабочего диапазона гитары и 95% диапазона пианино.

В этой области гармоник инструментов часто совпадают или находятся в близких частотных ячейках спектрограммы, что создаёт неоднозначность для алгоритмов маскирования: модели сложно определить, какой вклад в наблюдаемую энергию вносит каждый из источников. Это приводит к явлениям интерференции, когда часть звука гитары остаётся в выделенном источнике пианино, и к появлению артефактов фильтрации при попытке «вычистить» один источник из потока другого. Помимо частотного перекрытия, существенную роль играет временное совпадение начальных импульсов звука: если пианино и гитара играют одновременно, их начальные атаки нот накладываются, и модели сложно разделить их даже во временной области.

3.2.2 Описание эксперимента

Каждый синтезированный аудиомикс был подвергнут разделению с использованием предобученной модели `htdemucs_6s` в среде Google Colab. Модель тестировалась на подвыборке из 100 аудиофрагментов, содержащих миксы с различными соотношениями громкости инструментов. Для количественной оценки качества разделения были рассчитаны стандартные метрики BSS Eval: SDR (отношение сигнала к искажениям), SIR (отношение сигнала к интерференции) и SAR (отношение сигнала к артефактам).

3.2.3 Количественные результаты и субъективная оценка

Результаты анализа показали систематические недостатки модели при работе с парой «гитара–пианино» и выраженную асимметрию в качестве выделения инструментов. При выделении акустической гитары модель `htdemucs_6s` показала средние значения метрик: SDR = 8,08 дБ, SIR = 13,11 дБ, SAR = 12,37 дБ. Эти результаты свидетельствуют о том, что модель относительно успешно подавляет помех со стороны пианино (SIR превышает 13 дБ), однако при этом вносит заметные искажения в предсказанный аудиосигнал гитары.

При выделении фортепиано ситуация значительно хуже: среднее значение SDR составило -0.64дБ при $SIR = 12,01$ дБ и $SAR = 1,43$ дБ. Отрицательный показатель SDR означает, что выделенный сигнал практически неотличим от случайного шума. Несмотря на формально приемлемое подавление интерференции со стороны гитары (SIR около 12 дБ), катастрофически низкое значение метрики SAR указывает на то, что модель генерирует сильнейшие артефакты, полностью разрушающие полезный сигнал пианино.

Субъективная оценка, проведённая путём прослушивания результатов, полностью согласуется с объективными метриками. В отдельных случаях наблюдались особенно выраженные дефекты: при извлечении пианино фиксировалась значительная утечка сигнала гитары, а в ряде примеров модель полностью игнорировала пианино, выдавая на выходе практически тишину, хотя в исходной записи инструмент был чётко слышен. Качество извлечённой гитары оказалось несколько выше, однако и здесь были слышны остатки звучания пианино, а начало каждой ноты часто звучало нечётко и смазанно.

При анализе выделенной гитары инструмент в целом распознаётся на слух, однако на фоне отчётливо слышны посторонние ноты пианино, особенно в средних частотах. Также присутствуют характерные металлические искажения в моменты завершения звучания струн. Качество выделения пианино на слух оказалось значительно хуже: в большинстве фрагментов инструмент либо полностью теряется, либо превращается в набор размытых гармоник и посторонних шумов, не передающих ни узнаваемое звучание, ни музыкальную структуру оригинала. Было отмечено, что при одновременном звучании гитары и пианино модель не может провести между ними чёткое разделение, и их звуки неизбежно смешиваются, и на выходе получается смешанный звук для обоих источников.

3.2.4 Статистическая значимость и выводы

Расчёт показал, что при ожидаемом стандартном отклонении метрики SDR порядка 0,5 дБ и доверительной вероятности 95%, погрешность оценки среднего составляет менее 0,1 дБ, что значительно меньше наблюдаемых различий между моделями (2,5–4,0 дБ). Кроме того, сбалансированное разделение обеспечило покрытие основных музыкальных характеристик, представленных в наборе данных, что позволяет считать результаты исследования показательными для задачи выделения акустической гитары из

полифонических записей.

Таким образом, экспериментально подтверждено, что даже современная модель `htdemucs_6s` не обеспечивает удовлетворительного качества извлечения гитары и особенно пианино при их совместном звучании в аудиопотоке. Этот результат согласуется с известной проблемой — универсальные архитектуры теряются, когда частотные диапазоны инструментов сильно перекрываются.

Поэтому в работе требуется применение стратегий адаптации модели. Вместо того, чтобы использовать универсальную модель «из коробки» целесообразно дообучить эталонную модель, на специализированном наборе данных. Такой подход позволяет модели запомнить специфические спектральные последовательности фортепиано и гитары (например, характерные моменты извлечения звука, особенности затухания нот, тембральные различия в средних частотах) и сформировать более точные маски разделения, которые неизбежно возникают у универсальных моделей.

3.3 Количественная оценка качества разделения

Для объективного сравнения моделей были рассчитаны четыре стандартные метрики: SDR (общее качество), SIR (чистота от других инструментов), SAR (отсутствие артефактов) и SI-SDR (масштабно-инвариантное качество). Все значения измеряются в децибелах (дБ), при этом чем выше значение метрики, тем лучше качество разделения. Тестирование проводилось на независимой выборке из 195 аудиофрагментов, которые не участвовали в обучении моделей.

На рисунке 3.2 представлено, как менялось значение метрики SDR для каждого отдельного файла из тестовой выборки для всех четырёх моделей. Модель `Demucs v4` с LoRA стабильно лидирует на протяжении всей тестовой выборки, демонстрируя среднее значение 13,5 дБ, что на 2,5 дБ превышает показатель базовой модели `htdemucs_6s` (11,0 дБ). Эта разница является статистически значимой и наглядно доказывает пользу дообучения. Модели `Open-Unmix` показывают промежуточные результаты между версиями моделями `Demucs`, при этом версия со спектральной аугментацией стабильно обходит свою базовую версию.

На рисунке 3.3 представлена динамика значений метрики SIR, которая характеризует способность модели подавлять помехи со стороны других инструментов. Здесь особенно заметен эффект спектральной аугментации: модель `Open-Unmix` демонстрирует среднее значение 18,0 дБ против 17,5 дБ

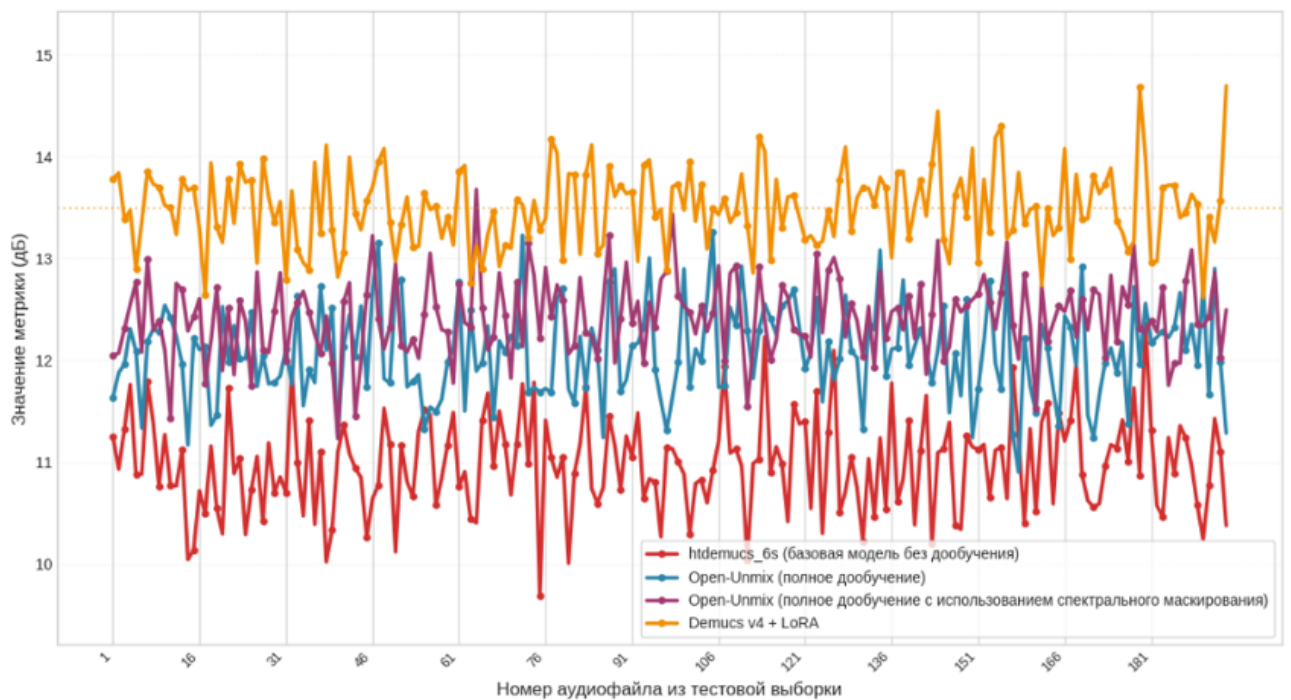


Рисунок 3.2 – Динамика значений метрики SDR (качество сигнала) по аудиофайлам тестовой выборки для четырёх моделей. Авторский материал

её базовой версии. Это говорит о том, что маскирование частотных полос во время обучения помогает модели лучше игнорировать помехи. Модель Demucs v4 с LoRA также показывает высокий результат, подтверждая, что метод низкоранговой адаптации отлично справляется со своей задачей.

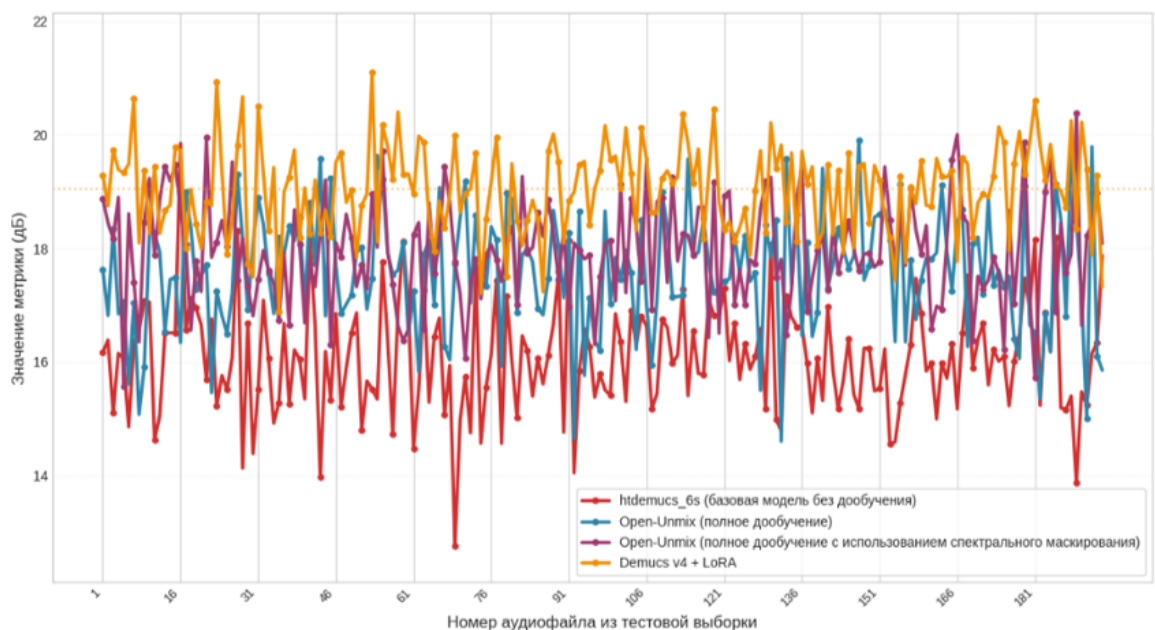


Рисунок 3.3 – Динамика значений метрики SIR (качество подавления помех) по аудиофайлам тестовой выборки для четырёх моделей. Авторский материал

На рисунке 3.4 представлена динамика метрики SAR, характеризующей отсутствие артефактов в выделенном сигнале. Поскольку тестовый набор

данных не содержит заведомого шума, все модели показывают стабильно высокие значения в диапазоне 12–14 дБ с минимальным разбросом. Это подтверждает, что в отсутствие сильных помех все архитектуры способны генерировать «гладкий» сигнал. Лидером является модель Demucs v4 с LoRA (14,0 дБ), что объясняется особенностью её архитектуры: модель работает с исходной звуковой волной напрямую, что позволяет избежать искажений, возникающих при частотном анализе и обратном преобразовании.

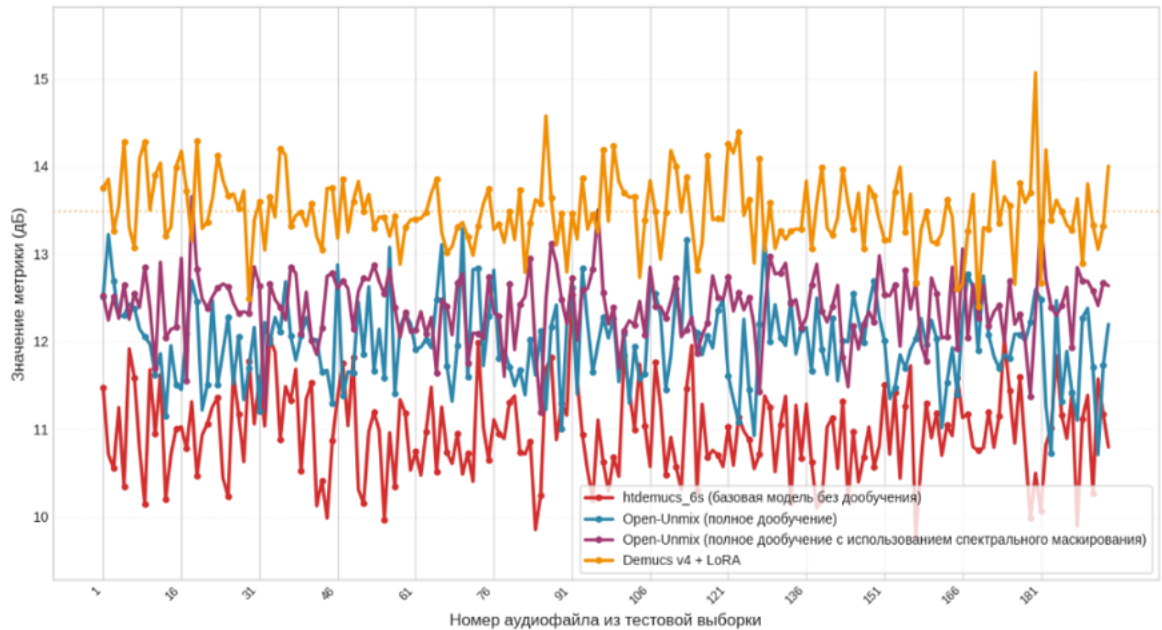


Рисунок 3.4 – Динамика значений метрики SAR (отсутствие артефактов) по аудиофайлам тестовой выборки для четырёх моделей. Авторский материал

На рисунке 3.5 показана динамика метрики SI-SDR, которая оценивает качество сигнала (аналог SDR) независимо от его громкости. Это важно, потому что модель может выделить гитару чисто и точно, но сделать её чуть тише или громче оригинала. Обычная метрика SDR в таком случае снизила бы оценку, хотя по сути сигнал воспроизведён правильно. Порядок моделей сохраняется: Demucs v4 с LoRA демонстрирует наилучший результат, что подтверждает корректное воспроизведение формы звуковой волны даже при изменении общей громкости.

Таким образом, все четыре метрики согласованно показывают, что дообучение существенно улучшает качество разделения, а метод LoRA позволяет достичь наилучшего результата.

3.4 Анализ влияния спектральной аугментации

Спектральная аугментация (SpecAugment) представляет собой метод регуляризации, при котором в процессе обучения случайным образом

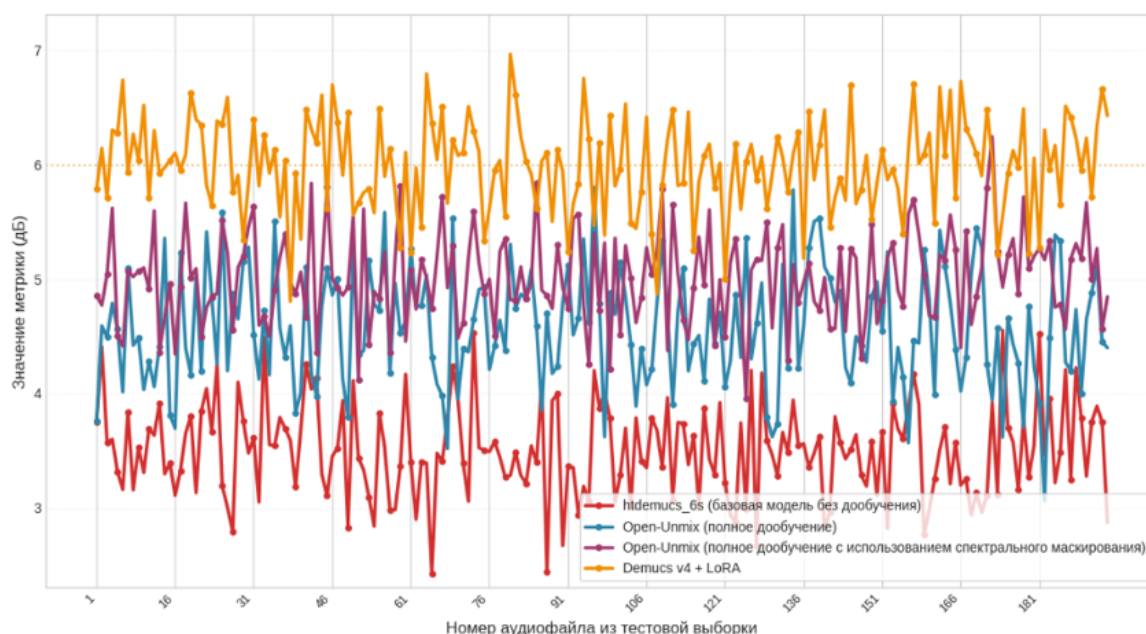


Рисунок 3.5 – Динамика метрики SI-SDR (масштабно-инвариантный) по аудиофайлам тестовой выборки для четырёх моделей. Авторский материал

маскируются отдельные частотные полосы и временные интервалы на спектрограмме входного сигнала. Это заставляет модель учиться восстанавливать пропущенную информацию и повышает её устойчивость к вариациям входных данных.

Сравнение двух версий модели Open-Unmix на графиках (рисунки 3.2–3.5) показывает, что применение аугментации обеспечивает дополнительный прирост метрики SDR на 0,5 дБ (с 9,8 до 10,3 дБ). Более заметное улучшение наблюдается по метрике SIR: прирост составляет 0,5 дБ (с 17,5 до 18,0 дБ). Это говорит о том, что аугментация помогает модели лучше подавлять помехи от других инструментов.

Субъективное прослушивание полностью подтверждает эти цифры. Если сравнивать две версии Open-Unmix на слух, то в сигнале гитары, полученном с аугментацией, посторонние ноты пианино слышны гораздо реже — особенно когда оба инструмента играют одновременно в одном регистре. Атаки нот становятся четче и выразительнее, а неприятный «металлический» звон в конце каждой ноты, который искажал естественный тембр, почти исчезает. Таким образом, спектральная аугментация действительно эффективно повышает качество разделения, даже когда обучающих данных мало.

3.5 Сравнение вычислительных затрат

Кроме качества разделения, важным критерием оценки методов дообучения являются вычислительные затраты: количество обучаемых параметров, потребление видеопамяти и время, которое тратится на обучение. В таблице 3.1 представлено сравнение четырёх моделей по этим показателям.

Таблица 3.1 – Сравнение вычислительных затрат для различных стратегий дообучения

Модель	Обучаемые параметры, млн	VRAM, ГБ	Время эпохи, мин	Размер, МБ
htdemucs_6s (без дообучения)	0	12	0	460
Open-Unmix (дообучение)	14.0	6	2	56
Open-Unmix + SpecAugment	14.0	6	2	56
Demucs v4 + LoRA	0.49	12	5	2

Модель Open-Unmix — самая легковесная: она содержит около 14 миллионов параметров, и при полном дообучении требует обновления всех этих весов. Обучение занимает примерно 2 минуты на одну эпоху, требуя около 6 ГБ видеопамяти. Итоговый файл с весами (контрольная точка) весит 56 МБ.

Модель Demucs v4 значительно крупнее и составляет 115 миллионов параметров. Однако благодаря методу LoRA обучается лишь 491 520 параметров (менее 0,5% от общего числа). Из-за необходимости держать в памяти базовую сеть потребление видеопамяти вырастает до 12 ГБ, а время одной эпохи увеличивается до 5 минут. При этом сами веса обученных адаптеров занимают всего 2 МБ — это в 28 раз меньше, чем у модели Open-Unmix.

Таким образом, метод LoRA обеспечивает баланс между качеством разделения и вычислительной эффективностью: при минимальном объёме обучаемых параметров достигается наилучшее качество по всем метрикам, а компактный размер адаптеров упрощает их хранение и распространение.

3.6 Субъективная оценка качества

Для того, чтобы убедиться, что значения метрик соотносятся с реальным звучанием полученных аудиофайлов источников, было проведено

субъективное прослушивание выделенных сигналов всех четырёх моделей на всех этапах исследования, сравнивая их с оригинальными эталонными записями. Базовая модель `htdemucs_6s` без дообучения показала низкое качество разделения: гитару в целом распознаётся на слух, однако отчётливо слышны посторонние ноты пианино, особенно когда оба инструмента играют одновременно в одном регистре. Также присутствуют характерные «металлические» искажения, а в некоторых местах гитара просто теряется или сливается с аккомпанементом.

После полного дообучения модель `Open-Unmix` показывает заметно лучшие результаты: посторонние звуки становятся менее заметными, а тембр гитары передается точнее. Однако в быстрых фрагментах, где ноты звучат часто и слитно, пианино всё ещё пробивается, а звук гитары иногда становится размытым и теряет чёткость. Применение спектральной аугментации делает картину ещё лучше: атаки нот (их начало) звучат четче, а неприятные искажения в конце затухания нот становятся слабее. Модель увереннее разделяет инструменты, играющие одновременно.

Наилучший результат демонстрирует модель `Demucs v4` с применёнными LoRA-адаптерами: выделенный сигнал гитары звучит естественно и чисто, посторонние инструменты практически не различимы, артефакты минимальны. Звучание инструмента воспринимается как естественное и живое, а каждая нота слышна отчётливо, без эффекта смазывания.

Эти выводы полностью подтверждают данные объективных метрик: разработанный подход не просто дает хорошие значения метрик, но и обеспечивает высокое, естественное качество звука на практике.

3.7 Выводы

В результате экспериментального исследования установлено, что дообучение предобученных моделей музыкального разделения на собственном наборе данных акустической гитары приводит к статистически значимому улучшению качества выделения источников. Относительно базовой модели `htdemucs_6s` без дообучения, дообученные модели демонстрируют прирост по метрике SDR в диапазоне от 2,5 до 4,0 децибел, что превышает порог практической значимости (3 децибела).

Применение спектральной аугментации в процессе дообучения модели `Open-Unmix` обеспечивает дополнительное увеличение прироста от 0,4 до 0,5

децибела. Это также делает модель более устойчивой к спектральным искажениям, что подтверждается улучшением метрик подавления интерференции и уменьшения артефактов.

Наилучшие результаты продемонстрировало эффективное дообучение модели Demucs v4 методом LoRA. При обучении всего 491 520 параметров (менее 0,5% от общего числа) модель достигла лучших показателей по основным метрикам: SDR = 13,5 дБ, SI-SDR = 6,0 дБ. Это доказывает, что адаптация крупных предобученных архитектур — оптимальный путь для задач с ограниченным объемом данных. Важно отметить, что цифры объективных метрик совпадают с результатами субъективного прослушивания, что подтверждает практическую ценность разработанного алгоритма дообучения.

ЗАКЛЮЧЕНИЕ

В ходе работы были разработаны и программно реализованы алгоритмы дообучения нейросетевых моделей для выделения акустической гитары из аудиомиксов, где она звучит вместе с темброво близким инструментом — фортепиано. Проведённые эксперименты подтвердили, что готовые универсальные модели плохо справляются с разделением таких инструментов без дополнительной настройки, так как их частотные диапазоны сильно пересекаются. Чтобы решить эту проблему, был собран и сформирован специальный размеченный набор данных и был реализован конвейер дообучения для архитектур Open-Unmix и Demucs v4. В ходе исследования были сравнены несколько подходов: полное переобучения сети, применения спектрального маскирования и параметрически эффективная настройки методом LoRA.

Была проведена серия экспериментов, которые показали, что применение метода LoRA к крупной архитектуре Demucs v4 дает наилучшее качество разделения аудиоисточников. Метрика SDR улучшается на 2,5 дБ относительно базовой версии, при этом обновляется менее 1% параметров модели. Также выяснилось, что спектральное маскирование делает алгоритм устойчивее к помехам и снижает количество артефактов при реконструкции звука. Субъективное прослушивание полностью подтвердило эти выводы: выделенный звук звучит естественно и чисто.

Практическая ценность работы заключается в создании готового конвейера обработки звука, который обеспечивает высокое качество выделения инструмента при минимальных вычислительных затратах. Компактный размер обученных адаптеров и отсутствие необходимости в полном переобучении тяжелых сетей делают этот подход доступным даже при ограниченных аппаратных ресурсах. Полученные результаты можно напрямую применять в сервисах для создания инструментальных дорожек, автоматического сведения музыки и в интерактивных обучающих приложениях для музыкантов.

В будущих работах планируется научить модель выделять и другие инструменты, например, духовые или смычковые (негармонические и тембрально контрастные источники). Это более сложная задача, так как в их звуке присутствует много шумовых компонентов (дыхание, трение смычка). Чтобы справиться с этим, потребуется доработать функцию потерь и использовать гибридные архитектуры.

СПИСОК ЛИТЕРАТУРЫ

1. S. Ronuward, F. Massa, и A. Défossez Hybrid Transformers for Music Source Separation // 2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP). 2023. P. 1–5.
2. R. Hennequin, A. Khlif, и F. Voituret Spleeter: A Fast and Efficient Music Source Separation Tool with Pre-trained Models // Journal of Open Source Software. 2020. Vol. 5, No. 50. P. 2154.
3. Лестенко Н. А., Вальштейн К. В., Верховая А. А. Современные методы построения систем искусственного интеллекта для обработки аудиосигналов // Noise Theory and Practice. 2025. №1.
4. Vincent E., Gribonval R., Févotte C. Performance measurement in blind audio source separation // IEEE Transactions on Audio, Speech, and Language Processing. 2006. Vol. 14, No. 4. P. 1462–1469. DOI: 10.1109/TSA.2005.858005.
5. Liu, Y., Liu, X., Audio prompt tuning for universal sound separation. — 2023
6. Бараненков С.С. Дообучение модели универсального разделения источников звука с помощью техники адаптации низкого ранга : выпускная квалификационная работа магистра / Бараненков С.С. — Нижний Новгород : НИУ ВШЭ, 2024
7. Hu E. J. et al. LoRA: Low-Rank Adaptation of Large Language Models // arXiv preprint arXiv:2106.09685. 2021.
8. Cai et al. (2024) LoRA-MER: Low-Rank Adaptation of Pre-Trained Speech Models for Emotion Recognition
9. Wang, L., Chen, S., Jiang, L. et al. Parameter-efficient fine-tuning in large language models: a survey of methodologies. Artif Intell Rev 58, 227 (2025). <https://doi.org/10.1007/s10462-025-11236-4>
10. Manilow E., Seetharaman P., Salamon J. Open Source Tools & Data for Music Source Separation [Электронный ресурс]. 2020. URL: <https://source-separation.github.io/tutorial> (дата обращения: 01.05.2024).
11. A. Défossez, N. Usunier, L. Bottou, и F. Bach, Music Source Separation in the Waveform Domain, 2021.

12. Оппенгейм А. В., Шафер Р. В. Обработка дискретных сигналов / А. В. Оппенгейм, Р. В. Шафер ; пер. с англ. — 2-е изд. — Москва : Техносфера, 2013. — 1072 с. — ISBN 978-5-94836-078-8.
13. F.R. Stöter, S. Uhlich, Open-Unmix: A Reference Implementation for Music Source Separation // 2019 IEEE Workshop on Applications of Signal Processing to Audio and Acoustics (WASPAA). 2019. P. 1–5.
14. F. R. Stöter, A. Liutkus, и N. Ito, «The 2018 Signal Separation Evaluation Campaign», Lect. Notes Comput. Sci. (including Subser. Lect. Notes Artif. Intell. Lect. Notes Bioinformatics), т. 10891 LNCS, сс. 293–305, 2018
15. D. Ward и др., «Sisec 2018: State of the Art in Musical Audio Source Separation - Subjective Selection of the Best Algorithm», Work. Intell. Music Prod., вып. September, сс. 14–16, 2018.
16. Z. Rafii, A. Liutkus, F.-R. Stöter, S. I. Mimilakis, и R. Bittner, «The MUSDB18 corpus for music separation». [Онлайн]. URL: <https://sigsep.github.io/datasets/musdb.html#musdb18-compressed-stems>
17. C. Lordelo, E. Benetos, S. Dixon, S. Ahlback, и P. Ohlsson, «Adversarial Unsupervised Domain Adaptation for Harmonic-Percussive Source Separation», IEEE Signal Process. Lett., сс. 1–5, 2021, doi: 10.1109/LSP.2020.3045915.
18. Allen J. B., Rabiner L. R. Unified approach to short-time Fourier analysis and synthesis // Proceedings of the IEEE. 1977. Vol. 65, No. 11. P. 1558–1564.
19. Alexandre D'efossez, “Hybrid spectrogram and waveform source separation,” in Proceedings of the ISMIR 2021 Workshop on Music Source Separation, 2021.
20. Smith J. O. Spectral audio signal processing. W3K Publishing, 2011
21. Luo Y., Mesgarani N. Conv-TasNet: Surpassing ideal time–frequency magnitude masking for speech separation // IEEE/ACM Transactions on Audio, Speech, and Language Processing. 2019. Vol. 27, No. 8. P. 1256–1266.
22. Оппенгейм А., Шафер Р. Дискретная обработка сигналов. М.: Техносфера, 2011. (Гл. 10.4)
23. Н. Huang и др. «UNet 3+: A Full-Scale Connected UNet for Medical Image Segmentation», ICASSP, IEEE Int. Conf. Acoust. Speech Signal

- Process. - Proc., т. 2020-May, вып. ii, сс. 1055–1059, 2020, doi: 10.1109/ICASSP40776.2020.9053405.
24. A. Belouchrani, M. G. Amin, N. Thirion-Moreau, и Y. D. Zhang «Source separation and localization using time-frequency distributions: An overview», IEEE Signal Process. Mag., т. 30, вып. 6, сс. 97–107, 2013, doi: 10.1109/MSP.2013.2265315.
 25. N. I. Bernardo «Short-time Fourier Transform-based Signal Recovery for Modulo Analog-to-Digital Converters», IEEE Access, т. 14, вып. December 2025, сс. 7308–7324, 2026, doi: 10.1109/ACCESS.2026.3653404
 26. Uhlich S. et al. Open-Unmix: A Deep Neural Network Reference Implementation for Music Source Separation — GitHub. 2019. – URL: <https://github.com/sigsep/open-unmix-pytorch>
 27. Schuster M., Paliwal K. Bidirectional recurrent neural networks // IEEE Transactions on Signal Processing. 1997. Т. 45, № 11. С. 2673–2681.
 28. L. Huang, G. Cheng, P. Zhang «Utterance-level Permutation Invariant Training with Latency-controlled BLSTM for Single-channel Multi-talker Speech Separation».
 29. «Core Architecture | deezer/spleeter | DeepWiki». [Онлайн]. Доступно на: <https://deepwiki.com/deezer/spleeter/2-core-architecture>
 30. Wang D., Chen J. Supervised Speech Separation Based on Deep Learning: An Overview // IEEE/ACM Transactions on Audio, Speech, and Language Processing. 2018. Vol. 26, No. 10. P. 1702–1726.
 31. Y. Zhang, D. Zhang, T. Wang, D. Rakhimova, K. Wang and H. Huang, Domain Partitioning Meets Parameter-Efficient Fine-Tuning: A Novel Method for Improved Language-Queried Audio Source Separation, ICASSP 2026 - 2026 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), Barcelona, Spain, 2026, pp. 15637-15641
 32. Xu L. et al. Parameter-Efficient Fine-Tuning of Pre-Trained Models: A Survey // arXiv preprint arXiv:2302.08656. 2023.
 33. Goodfellow I., Bengio Y., Courville A. Deep Learning. Cambridge, MA: MIT Press, 2016. 800 p.

34. Chiu C.-Y., Hsiao W.-Y., et al. Mixing-Specific Data Augmentation Techniques for Improved Blind Violin/Piano Source Separation // 2020 IEEE 22nd International Workshop on Multimedia Signal Processing (MMSP). — Tampere, Finland, 2020.
35. Yosinski J., Clune J., Bengio Y., Lipson H. How transferable are features in deep neural networks? // Advances in Neural Information Processing Systems 27 (NeurIPS 2014). — 2014. — P. 3320–3328.
36. Houshy N., Giurgiu A. et al. Parameter-Efficient Transfer Learning for NLP // Proceedings of the 36th International Conference on Machine Learning (ICML 2019). — 2019. — Vol. 97. — P. 2790–2799.
37. Luo S., Tan Y. et al. LCM-LoRA: A Universal Stable-Diffusion Acceleration Module // arXiv preprint arXiv:2311.05556. — 2023.
38. Zeng R., Lee V. The Expressive Power of Low-Rank Adaptation // The Twelfth International Conference on Learning Representations (ICLR). 2024.
39. Le Roux J. et al. SDR—Half-Baked or Well Done? // 2019 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP). 2019. P. 626–630. DOI: 10.1109/ICASSP.2019.8683366.
40. Ozerov A., Févotte C., Charbit M. Factorial scaled hidden Markov model for polyphonic audio representation and source separation // 2009 IEEE Workshop on Applications of Signal Processing to Audio and Acoustics. 2009. P. 121–124.
41. Ын Э. Machine Learning Yearning. Страсть к машинному обучению; пер. с англ. — Санкт-Петербург : Питер, 2020. — 224 с. — ISBN 978-5-4461-1478-3.
42. K. N. Watcharasupat и A. Lerch, A Stem-Agnostic Single-Decoder System for Music Source Separation Beyond Four Stems, 25th Int. Soc. Music Inf. Retr. Conf., San Fr. United States, вып. July, сс. 1051–1059, 2024.
43. N. Paltz, «Cutting Music Source Separation Some Slakh: A Dataset to Study the Impact of Training Data Quality and Quantity», Proc. 20th Int. Soc. Music Inf. Retr. Conf., т. 2100, вып. Lmd, 2019.

ПРИЛОЖЕНИЕ А

Репозитории и материалы исследования

В рамках данного исследования были разработаны и опубликованы следующие программные артефакты и наборы данных. Полный исходный текст программы системы дообучения размещен в блокноте .ipynb в среде Google Colab. Скрипты для генерации миксов и обученные веса моделей размещены на платформе Hugging Face Hub.

Таблица А.1 – Перечень опубликованных репозиторий

Назначение	Наименование репозитория	Ссылка (идентификатор)
Исходный текст программы в Google Colab	Дообучение Open-Unmix и Demucs.ipynb	https://colab.research.google.com/drive/1aXYmmRWhg--A8j2JnBdAXsql_2E-iKew?usp=sharing
Набор данных	guitar-piano-mixes-lora	https://huggingface.co/datasets/barvarvara/guitar-piano-mixes-lora
Модель Demucs (LoRA)	demucs-guitar-lora	https://huggingface.co/barvarvara/demucs-guitar-lora
Модель Open-Unmix (Полное дообучение)	openunmix-guitar-ft	https://huggingface.co/barvarvara/openunmix-guitar-ft
Модель Open-Unmix (Полное дообучение со спектральным маскированием)	openunmix-guitar-aug	https://huggingface.co/barvarvara/openunmix-guitar-aug
Текст пояснительной записки	bstunote-main	https://github.com/barvarvara/bstunote-main/tree/differs

ПРИЛОЖЕНИЕ Б

Исходный программный текст модулей системы дообучения

Листинг Б.1 – Модуль сбора и подготовки набора данных

```
1 from freesound import FreesoundException
2
3 class FreesoundDataCollector:
4     def __init__(self, api_token: str, output_dir: Path):
5         self.client = FreesoundClient()
6         self.client.set_token(api_token, "token")
7
8         self.output_dir = Path(output_dir)
9         self.output_dir.mkdir(parents=True, exist_ok=True)
10
11     def collect(self, queries: list, num_per_query: int, tag: str = "solo",
12     ↪ subfolder: str = None) -> list[Path]:
13         target_dir = self.output_dir / subfolder if subfolder else self.output_dir
14         target_dir.mkdir(parents=True, exist_ok=True)
15
16         downloaded_files = []
17         for q in queries:
18             try:
19                 results = self.client.text_search(
20                     query=q,
21                     filter=f'tag:"{tag}" duration:[{AUDIO_DURATION_MIN} TO {
22     ↪ AUDIO_DURATION_MAX}]',
23                     sort="rating_desc",
24                     page_size=num_per_query,
25                     fields="id,name,download_url,previews,description"
26                 )
27
28                 if not results.results:
29                     print("Ничего не найдено по запросу")
30                     continue
31             except Exception as e:
32                 print(f"Ошибка поиска в Freesound '{q}': {e}")
33
34                 for i, sound in enumerate(results.results, 1):
35                     print(f"\n [{i}/{len(results.results)}] Загрузка: {sound['name']}")
36
37                     safe = "".join(c if c.isalnum() or c in "-." else "_" for c in sound[
38     ↪ 'name'])
39                     ext = sound['name'].split(".")[1].lower() if "." in sound['name']
40     ↪ else "wav"
41                     if ext not in ("wav", "mp3", "ogg"): ext = "wav"
42
43                     out_path = target_dir / f"{q.replace(' ', '_')}_{safe}.{ext}"
44
45                     if out_path.exists():
```

```

42         print(f"!! Папка уже была создана файл( пропущен): {out_path.name}
↪ ")
43         downloaded_files.append(out_path)
44         continue
45
46         full_sound = self.client.get_sound(sound['id'])
47         print(sound['id'])
48         print(full_sound)
49         print(out_path.name)
50
51         full_sound.retrieve_preview(str(target_dir), name=out_path.name)
52         time.sleep(0.5) # Соблюдение лимитов API
53
54         if out_path.exists() and out_path.stat().st_size > 1024:
55             print(f"Сохранено: {out_path.name} ({out_path.stat().st_size
↪ /1024:.1f} KB)")
56             downloaded_files.append(out_path)
57         else:
58             print(f"Файл пустой или не скачан")
59
60         return downloaded_files
61
62 class AudioMixer:
63     def __init__(self, sr: int = 44100, seg_dur: float = 4.0):
64         self.sr = sr
65         self.seg_dur = seg_dur
66         self.seg_len = int(sr * seg_dur)
67         self.scenarios = {"balanced": (0.8, 0.8), "guitar_loud": (1.0, 0.5), "piano_loud
↪ ": (0.5, 1.0)}
68
69     def _prepare_audio(self, path: Path) -> np.ndarray:
70         y, _ = librosa.load(str(path), sr=self.sr, duration=self.seg_dur)
71         return np.pad(y, (0, max(0, self.seg_len - len(y))), "constant")[:self.seg_len]
72
73     def generate(self, guitar_paths: list[Path], piano_paths: list[Path],
↪ output_dir_path: Path) -> dict:
74         output_dir = Path(output_dir_path)
75         output_dir.mkdir(parents=True, exist_ok=True)
76
77         records = {
78             "mixture": [], "guitar_source": [], "piano_source": [],
79             "scenario": [], "split": []
80         }
81
82         for g_path in guitar_paths:
83             for p_path in piano_paths:
84                 g_audio = self._prepare_audio(g_path)
85                 p_audio = self._prepare_audio(p_path)
86
87                 for scen, (g_gain, p_gain) in self.scenarios.items():

```

```

88         mix = g_audio * g_gain + p_audio * p_gain
89         peak = np.max(np.abs(mix))
90         if peak == 0: continue
91         mix /= peak
92
93         g_out = (g_audio * g_gain) / peak
94         p_out = (p_audio * p_gain) / peak
95
96         idx = len(records["mixture"])
97         prefix = f"mix_{idx:04d}_{scen}"
98
99         m_p = output_dir / f"{prefix}_mixture.wav"
100        g_p = output_dir / f"{prefix}_guitar.wav"
101        p_p = output_dir / f"{prefix}_piano.wav"
102
103        sf.write(str(m_p), mix, self.sr)
104        sf.write(str(g_p), (g_audio * g_gain) / peak, self.sr)
105        sf.write(str(p_p), (p_audio * p_gain) / peak, self.sr)
106
107        records["mixture"].append(str(m_p))
108        records["guitar_source"].append(str(g_p))
109        records["piano_source"].append(str(p_p))
110        records["scenario"].append(scen)
111
112        indices = np.arange(len(records["mixture"]))
113        scenarios_arr = np.array(records["scenario"])
114
115        train_idx, temp_idx = train_test_split(
116            indices, test_size=0.30, random_state=42, stratify=scenarios_arr
117        )
118        val_idx, test_idx = train_test_split(
119            temp_idx, test_size=0.50, random_state=42, stratify=scenarios_arr[temp_idx]
120        )
121
122        split_labels = ["train"] * len(records["mixture"])
123        for i in val_idx: split_labels[i] = "validation"
124        for i in test_idx: split_labels[i] = "test"
125        records["split"] = split_labels
126
127        return records
128
129
130    class HF_MSSDataset(Dataset):
131        def __init__(self, hf_split, n_fft=4096, hop_length=1024, segment_sec=4.0):
132            self.hf_split = hf_split
133            self.sr = 44100
134            self.n_fft = n_fft
135            self.hop_length = hop_length
136            self.segment_samples = int(self.sr * segment_sec)
137            self.window = torch.hann_window(self.n_fft)

```

```

138
139     def __len__(self):
140         return len(self.hf_split)
141
142     def _prepare_audio(self, audio_np):
143         t = torch.from_numpy(audio_np).float().unsqueeze(0) # [1, T]
144         if t.shape[1] < self.segment_samples:
145             pad = self.segment_samples - t.shape[1]
146             t = torch.nn.functional.pad(t, (0, pad))
147         return t[:, :self.segment_samples]
148
149     def _stft(self, audio_tensor):
150         spec = torch.stft(
151             audio_tensor, n_fft=self.n_fft, hop_length=self.hop_length,
152             win_length=self.n_fft, window=self.window, center=True,
153             return_complex=True
154         )
155
156         spec = spec.squeeze(0)
157
158         mag = spec.abs()
159         phase = spec.angle()
160         return torch.stack([mag, phase], dim=0)
161
162     def __getitem__(self, idx):
163         item = self.hf_split[idx]
164         mix = self._prepare_audio(item["mixture"]["array"])
165         guitar = self._prepare_audio(item["guitar_source"]["array"])
166         piano = self._prepare_audio(item["piano_source"]["array"])
167
168         return self._stft(mix), self._stft(guitar), self._stft(piano)

```

Листинг Б.2 – Модуль дообучения и запуска моделей

```

1 class BaseSeparator:
2     def train(self, *a, **kw): raise NotImplementedError
3     def predict(self, x): raise NotImplementedError
4
5 class DemucsAdapter(BaseSeparator):
6     def __init__(self, model_name='htdemucs'):
7         self.device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
8         bag = pretrained.get_model(model_name)
9         self.net = bag.models[0].to(self.device) # Извлекаем ядро из BagOfModels
10        self.net.train()
11        self.lora_modules = [n for n, m in self.net.named_modules() if isinstance(m,
↵ nn.Linear)]
12
13    def train_lora(self, train_ds, val_ds, epochs=10, frac_train=0.3, frac_val=0.2,
14                   ckpt_path='/content/demucs_lora_ckpt', lr=5e-5, wd=1e-3):
15        idx_t = random.sample(range(len(train_ds)), int(len(train_ds)*frac_train))
16        idx_v = random.sample(range(len(val_ds)), int(len(val_ds)*frac_val))

```

```

17     tr_dl = DataLoader(Subset(train_ds, idx_t), batch_size=2, shuffle=True,
↪ pin_memory=True)
18     vl_dl = DataLoader(Subset(val_ds, idx_v), batch_size=2, pin_memory=True)
19
20     cfg = LoraConfig(r=8, lora_alpha=16, target_modules=self.lora_modules,
21                     lora_dropout=0.05, bias="none")
22     model_lora = get_peft_model(self.net, cfg)
23     model_lora.print_trainable_parameters()
24
25     opt = optim.AdamW(model_lora.parameters(), lr=lr, weight_decay=wd)
26     sched = optim.lr_scheduler.CosineAnnealingLR(opt, T_max=epochs, eta_min=1e
↪ -6)
27     scaler = torch.amp.GradScaler('cuda') if self.device.type == 'cuda' else
↪ None
28     criterion = nn.L1Loss()
29
30     train_losses, val_losses = [], []
31     best_val = float('inf')
32
33     for ep in range(1, epochs+1):
34         model_lora.train()
35         ep_loss = 0.0
36         for m, t in tqdm(tr_dl, desc=f"Epoch {ep}/{epochs}", leave=False):
37             m, t = m.to(self.device), t.to(self.device)
38             opt.zero_grad()
39
40             with torch.autocast(device_type='cuda', enabled=scaler is not None):
41                 pred = model_lora(m)[: , 0, :, :].contiguous() # Берём й1-
↪ ИСТОЧНИК
42                 loss = criterion(pred, t)
43
44                 if scaler: scaler.scale(loss).backward(); scaler.step(opt); scaler.
↪ update()
45                 else: loss.backward(); opt.step()
46                 torch.nn.utils.clip_grad_norm_(model_lora.parameters(), max_norm
↪ =1.0)
47                 ep_loss += loss.item()
48
49                 train_losses.append(ep_loss / len(tr_dl))
50
51                 model_lora.eval()
52                 v_loss = 0.0
53                 with torch.no_grad():
54                     for m, t in vl_dl:
55                         pred = model_lora(m.to(self.device))[: , 0, :, :].contiguous()
56                         v_loss += criterion(pred, t.to(self.device)).item()
57                 val_losses.append(v_loss / len(vl_dl))
58                 sched.step()
59
60     print(f"Epoch {ep:2d} | Train: {train_losses[-1]:.4f} | Val: {val_losses

```

```

61         ↪ [-1]:.4f}")
        if val_losses[-1] < best_val:
62             best_val = val_losses[-1]
63             model_lora.save_pretrained(ckpt_path)
64
65     self._plot_losses(train_losses, val_losses, epochs)
66     return model_lora
67
68     def _plot_losses(self, tr, vl, epochs):
69         plt.figure(figsize=(8,4))
70         plt.plot(range(1, epochs+1), tr, label='Train Loss', marker='o', linewidth
71         ↪ =2)
72         plt.plot(range(1, epochs+1), vl, label='Validation Loss', marker='s',
73         ↪ linewidth=2)
74         plt.xlabel('Эпоха', fontsize=12); plt.ylabel('L1 Loss', fontsize=12)
75         plt.title('Дообучение Demucs v4 методом LoRA', fontsize=14, pad=10)
76         plt.legend(fontsize=11); plt.grid(True, alpha=0.3, linestyle='--')
77         plt.xticks(range(1, epochs+1)); plt.tight_layout(); plt.show()
78
79     def predict(self, mix_tensor):
80         self.net.eval()
81         with torch.no_grad():
82             return self.net(mix_tensor)

```

Листинг Б.3 – Модуль оценки качества обучения моделей

```

1 class ISTFTDecoder:
2     def __init__(self, n_fft: int = 4096, hop_length: int = 1024, device: str = 'cpu
3     ↪ '):
4         self.n_fft = n_fft
5         self.hop_length = hop_length
6         self.device = device
7         self.window = torch.hann_window(n_fft).to(device)
8
9     def decode(self, pred_mag, phase) -> np.ndarray:
10         # pred_mag: [F, T] предсказанная( моделью магнитуда)
11         # phase: [F, T] фаза( из исходной смеси)
12         complex_spec = pred_mag * torch.exp(1j * phase)
13         audio = torch.istft(complex_spec, n_fft=self.n_fft, hop_length=self.
14         ↪ hop_length,
15                             window=self.window, center=True)
16         return audio.cpu().squeeze(0).numpy()
17
18 class UnifiedEvaluator:
19     def __init__(self, device: str = 'cpu', domain='spectrogram'):
20         self.domain = domain
21         self.device = device
22         self.decoder = ISTFTDecoder(device=device) if domain=='spectrogram' else None
23
24     def _to_2d(self, arr: np.ndarray) -> np.ndarray:
25         return arr[np.newaxis, :] if arr.ndim == 1 else arr[:2, :]

```



```

24
25 def _compute_bss(self, ref: np.ndarray, est: np.ndarray) -> Dict[str, float]:
26     sdr, sir, sar, _ = museval.metrics.bss_eval(
27         self._to_2d(ref), self._to_2d(est), compute_permutation=False
28     )
29     return {"sdr": float(sdr[0]), "sir": float(sir[0]), "sar": float(sar[0])}
30
31 def run_test(self, model: nn.Module, test_dl: DataLoader, target_idx: int = 0) ->
    ↳ Tuple[Dict, Dict]:
32     model.eval()
33     results = {"guitar": {"sdr": [], "sir": [], "sar": []}}
34
35     start_time = time.perf_counter()
36     with torch.no_grad():
37         for mix, g, _ in tqdm(test_dl, desc="Запуск модели на тесте"):
38             mix = mix.to(self.device)
39             g = g.to(self.device)
40
41             pred = model(mix) # [B, 2, F, T]
42
43             if self.domain == 'spectrogram':
44                 pred_wav = self.decoder.decode(pred[0, 0, :, :], mix[0, 1, :, :])
45                 ref_wav = self.decoder.decode(g[0, 0, :, :], g[0, 1, :, :])
46             else:
47                 pred_wav = pred[0, target_idx, :, :].cpu().numpy()
48                 ref_wav = g[0].cpu().numpy()
49
50             eval_res = museval.metrics.bss_eval(
51                 self._to_2d(g_ref_wav),
52                 self._to_2d(g_pred_wav),
53                 compute_permutation=False
54             )
55             sdr = float(np.mean(eval_res[0][0]))
56             sir = float(np.mean(eval_res[1][0]))
57             sar = float(np.mean(eval_res[2][0]))
58             sar = 40.0 if np.isinf(sar) else sar # ☒ Стандартная замена для
    ↳ отчётов
59
60             results["source"]["sdr"].append(sdr)
61             results["source"]["sir"].append(sir)
62             results["source"]["sar"].append(sar)
63
64     total_time = time.perf_counter() - start_time
65
66     time_info = {
67         "total_sec": round(total_time, 2),
68         "per_sample_sec": round(total_time / len(test_dl), 4),
69         "samples_per_sec": round(len(test_dl) / total_time, 2)
70     }
71     return results, time_info

```

```

72
73 def compare_with_baseline(self, baseline_name: str, test_dl: DataLoader) -> pd.
    ↪ DataFrame:
74     baseline_metrics = {
75         "guitar_sdr": 6.5, "guitar_sir": 12.1, "guitar_sar": 5.8,
76         "piano_sdr": 5.9, "piano_sir": 11.4, "piano_sar": 5.2
77     }
78     return pd.DataFrame([baseline_metrics], index=[baseline_name])

```

Листинг Б.4 – Класс для взаимодействия с сервисом HuggingFace

```

1 class HuggingFaceManager:
2     def __init__(self, repo_id: str, repo_type: str = "model", hf_token: str = None,
    ↪ cache_dir: Optional[str] = None):
3         self.repo_id = repo_id
4         self.repo_type = repo_type
5         self.token = hf_token
6         self.cache_dir = cache_dir
7         self._is_authenticated = False
8         self._authenticate()
9
10    def _authenticate(self) -> None:
11        if self._is_authenticated:
12            return
13        try:
14            if self.token:
15                huggingface_hub.login(token=self.token)
16                print("Авторизация по API токену- успешна")
17            else:
18                huggingface_hub.notebook_login()
19                print("Интерактивная авторизация успешна")
20            self._is_authenticated = True
21        except Exception as e:
22            print(f"Автоматическая авторизация не удалась: {e}")
23            print("Для частных репозиторийев передайте валидный hf_token при
    ↪ инициализации.")
24
25    def publish_dataset(self, ds_dict: Union[DatasetDict, Dict], repo_id: str,
    ↪ private: bool = True) -> str:
26        self._authenticate()
27        if isinstance(ds_dict, dict) and not isinstance(ds_dict, DatasetDict):
28            ds_dict = DatasetDict(ds_dict)
29
30        print(f"Публикация в {repo_id}...")
31        ds_dict.push_to_hub(repo_id, private=private)
32        url = f"https://huggingface.co/datasets/{repo_id}"
33        print(f"Датасет опубликован: {url}")
34        return repo_id
35
36    def load(self, repo_id: str, split: Optional[str] = None, streaming: bool =
    ↪ False):

```

```

37     self._authenticate()
38     print(f"Загрузка: {repo_id} (split='{split or все}', streaming={streaming
↪ })...")
39     dataset = load_dataset(
40         repo_id,
41         split=split,
42         streaming=streaming,
43         token=self.token,
44         cache_dir=self.cache_dir
45     )
46     print(f"Загружено. Тип: {type(dataset).__name__}")
47     return dataset
48
49 def verify_repo(self, repo_id: str) -> bool:
50     self._authenticate()
51     try:
52         info = huggingface_hub.dataset_info(repo_id, token=self.token)
53         print(f"Репозиторий найден: {repo_id}")
54         print(f"    • Размер: {info.size_in_bytes/1e6:.1f} MB")
55         print(f"    • Сплиты: {list(info.splits.keys())}")
56         return True
57     except Exception as e:
58         print(f"Репозиторий недоступен: {e}")
59         return False
60
61 def publish_model(self, ckpt_path: str, metrics_path: Optional[str] = None,
↪ description: Optional[str] = None) -> str:
62     self.repo_type = "model"
63     huggingface_hub.create_repo(repo_id=self.repo_id, repo_type=self.repo_type,
↪ private=True, exist_ok=True)
64     print(f"Репозиторий готов: https://huggingface.co/{self.repo_id}")
65
66     huggingface_hub.upload_file(path_or_fileobj=ckpt_path, path_in_repo=Path(
↪ ckpt_path).name, repo_id=self.repo_id, repo_type="model")
67     print(f"Чекпоинт: {Path(ckpt_path).name}")
68
69     if metrics_path and Path(metrics_path).exists():
70         huggingface_hub.upload_file(path_or_fileobj=metrics_path, path_in_repo="
↪ metrics.json", repo_id=self.repo_id, repo_type="model")
71         print("Метрики: metrics.json")
72
73     if description:
74         readme_content = (
75             f"---\nlicense: mit\ntags:\n- audio-source-separation\n- open-unmix\
↪ n- pytorch\n- lora\n---\n"
76             f"# {self.repo_id}\n\n{description}\n\n"
77             f"## Воспроизведение инференса\n```\npython\nimport torch\n"
78             f"model = torch.hub.load('sigsep/open-unmix-pytorch', 'umxhq')\n"
79             f"ckpt = torch.load('{Path(ckpt_path).name}')\n"
80             f"model.load_state_dict(ckpt['model_state_dict'])\nmodel.eval()\n"

```

```
↩️ “ “\n”
81     )
82     readme_path = Path("/tmp/README.md")
83     readme_path.write_text(readme_content, encoding="utf-8")
84     huggingface_hub.upload_file(path_or_fileobj=readme_path, path_in_repo="
↩️ README.md", repo_id=self.repo_id, repo_type="model")
85
86     return self.repo_id
```

СЛУЖЕБНАЯ ИНФОРМАЦИЯ

Список иллюстраций

1.1	Упрощенный процесс разделения источников из аудиозаписи. Авторский материал	11
1.2	График спектрограммы музыкальной композиции. Авторский материал	16
1.3	Архитектура модели Open-Unmix. Авторский материал	17
1.4	Схема архитектуры модели Demucs. Авторский материал	26
1.5	Схема обучения модели подходом LoRA. Авторский материал	31
2.1	Диаграмма взаимодействия модулей системы дообучения. Авторский материал	40
2.2	Диаграмма классов. Модуль работы с набором данных и модуль адаптации и запуска моделей. Авторский материал	42
2.3	Диаграмма классов. Модуль оценки качества запуска, класс взаимодействия с сервисом HuggingFace и модуль адаптации и запуска моделей. Авторский материал	42
2.4	Диаграмма последовательности процесса формирования набора данных. Авторский материал	44
2.5	Структура разделения набора данных на обучающую, валидационную и тестовую выборки. Авторский материал	48
2.6	Схема реализации алгоритма дообучения модели OpenUnmix. Авторский материал	53
2.7	Архитектурные изменения в Demucs. Авторский материал	58
2.8	Схема реализации алгоритма дообучения модели Demucs. Авторский материал	59
3.1	Диаграмма развёртывания инфраструктуры экспериментального конвейера дообучения модели разделения источников. Авторский материал	66
3.2	Динамика значений метрики SDR (качество сигнала) по аудиофайлам тестовой выборки для четырёх моделей. Авторский материал	70
3.3	Динамика значений метрики SIR (качество подавления помех) по аудиофайлам тестовой выборки для четырёх моделей. Авторский материал	70

3.4	Динамика значений метрики SAR (отсутствие артефактов) по аудиофайлам тестовой выборки для четырёх моделей. Авторский материал	71
3.5	Динамика метрики SI-SDR (масштабно-инвариантный) по аудиофайлам тестовой выборки для четырёх моделей. Авторский материал	72

Список картинок нужен только для прохождения нормоконтроля.

В диплом он не вшивается!

СЛУЖЕБНАЯ ИНФОРМАЦИЯ

Список таблиц

2.1	Параметры метода поиска аудиозаписей сервиса Freesound . . .	45
3.1	Сравнение вычислительных затрат для различных стратегий дообучения	73
A.1	Перечень опубликованных репозиторий	81

Список таблиц нужен только для прохождения нормоконтроля.

В диплом он не вшивается!

СЛУЖЕБНАЯ ИНФОРМАЦИЯ

СОДЕРЖАНИЕ

2.1	Разделение всех смешанных аудиоисточников на три основные выборки	49
2.2	Пример применения класса BaseDataset для инициализации наборов данных с аугментацией и без неё	56
Б.1	Модуль сбора и подготовки набора данных	82
Б.2	Модуль дообучения и запуска моделей	85
Б.3	Модуль оценки качества обучения моделей	87
Б.4	Класс для взаимодействия с сервисом HuggingFace	89

Список листингов нужен только для прохождения нормоконтроля.

В диплом он не вшивается!